Building the JUMHC Portal

A step-by-step walkthrough of every file in your project —
what it does, and why it was written that way

Jadavpur University Mountaineering & Hiking Club • MERN Stack Web Application
Generated as a companion learning document

Table of Contents

0 How This Document Works

1 The Big Picture: What Are We Building, and With What?

2 Project Setup: The Root Folder

3 Backend, Step 1: Dependencies & Secrets

4 Backend, Step 2: Connecting to the Database

5 Backend, Step 3: The Server Entry Point

6 Backend, Step 4: The Data Models (Schemas)

7 Backend, Step 5: Middleware (Auth Guard & File Uploads)

8 Backend, Step 6: The API Routes

9 Backend, Step 7: Seeding Starter Data

10 Frontend, Step 1: Dependencies & Build Tools

11 Frontend, Step 2: The HTML Shell & Entry Point

12 Frontend, Step 3: The API Helper

13 Frontend, Step 4: Global Login State (AuthContext)

14 Frontend, Step 5: Shared Components

15 Frontend, Step 6: The Router (App.jsx)

16 Frontend, Step 7: The Public Pages

17 Frontend, Step 8: The Admin Dashboard (the big one)

18 Running Everything Together

19 Appendix: Small Config Files

0. How This Document Works

This document walks through **every file in your JUMHC-Portal.zip**, in the order you would actually write them if you were building this project from a blank folder. For each file you'll get three things: a short explanation of **what it does**, the **full code exactly as it appears in your project**, and a short note on **why it was written that way** — not just what the syntax means, but the reasoning behind the decision.

The project has two independent halves that talk to each other over the network:

- A **backend** (folder `/backend`) — a Node.js server that stores data in MongoDB and exposes it as a JSON API.
- A **frontend** (folder `/frontend`) — a React website that calls that API and renders it as pages a visitor can click around.

We'll build the backend first (because the frontend has nothing to display until there's an API to call), then the frontend.

1. The Big Picture: What Are We Building, and With What?

The JUMHC Portal is a website for a university mountaineering club. It has two faces:

- A **public site** — Home, About, Activities, Alumni, Gallery, Events, a course-registration form, and a Contact page. Anyone can view these.
- A **private admin dashboard** — where a committee member logs in and manages events, the secretaries' timeline, alumni profiles, gallery photos, and course registrations, without touching a single line of code.

This is a classic **MERN stack** application. Here's what that acronym means and where each piece lives in your project:

M – MongoDB	The database. Stores everything as flexible JSON-like documents (events, registrations, alumni, etc). It runs as its own separate process; your backend just connects to it.
E – Express	A minimal web framework for Node.js. It listens for HTTP requests (like `GET /api/events`) and decides how to respond. Lives entirely in `/backend`.
R – React	A JavaScript library for building the user interface out of reusable components. Lives entirely in `/frontend/src`.
N – Node.js	The JavaScript runtime that lets Express (and all your backend code) run outside a browser, directly on a server/your machine.

Two more tools glue things together on the frontend side:

- **Vite** — a fast dev server and build tool. When you run `npm run dev` in the frontend, Vite is what serves your React app in the browser and instantly refreshes it whenever you save a file.
- **Tailwind CSS** — a utility-first CSS framework. Instead of writing custom CSS files for each component, you style elements by stacking small utility classes directly in the JSX, e.g. `className="px-4 py-2 bg-earth-600 rounded-lg"`.

> **WHY SPLIT FRONTEND AND BACKEND AT ALL?**
>
> It mirrors how real production sites work: the API doesn't care whether it's being called by a website, a mobile app, or another server — it just answers JSON requests. And the two halves can be developed, tested, and even deployed independently. It's more moving parts up front, but it's the standard shape of a modern web app, so it's worth getting comfortable with early.

How a page actually gets its data, in one sentence: your React component runs a `fetch()` to a URL like `http://localhost:5000/api/events`, Express matches that URL to a route handler, the handler asks Mongoose (MongoDB's query library for Node) to fetch documents from MongoDB, and the resulting JSON travels all the way back up to React's `useState`, which triggers a re-render showing the data on screen. We'll trace this exact path again at the very end of this document, once every piece has been introduced.

2. Project Setup: The Root Folder

Before writing any backend or frontend code, this project starts with one small file sitting outside both folders.

`package.json` *(project root)*

This file is not an app by itself — the root folder has no server and no React code. It exists purely as a convenience wrapper so that one person, on one command, can install and run *both* the backend and the frontend at once, instead of having to open two terminals and remember two sets of commands.

```
{
  "name": "jumhc-portal",
  "version": "1.0.0",
  "description": "Jadavpur University Mountaineering & Hiking Club Website Portal",
  "main": "backend/server.js",
  "scripts": {
    "install-all": "npm install --legacy-peer-deps && npm install --prefix backend --legacy-peer-deps && npm install --prefix frontend --legacy-peer-deps",
    "backend": "npm start --prefix backend",
    "backend-dev": "npm run dev --prefix backend",
    "frontend": "npm run dev --prefix frontend",
    "dev": "concurrently \"npm run backend-dev\" \"npm run frontend\"",
    "seed": "npm run seed --prefix backend"
  },
  "dependencies": {},
  "devDependencies": {
    "concurrently": "^8.2.2"
  }
}
```

Walking through the scripts:

- `install-all` — installs the root dependencies, then *also* runs `npm install` inside `/backend` and `/frontend` using `--prefix`, so a single command sets up the whole project.
- `backend` / `backend-dev` — start the backend server (production mode vs. dev mode with auto-restart).
- `frontend` — starts the Vite dev server for React.
- `dev` — the one you'll use most: runs the backend and frontend *at the same time*, in the same terminal.
- `seed` — runs a script that fills the empty database with realistic sample data (events, alumni, etc.) so the site isn't blank the first time you open it.

> **WHY THE <EM>CONCURRENTLY</EM> PACKAGE?**
>
> Node.js normally runs one script per terminal tab and blocks until you stop it. The `dev` script needs to run *two* long-running processes (the Express server and the Vite dev server) side by side. `concurrently` is a small devDependency whose only job is to launch multiple npm scripts in parallel inside one terminal, prefixing their output so you can tell which log line came from which process.

3. Backend, Step 1: Dependencies & Secrets

Everything under /backend

The backend is a plain Node.js project. Like the root, it starts with its own `package.json` declaring which third-party libraries it needs.

backend/package.json

```
{
  "name": "jumhc-backend",
  "version": "1.0.0",
  "description": "Backend API for JUMHC website",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node scripts/seed.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.4.1",
    "multer": "^1.4.5-lts.1",
    "nodemailer": "^6.9.13"
  },
  "devDependencies": {
    "nodemon": "^3.1.3"
  }
}
```

Each dependency exists to solve one specific problem, so it's worth knowing what each one is *for* before you see it used:

- **express** — the web framework that receives HTTP requests and routes them to your code.
- **mongoose** — a library that maps MongoDB documents to JavaScript objects/schemas, so you write `Event.find()` instead of raw database queries.
- **bcryptjs** — hashes passwords before they're stored, so even you (or an attacker) can never see an admin's real password in the database.
- **jsonwebtoken** — creates and verifies the login tokens (JWTs) that prove a request is coming from a logged-in admin.
- **cors** — without it, a browser would block your React app (running on port 5173) from calling your API (running on port 5000), because they're technically different "origins". This middleware tells the browser that's allowed.
- **dotenv** — loads secret configuration (database URL, JWT secret, etc.) from a `.env` file into `process.env`, so secrets never have to be hard-coded into your source files.
- **multer** — handles file uploads (used for alumni photos and gallery images), since Express can't parse uploaded files on its own.
- **nodemailer** — sends the registration-confirmation emails.
- **nodemon** (dev only) — watches your backend files and automatically restarts the server whenever you save a change, so you don't have to stop/start it by hand.

backend/.env.example ***your real secrets live in backend/.env, which is never committed to version control***

Before writing any server code, you need a place to put configuration that (a) changes between your machine and a real deployed server, and (b) should never be visible in your source code — like database URLs and signing keys. That's what a `.env` file is for. The project ships an `.env.example` as a template — you copy it to a real `.env` and fill in your own values:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/jumhc
JWT_SECRET=some_strong_jwt_secret_key_here
ADMIN_EMAIL=admin@jumhc.org
FRONTEND_URL=http://localhost:5173

# Nodemailer SMTP Configuration (Optional: fallback to console log if blank)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password
```

- **PORT** — which port the Express server listens on.
- **MONGO_URI** — the connection string for your MongoDB database (local or cloud).
- **JWT_SECRET** — a secret string used to cryptographically sign login tokens. If someone else knew this value, they could forge valid admin logins — hence keeping it in `.env` and out of Git.
- **ADMIN_EMAIL** — where new-registration notification emails get sent.
- **FRONTEND_URL** — documents which URL the React app runs on during development.
- **SMTP_*** — mail server credentials for actually sending emails via Nodemailer. If left blank, the code falls back to just logging the email

contents to the console instead of failing — handy for local development without a real mail account.

WHY KEEP A REAL .ENV OUT OF THE PDF

Your project's actual `backend/.env` file contains a real (if only locally-used) value for `JWT_SECRET`. This document shows `.env.example` instead, which is the placeholder template meant to be shared — the same reason your project's `.gitignore` excludes the real `.env` from version control.

4. Backend, Step 2: Connecting to the Database

`backend/config/db.js`

With secrets in place, the next thing the backend needs is a way to actually connect to MongoDB. This tiny module's only job is exporting one function that does that.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://localhost:27017/jumhc');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

`connectDB` is an `async` function that calls `mongoose.connect(...)` with the connection string from `.env` (falling back to a local default if it's missing). If the connection succeeds, it logs which host it connected to. If it fails, it logs the error and calls `process.exit(1)`.

WHY CRASH THE WHOLE SERVER ON A FAILED DB CONNECTION?

A web app that can't reach its database can't do anything useful anyway — every route in this project reads or writes MongoDB. Rather than let the server start up and then throw confusing errors on the first real request, it fails loudly and immediately, which makes the actual problem (bad connection string, database not running, etc.) obvious right away instead of showing up later as a mysterious 500 error.

5. Backend, Step 3: The Server Entry Point

`backend/server.js` *the file you actually run with "node server.js"*

This is where the Express app itself is assembled. Every other backend file (models, routes, middleware) exists to be plugged into this one.

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

// Load environment variables
dotenv.config();

// Connect to Database
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve static upload folder
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/secretaries', require('./routes/secretaries'));
app.use('/api/alumni', require('./routes/alumni'));
app.use('/api/events', require('./routes/events'));
app.use('/api/registrations', require('./routes/registrations'));
app.use('/api/gallery', require('./routes/gallery'));

// Base Route
app.get('/', (req, res) => {
  res.send('JUMHC API is running successfully');
});

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: err.message || 'Something broke on the server!' });
});

// Start Server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT} in ${process.env.NODE_ENV || 'development'} mode`);
});
```

Reading it top to bottom, in the order things actually happen when the server starts:

- `dotenv.config()` runs first, so every `process.env.SOMETHING` used anywhere else in the app is already populated by the time it's needed.
- `connectDB()` is called next, kicking off the MongoDB connection in the background.
- `express()` creates the actual app object.
- Three `app.use(...)` calls register global middleware: `cors()` (allow cross-origin requests from the React app), `express.json()` (parse incoming JSON request bodies into `req.body`), and `express.urlencoded()` (parse traditional form submissions).
- `app.use('/uploads', express.static(...))` makes the `backend/uploads` folder (where Multer saves photos) directly browsable over HTTP, so an uploaded alumni photo becomes reachable at a URL like `http://localhost:5000/uploads/photo-12345.jpg`.
- Six `app.use('/api/…', require('./routes/…'))` lines mount each route file under its own URL prefix — this is how a request to `/api/events/123` ends up inside `routes/events.js`.
- A catch-all error-handling middleware (the one with *four* parameters — `(err, req, res, next)`) sits at the bottom, so if any route handler throws, the request still gets a clean JSON error response instead of the connection just hanging.
- Finally, `app.listen(PORT, …)` starts the server listening for real traffic.

> **WHY DOES MIDDLEWARE ORDER MATTER HERE?**
>
> Express processes `app.use()` calls top-to-bottom for every incoming request. `express.json()` has to run *before* any route that reads `req.body`, or `req.body` would simply be undefined. Likewise, the error handler has to be registered *last*, because Express only routes a request into it if every route above it either finished or explicitly called `next(err)`.

6. Backend, Step 4: The Data Models (Schemas)

Before you can write a single API route, Mongoose needs to know the *shape* of each type of document you'll store — what fields it has, what type each field is, which ones are required, and any default values. That's what a **schema** is, and each model file below defines exactly one.

> **WHY BOTHER WITH SCHEMAS &MDASH; ISN'T MONGODB "SCHEMALESS"?**
>
> MongoDB itself will happily store any shape of document you throw at it. Mongoose schemas exist on top of that to give *your application code* some safety: they validate incoming data before it's saved (so a missing required field is rejected with a clear error instead of silently saving broken data), they auto-convert types, and they let editors autocomplete field names. It's a deliberate trade of MongoDB's raw flexibility for fewer bugs.

backend/models/AdminUser.js

Stores the login credentials for committee members who can access the admin dashboard.

```
const mongoose = require('mongoose');

const AdminUserSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  passwordHash: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ['Super Admin', 'Editor'],
    default: 'Editor'
  }
}, { timestamps: true });

module.exports = mongoose.model('AdminUser', AdminUserSchema);
```

- `email` is marked `unique` so two admins can never share a login, and `lowercase: true` normalizes it automatically so "Admin@JUMHC.org" and "admin@jumhc.org" are treated as the same account.
- Notice the field is called `passwordHash`, not `password` — the model never stores a plain-text password at all; only the bcrypt-hashed version ever reaches this schema.
- `role` is restricted to an `enum` of exactly two values, so it's impossible to accidentally save a typo'd role like "superadmin".
- `{ timestamps: true }` is a one-line Mongoose option that automatically adds and maintains `createdAt` / `updatedAt` fields on every document — used across every model in this project.

backend/models/Secretary.js

One entry per club secretary's term of office — this powers the timeline shown on the About page.

```
const mongoose = require('mongoose');

const SecretarySchema = new mongoose.Schema({
  yearStart: {
    type: Number,
    required: true
  },
  yearEnd: {
    type: Number,
    required: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  note: {
    type: String,
    trim: true
  }
}, { timestamps: true });

// Ensure years are sorted properly by default
SecretarySchema.index({ yearStart: -1 });

module.exports = mongoose.model('Secretary', SecretarySchema);
```

The one thing worth calling out here is `SecretarySchema.index({ yearStart: -1 })` — this tells MongoDB to maintain a sorted index on `yearStart` in descending order, since the timeline is almost always queried "most recent secretary first". It's a small performance hint, not something that changes behavior.

backend/models/Alumni.js

Profile data for alumni and supporters shown on the Alumni page.

```
const mongoose = require('mongoose');

const AlumniSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  batchYear: {
    type: Number,
    required: true
  },
  roleHistory: {
    type: String,
    required: true,
    trim: true
  },
  currentContribution: {
    type: String,
    trim: true
  },
  photoUrl: {
    type: String,
    trim: true
  },
  bio: {
    type: String,
    trim: true
  },
  isActiveInstructor: {
    type: Boolean,
    default: false
  }
}, { timestamps: true });

module.exports = mongoose.model('Alumni', AlumniSchema);
```

Most fields are self-explanatory strings, but `isActiveInstructor` is a boolean flag (defaulting to `false`) that the Alumni page later uses to show a "Star" badge and to power a filter checkbox.

backend/models/Event.js

Represents a course, expedition, competition, or other bookable activity.

```
const mongoose = require('mongoose');

const EventSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  type: {
    type: String,
    required: true,
    enum: ['expedition', 'competition', 'course', 'other']
  },
  date: {
    type: Date,
    required: true
  },
  description: {
    type: String,
    required: true,
    trim: true
  },
  location: {
    type: String,
    required: true,
    trim: true
  },
  registrationOpen: {
    type: Boolean,
    default: true
  },
  capacity: {
    type: Number,
```

```
    default: null // null means unlimited
  },
  registrationsCount: {
    type: Number,
    default: 0
  },
  deadline: {
    type: Date,
    default: null
  }
}, { timestamps: true });

module.exports = mongoose.model('Event', EventSchema);
```

- `type` is restricted to an enum of the four categories the frontend knows how to badge/style.
- `capacity` defaults to `null`, which the rest of the app treats as a special "unlimited seats" value — you'll see this checked explicitly (`capacity !== null`) throughout the registration logic.
- `registrationsCount` is a running counter maintained by the server (never sent by a client) — it's incremented every time someone successfully registers, which is how the "12 / 40 seats filled" progress bars on the Events page get their numbers without having to count Registration documents on every page load.

backend/models/GalleryItem.js

A single photo (or scanned flyer) in the archive/gallery.

```
const mongoose = require('mongoose');

const GalleryItemSchema = new mongoose.Schema({
  imageUrl: {
    type: String,
    required: true
  },
  year: {
    type: Number,
    required: true
  },
  eventTag: {
    type: String,
    required: true,
    trim: true
  },
  caption: {
    type: String,
    trim: true
  }
}, { timestamps: true });

module.exports = mongoose.model('GalleryItem', GalleryItemSchema);
```

backend/models/Registration.js

One signup for one event — created whenever someone submits the registration form.

```
const mongoose = require('mongoose');

const RegistrationSchema = new mongoose.Schema({
  eventId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    trim: true,
    lowercase: true
  },
  phone: {
    type: String,
    required: true,
    trim: true
  },
  rollNo: {
    type: String,
    trim: true
  },
  experienceLevel: {
    type: String,
    required: true,
```

```
    trim: true
  },
  emergencyContact: {
    type: String,
    required: true,
    trim: true
  },
  tShirtSize: {
    type: String,
    trim: true
  },
  paymentStatus: {
    type: String,
    enum: ['Pending', 'Received'],
    default: 'Pending'
  },
  registeredAt: {
    type: Date,
    default: Date.now
  }
}, { timestamps: true });

module.exports = mongoose.model('Registration', RegistrationSchema);
```

The most important line here is `eventId: { type: mongoose.Schema.Types.ObjectId, ref: 'Event' }`. This is how Mongoose models a relationship between two collections: rather than duplicating the event's title/date/location into every single registration document, a Registration just stores a reference (the event's MongoDB `_id`) and the two are joined together at query time whenever needed.

7. Backend, Step 5: Middleware (Auth Guard & File Uploads)

"Middleware" in Express just means a function that runs *in between* a request arriving and a route's main logic running — usually to check something or transform the request before the real work happens. This project has two custom ones.

backend/middleware/auth.js

This is the gatekeeper that protects every admin-only route (creating an event, deleting a gallery photo, etc.) from being called by an anonymous visitor.

```
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'supersecretkeyforjumhcadmins1966');
      req.user = decoded;
      next();
    } catch (error) {
      console.error('JWT verification error:', error);
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
  }

  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token provided' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'Super Admin') {
    next();
  } else {
    res.status(403).json({ message: 'Access denied: Super Admin role required' });
  }
};

module.exports = { protect, adminOnly };
```

- `protect` looks for an `Authorization: Bearer <token>` header. If present, it verifies the JWT using the same secret the login route signed it with. On success, it attaches the decoded payload (id/email/role) onto `req.user` and calls `next()` to let the request continue into the actual route handler. On failure (missing or invalid token), it responds `401 Unauthorized` and never calls `next()` — which is what actually stops the request from reaching protected logic.
- `adminOnly` is a second, stricter guard that checks `req.user.role === 'Super Admin'`, for actions that should be restricted even further than "any logged-in admin". (It's exported and available, though the current route files only apply `protect` — a natural place to tighten permissions later if an "Editor" role should be limited.)

> **WHY A JWT INSTEAD OF, SAY, STORING SESSIONS IN THE DATABASE?**
>
> A JSON Web Token is *self-contained* — it carries the admin's id/email/role right inside it, cryptographically signed so it can't be tampered with. That means the server can verify a request is legitimate without a database lookup or any server-side session storage at all: just re-run the same signing math and see if the signatures match. The trade-off is that a JWT can't be "revoked" early (it's just trusted until it expires, here after 30 days) — a deliberate simplicity-for-this-project trade-off rather than an oversight.

backend/middleware/upload.js

Wraps the Multer library to handle the two places this app accepts an uploaded image: alumni photos and gallery items.

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

// Ensure upload directory exists
const uploadDir = path.join(__dirname, '..', 'uploads');
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

// Storage Configuration
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, uploadDir);
  },
  filename: function (req, file, cb) {
```

```
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1e9);
    cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));
  }
});

// File Filter for Images
const fileFilter = (req, file, cb) => {
  const allowedFileTypes = /jpeg|jpg|png|gif|webp/;
  const extname = allowedFileTypes.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedFileTypes.test(file.mimetype);

  if (extname && mimetype) {
    cb(null, true);
  } else {
    cb(new Error('Only images (jpg, jpeg, png, gif, webp) are allowed!'), false);
  }
};

const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit
  fileFilter: fileFilter
});

module.exports = upload;
```

- It first makes sure a `backend/uploads` folder actually exists on disk (creating it if this is the very first upload), since Multer will error out trying to save into a folder that isn't there.
- `multer.diskStorage` configures *where* files land (the uploads folder) and *what they're named* — a timestamp plus a random number is appended to keep every filename unique, so two people uploading `photo.jpg` on the same day never overwrite each other.
- `fileFilter` rejects anything that isn't an image (checked both by file extension and by MIME type, since either one can lie on its own), and `limits: { fileSize: 5 * 1024 * 1024 }` caps uploads at 5MB so a huge file can't fill up the server's disk or stall the request.

8. Backend, Step 6: The API Routes

Routes are where the actual "what happens when someone calls this URL" logic lives. Every route file follows the same overall shape: create an Express `Router`, define a handler for each URL + HTTP method combination, and export the router so `server.js` can mount it. We'll look at `events.js` first in full detail since it's the cleanest example of the pattern — the rest reuse the same shape with small, specific differences that we'll call out.

backend/routes/events.js

```
const express = require('express');
const router = express.Router();
const Event = require('../models/Event');
const { protect } = require('../middleware/auth');

// @desc    Get all events
// @route   GET /api/events
// @access  Public
router.get('/', async (req, res) => {
  try {
    const events = await Event.find().sort({ date: 1 });
    res.json(events);
  } catch (error) {
    console.error('Error fetching events:', error);
    res.status(500).json({ message: 'Server error fetching events' });
  }
});

// @desc    Get single event by ID
// @route   GET /api/events/:id
// @access  Public
router.get('/:id', async (req, res) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) {
      return res.status(404).json({ message: 'Event not found' });
    }
    res.json(event);
  } catch (error) {
    console.error('Error fetching event by ID:', error);
    res.status(500).json({ message: 'Server error fetching event' });
  }
});

// @desc    Create a new event
// @route   POST /api/events
// @access  Private (Admin)
router.post('/', protect, async (req, res) => {
  const { title, type, date, description, location, registrationOpen, capacity, deadline } = req.body;

  if (!title || !type || !date || !description || !location) {
    return res.status(400).json({ message: 'Please enter all required fields: title, type, date, description, location' });
  }

  try {
    const newEvent = new Event({
      title,
      type,
      date,
      description,
      location,
      registrationOpen: registrationOpen !== undefined ? registrationOpen : true,
      capacity: capacity ? Number(capacity) : null,
      deadline: deadline ? new Date(deadline) : null
    });

    const saved = await newEvent.save();
    res.status(201).json(saved);
  } catch (error) {
    console.error('Error creating event:', error);
    res.status(500).json({ message: 'Server error creating event' });
  }
});

// @desc    Update an event
// @route   PUT /api/events/:id
// @access  Private (Admin)
router.put('/:id', protect, async (req, res) => {
  const { title, type, date, description, location, registrationOpen, capacity, deadline } = req.body;

  try {
    const event = await Event.findById(req.params.id);
    if (!event) {
      return res.status(404).json({ message: 'Event not found' });
    }
```

```
    if (title !== undefined) event.title = title;
    if (type !== undefined) event.type = type;
    if (date !== undefined) event.date = date;
    if (description !== undefined) event.description = description;
    if (location !== undefined) event.location = location;
    if (registrationOpen !== undefined) event.registrationOpen = registrationOpen;
    if (capacity !== undefined) event.capacity = capacity ? Number(capacity) : null;
    if (deadline !== undefined) event.deadline = deadline ? new Date(deadline) : null;

    const updated = await event.save();
    res.json(updated);
  } catch (error) {
    console.error('Error updating event:', error);
    res.status(500).json({ message: 'Server error updating event' });
  }
});

// @desc    Delete an event
// @route   DELETE /api/events/:id
// @access  Private (Admin)
router.delete('/:id', protect, async (req, res) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) {
      return res.status(404).json({ message: 'Event not found' });
    }

    await event.deleteOne();
    res.json({ message: 'Event deleted successfully' });
  } catch (error) {
    console.error('Error deleting event:', error);
    res.status(500).json({ message: 'Server error deleting event' });
  }
});

module.exports = router;
```

This one file defines the five operations the frontend needs for events — the acronym for this shape is **CRUD** (Create, Read, Update, Delete), and you'll see it repeated across almost every route file in this project:

- `GET /` — public, returns every event sorted by date. No `protect` middleware, so anyone can view the list — that's intentional, this is what the public Events page calls.
- `GET /:id` — public, returns one event by its Mongo `_id` (used by the registration page to show which course you're signing up for).
- `POST /` — *protected* (notice `protect` is passed as a second argument before the handler function). Validates the required fields are present, then saves a new Event.
- `PUT /:id` — protected, edits an existing event. Notice the pattern `if (title !== undefined) event.title = title;` repeated for every field — this lets an admin update just *one* field (say, just the description) without being forced to resend every other field too.
- `DELETE /:id` — protected, removes the event.

> **WHY <SPAN CLASS='INLINE-CODE'>ROUTER.POST('/', PROTECT, ASYNC (REQ, RES) => {&HELLIP;})</SPAN> AND NOT JUST TWO ARGUMENTS?**
>
> Express lets you chain any number of middleware functions before the final handler, and it runs them in order. `protect` runs first; if it calls `next()` (token was valid), control passes to the `async (req, res) => {…}` handler right after it. If `protect` instead sends a 401 response itself, the handler never runs at all. This is the exact same mechanism used for the file-upload middleware you'll see next.

backend/routes/secretaries.js *same CRUD shape as events.js*

```
const express = require('express');
const router = express.Router();
const Secretary = require('../models/Secretary');
const { protect } = require('../middleware/auth');

// @desc    Get all secretaries timeline
// @route   GET /api/secretaries
// @access  Public
router.get('/', async (req, res) => {
  try {
    const secretaries = await Secretary.find().sort({ yearStart: -1 });
    res.json(secretaries);
  } catch (error) {
    console.error('Error fetching secretaries:', error);
    res.status(500).json({ message: 'Server error fetching secretaries' });
  }
});

// @desc    Create a new secretary entry
// @route   POST /api/secretaries
// @access  Private (Admin)
router.post('/', protect, async (req, res) => {
  const { yearStart, yearEnd, name, note } = req.body;

  if (!yearStart || !yearEnd || !name) {
```

```
    return res.status(400).json({ message: 'Year Start, Year End, and Name are required' });
  }

  try {
    const newSecretary = new Secretary({
      yearStart,
      yearEnd,
      name,
      note
    });

    const saved = await newSecretary.save();
    res.status(201).json(saved);
  } catch (error) {
    console.error('Error creating secretary:', error);
    res.status(500).json({ message: 'Server error creating secretary' });
  }
});

// @desc    Update a secretary entry
// @route   PUT /api/secretaries/:id
// @access  Private (Admin)
router.put('/:id', protect, async (req, res) => {
  const { yearStart, yearEnd, name, note } = req.body;

  try {
    const secretary = await Secretary.findById(req.params.id);
    if (!secretary) {
      return res.status(404).json({ message: 'Secretary entry not found' });
    }

    if (yearStart !== undefined) secretary.yearStart = yearStart;
    if (yearEnd !== undefined) secretary.yearEnd = yearEnd;
    if (name !== undefined) secretary.name = name;
    if (note !== undefined) secretary.note = note;

    const updated = await secretary.save();
    res.json(updated);
  } catch (error) {
    console.error('Error updating secretary:', error);
    res.status(500).json({ message: 'Server error updating secretary' });
  }
});

// @desc    Delete a secretary entry
// @route   DELETE /api/secretaries/:id
// @access  Private (Admin)
router.delete('/:id', protect, async (req, res) => {
  try {
    const secretary = await Secretary.findById(req.params.id);
    if (!secretary) {
      return res.status(404).json({ message: 'Secretary entry not found' });
    }

    await secretary.deleteOne();
    res.json({ message: 'Secretary entry deleted successfully' });
  } catch (error) {
    console.error('Error deleting secretary:', error);
    res.status(500).json({ message: 'Server error deleting secretary' });
  }
});

module.exports = router;
```

Structurally identical to `events.js` — public `GET`, protected `POST` / `PUT` / `DELETE` — just operating on the `Secretary` model instead. Once you've understood one CRUD route file in this project, you've effectively understood most of them.

backend/routes/alumni.js *CRUD + file upload*

```
const express = require('express');
const router = express.Router();
const Alumni = require('../models/Alumni');
const { protect } = require('../middleware/auth');
const upload = require('../middleware/upload');
const fs = require('fs');
const path = require('path');

// @desc    Get all alumni and supporters
// @route   GET /api/alumni
// @access  Public
router.get('/', async (req, res) => {
  try {
    const alumni = await Alumni.find().sort({ batchYear: -1 });
    res.json(alumni);
  } catch (error) {
    console.error('Error fetching alumni:', error);
    res.status(500).json({ message: 'Server error fetching alumni' });
```

```
  }
});

// @desc    Create an alumni profile
// @route   POST /api/alumni
// @access  Private (Admin)
router.post('/', protect, upload.single('photo'), async (req, res) => {
  try {
    const { name, batchYear, roleHistory, currentContribution, bio, isActiveInstructor, photoUrl } = req.body;

    if (!name || !batchYear || !roleHistory) {
      return res.status(400).json({ message: 'Name, batch year, and role history are required' });
    }

    // Determine final photo URL: uploaded file path or fallback to provided URL or default
    let finalPhotoUrl = photoUrl || '';
    if (req.file) {
      finalPhotoUrl = `/uploads/${req.file.filename}`;
    }

    const newAlumni = new Alumni({
      name,
      batchYear: Number(batchYear),
      roleHistory,
      currentContribution,
      bio,
      isActiveInstructor: isActiveInstructor === 'true' || isActiveInstructor === true,
      photoUrl: finalPhotoUrl
    });

    const saved = await newAlumni.save();
    res.status(201).json(saved);
  } catch (error) {
    console.error('Error creating alumni profile:', error);
    res.status(500).json({ message: 'Server error creating alumni profile' });
  }
});

// @desc    Update an alumni profile
// @route   PUT /api/alumni/:id
// @access  Private (Admin)
router.put('/:id', protect, upload.single('photo'), async (req, res) => {
  try {
    const alumni = await Alumni.findById(req.params.id);
    if (!alumni) {
      return res.status(404).json({ message: 'Alumni profile not found' });
    }

    const { name, batchYear, roleHistory, currentContribution, bio, isActiveInstructor, photoUrl } = req.body;

    if (name !== undefined) alumni.name = name;
    if (batchYear !== undefined) alumni.batchYear = Number(batchYear);
    if (roleHistory !== undefined) alumni.roleHistory = roleHistory;
    if (currentContribution !== undefined) alumni.currentContribution = currentContribution;
    if (bio !== undefined) alumni.bio = bio;
    if (isActiveInstructor !== undefined) {
      alumni.isActiveInstructor = isActiveInstructor === 'true' || isActiveInstructor === true;
    }

    if (req.file) {
      // Delete old photo if it was a locally uploaded file and differs
      if (alumni.photoUrl && alumni.photoUrl.startsWith('/uploads/')) {
        const oldPath = path.join(__dirname, '..', alumni.photoUrl);
        if (fs.existsSync(oldPath)) {
          fs.unlinkSync(oldPath);
        }
      }
      alumni.photoUrl = `/uploads/${req.file.filename}`;
    } else if (photoUrl !== undefined) {
      alumni.photoUrl = photoUrl;
    }

    const updated = await alumni.save();
    res.json(updated);
  } catch (error) {
    console.error('Error updating alumni profile:', error);
    res.status(500).json({ message: 'Server error updating alumni profile' });
  }
});

// @desc    Delete an alumni profile
// @route   DELETE /api/alumni/:id
// @access  Private (Admin)
router.delete('/:id', protect, async (req, res) => {
  try {
    const alumni = await Alumni.findById(req.params.id);
    if (!alumni) {
      return res.status(404).json({ message: 'Alumni profile not found' });
    }
```

```
    // Delete local photo file
    if (alumni.photoUrl && alumni.photoUrl.startsWith('/uploads/')) {
      const photoPath = path.join(__dirname, '..', alumni.photoUrl);
      if (fs.existsSync(photoPath)) {
        fs.unlinkSync(photoPath);
      }
    }

    await alumni.deleteOne();
    res.json({ message: 'Alumni profile deleted successfully' });
  } catch (error) {
    console.error('Error deleting alumni profile:', error);
    res.status(500).json({ message: 'Server error deleting alumni profile' });
  }
});

module.exports = router;
```

This one adds a wrinkle on top of the basic CRUD shape: photo handling.

- `router.post('/', protect, upload.single('photo'), async (…) => {…})` chains *two* middlewares before the handler — first check the JWT, then let Multer parse the incoming multipart form and save any attached photo to disk.
- Inside the handler, `finalPhotoUrl` prefers an uploaded file's path, but falls back to a plain `photoUrl` string if the admin instead pasted an image link rather than uploading a file — the form supports both.
- The `PUT` and `DELETE` handlers both call `fs.unlinkSync(oldPath)` to delete the *old* photo file from disk when it's being replaced or the profile is removed — without this, every edit or deletion would leave an orphaned image file behind forever.

backend/routes/gallery.js *CRUD + file upload, no update route*

```
const express = require('express');
const router = express.Router();
const GalleryItem = require('../models/GalleryItem');
const { protect } = require('../middleware/auth');
const upload = require('../middleware/upload');
const fs = require('fs');
const path = require('path');

// @desc    Get all gallery items
// @route   GET /api/gallery
// @access  Public
router.get('/', async (req, res) => {
  try {
    const items = await GalleryItem.find().sort({ year: -1, createdAt: -1 });
    res.json(items);
  } catch (error) {
    console.error('Error fetching gallery items:', error);
    res.status(500).json({ message: 'Server error fetching gallery items' });
  }
});

// @desc    Create a new gallery item
// @route   POST /api/gallery
// @access  Private (Admin)
router.post('/', protect, upload.single('image'), async (req, res) => {
  try {
    const { year, eventTag, caption, imageUrl } = req.body;

    if (!year || !eventTag) {
      return res.status(400).json({ message: 'Year and Event Tag are required' });
    }

    let finalImageUrl = imageUrl || '';
    if (req.file) {
      finalImageUrl = `/uploads/${req.file.filename}`;
    }

    if (!finalImageUrl) {
      return res.status(400).json({ message: 'Please upload an image or provide an image URL' });
    }

    const newItem = new GalleryItem({
      imageUrl: finalImageUrl,
      year: Number(year),
      eventTag,
      caption
    });

    const saved = await newItem.save();
    res.status(201).json(saved);
  } catch (error) {
    console.error('Error creating gallery item:', error);
    res.status(500).json({ message: 'Server error creating gallery item' });
  }
});
```

```
// @desc    Delete a gallery item
// @route   DELETE /api/gallery/:id
// @access  Private (Admin)
router.delete('/:id', protect, async (req, res) => {
  try {
    const item = await GalleryItem.findById(req.params.id);
    if (!item) {
      return res.status(404).json({ message: 'Gallery item not found' });
    }

    // Delete local file if it exists
    if (item.imageUrl && item.imageUrl.startsWith('/uploads/')) {
      const filePath = path.join(__dirname, '..', item.imageUrl);
      if (fs.existsSync(filePath)) {
        fs.unlinkSync(filePath);
      }
    }

    await item.deleteOne();
    res.json({ message: 'Gallery item deleted successfully' });
  } catch (error) {
    console.error('Error deleting gallery item:', error);
    res.status(500).json({ message: 'Server error deleting gallery item' });
  }
});

module.exports = router;
```

Very similar to `alumni.js`'s upload handling, but note there's no `PUT /:id` route here — the admin dashboard only lets you upload a new gallery item or delete an existing one, not edit one in place. That's a deliberate scope decision (gallery items are simple enough that "delete and re-upload" was good enough), not a missing feature.

`backend/routes/registrations.js` *the most involved route in the project*

```
const express = require('express');
const router = express.Router();
const Registration = require('../models/Registration');
const Event = require('../models/Event');
const { protect } = require('../middleware/auth');
const nodemailer = require('nodemailer');

// Helper to construct mail transporter
const getTransporter = () => {
  if (
    process.env.SMTP_HOST &&
    process.env.SMTP_PORT &&
    process.env.SMTP_USER &&
    process.env.SMTP_PASS
  ) {
    return nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: Number(process.env.SMTP_PORT),
      secure: Number(process.env.SMTP_PORT) === 465, // true for 465, false for other ports
      auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS
      }
    });
  }
  return null;
};

// @desc    Register for an event/course
// @route   POST /api/registrations
// @access  Public
router.post('/', async (req, res) => {
  const { eventId, name, email, phone, rollNo, experienceLevel, emergencyContact, tShirtSize } = req.body;

  if (!eventId || !name || !email || !phone || !experienceLevel || !emergencyContact) {
    return res.status(400).json({ message: 'Please enter all required fields: name, email, phone, experience level, emergency contact' });
  }

  try {
    // 1. Find the event
    const event = await Event.findById(eventId);
    if (!event) {
      return res.status(404).json({ message: 'Event/Course not found' });
    }

    // 2. Check registration status
    if (!event.registrationOpen) {
      return res.status(400).json({ message: 'Registrations are closed for this event/course' });
    }

    // 3. Check deadline
```

```
    if (event.deadline && new Date() > new Date(event.deadline)) {
      return res.status(400).json({ message: 'The registration deadline for this event has passed' });
    }

    // 4. Atomically check and increment capacity if applicable
    if (event.capacity !== null) {
      // Perform atomic update: increment registrationsCount ONLY IF registrationsCount < capacity
      const updateResult = await Event.updateOne(
        {
          _id: eventId,
          registrationOpen: true,
          $expr: { $lt: ['$registrationsCount', '$capacity'] }
        },
        { $inc: { registrationsCount: 1 } }
      );

      if (updateResult.modifiedCount === 0) {
        return res.status(400).json({ message: 'Registration failed. The course is already full!' });
      }
    } else {
      // Unlimited capacity: just increment count
      await Event.updateOne({ _id: eventId }, { $inc: { registrationsCount: 1 } });
    }

    // 5. Create the registration record
    const registration = new Registration({
      eventId,
      name,
      email,
      phone,
      rollNo,
      experienceLevel,
      emergencyContact,
      tShirtSize,
      paymentStatus: 'Pending'
    });

    const savedRegistration = await registration.save();

    // 6. Send Email Confirmation
    const adminEmail = process.env.ADMIN_EMAIL || 'admin@jumhc.org';
    const emailSubject = `Registration Confirmation: ${event.title}`;
    const emailBodyHTML = `
      <h2>Hello ${name},</h2>
      <p>Thank you for registering for the <strong>${event.title}</strong> organised by the Jadavpur University Mountaineering & Hiking Club.
</p>
      <h3>Registration Details:</h3>
      <ul>
        <li><strong>Event/Course:</strong> ${event.title}</li>
        <li><strong>Location:</strong> ${event.location}</li>
        <li><strong>Date:</strong> ${new Date(event.date).toLocaleDateString()}</li>
        <li><strong>Name:</strong> ${name}</li>
        <li><strong>Email:</strong> ${email}</li>
        <li><strong>Phone:</strong> ${phone}</li>
        <li><strong>Roll Number:</strong> ${rollNo || 'N/A'}</li>
        <li><strong>Experience Level:</strong> ${experienceLevel}</li>
        <li><strong>Emergency Contact:</strong> ${emergencyContact}</li>
        <li><strong>T-Shirt Size:</strong> ${tShirtSize || 'N/A'}</li>
        <li><strong>Payment Status:</strong> Pending</li>
      </ul>
      <p>For payments or queries, please reach out to the club committee or reply directly to this email.</p>
      <br/>
      <p>Best regards,<br/>Jadavpur University Mountaineering & Hiking Club (Est. 1966)</p>
    `;

    const transporter = getTransporter();
    if (transporter) {
      try {
        // Send email to registrant
        await transporter.sendMail({
          from: `"JUMHC Admin" <${process.env.SMTP_USER}>`,
          to: email,
          subject: emailSubject,
          html: emailBodyHTML
        });

        // Send email notification to admin
        await transporter.sendMail({
          from: `"JUMHC Portal" <${process.env.SMTP_USER}>`,
          to: adminEmail,
          subject: `[New Registration] ${event.title} - ${name}`,
          html: `<h3>New registrant for ${event.title}:</h3>` + emailBodyHTML
        });

        console.log(`Confirmation emails sent successfully to ${email} and ${adminEmail}.`);
      } catch (err) {
        console.error('Nodemailer error sending email:', err.message);
        // Do not crash the response since registration was saved successfully
      }
```

```
    } else {
      console.log('--- EMAIL FALLBACK LOG ---');
      console.log(`To: ${email}`);
      console.log(`Subject: ${emailSubject}`);
      console.log('Body HTML:\n', emailBodyHTML);
      console.log('--- END OF EMAIL LOG (Configure SMTP credentials in .env to send real emails) ---');
    }

    res.status(201).json({
      message: 'Registration successful!',
      registration: savedRegistration
    });
  } catch (error) {
    console.error('Error during registration:', error);
    res.status(500).json({ message: 'Server error during registration' });
  }
});

// @desc    Get all registrations for a specific event
// @route   GET /api/registrations/event/:eventId
// @access  Private (Admin)
router.get('/event/:eventId', protect, async (req, res) => {
  try {
    const registrations = await Registration.find({ eventId: req.params.eventId }).sort({ registeredAt: -1 });
    res.json(registrations);
  } catch (error) {
    console.error('Error fetching registrations:', error);
    res.status(500).json({ message: 'Server error fetching registrations' });
  }
});

// @desc    Export event registrations to CSV
// @route   GET /api/registrations/event/:eventId/export
// @access  Private (Admin)
router.get('/event/:eventId/export', protect, async (req, res) => {
  try {
    const event = await Event.findById(req.params.eventId);
    if (!event) {
      return res.status(404).json({ message: 'Event not found' });
    }

    const registrations = await Registration.find({ eventId: req.params.eventId }).sort({ registeredAt: -1 });

    let csvContent = 'Name,Email,Phone,Roll Number,Experience Level,Emergency Contact,T-Shirt Size,Payment Status,Registered At\n';

    registrations.forEach(r => {
      const escape = (str) => {
        if (!str) return '""';
        return `"${str.toString().replace(/"/g, '""')}"`;
      };

      csvContent +=
`${escape(r.name)},${escape(r.email)},${escape(r.phone)},${escape(r.rollNo)},${escape(r.experienceLevel)},${escape(r.emergencyContact)},${escape
(r.tShirtSize)},${escape(r.paymentStatus)},"${r.registeredAt.toISOString()}"\n`;
    });

    res.setHeader('Content-Type', 'text/csv');
    res.setHeader('Content-Disposition', `attachment; filename=registrations-${event.title.toLowerCase().replace(/[^a-z0-9]+/g, '-')}.csv`);
    res.status(200).send(csvContent);
  } catch (error) {
    console.error('Error exporting registrations to CSV:', error);
    res.status(500).json({ message: 'Server error exporting registrations' });
  }
});

// @desc    Update registration payment status
// @route   PUT /api/registrations/:id/payment
// @access  Private (Admin)
router.put('/:id/payment', protect, async (req, res) => {
  const { paymentStatus } = req.body;

  if (!paymentStatus || !['Pending', 'Received'].includes(paymentStatus)) {
    return res.status(400).json({ message: 'Invalid payment status. Must be Pending or Received' });
  }

  try {
    const registration = await Registration.findById(req.params.id);
    if (!registration) {
      return res.status(404).json({ message: 'Registration not found' });
    }

    registration.paymentStatus = paymentStatus;
    const updated = await registration.save();
    res.json(updated);
  } catch (error) {
    console.error('Error updating payment status:', error);
    res.status(500).json({ message: 'Server error updating payment status' });
  }
});
```

```
// @desc    Delete a registration
// @route   DELETE /api/registrations/:id
// @access  Private (Admin)
router.delete('/:id', protect, async (req, res) => {
  try {
    const registration = await Registration.findById(req.params.id);
    if (!registration) {
      return res.status(404).json({ message: 'Registration not found' });
    }

    const eventId = registration.eventId;

    // Delete registration
    await registration.deleteOne();

    // Decrement the registrationsCount in the event (never below 0)
    await Event.updateOne(
      { _id: eventId, registrationsCount: { $gt: 0 } },
      { $inc: { registrationsCount: -1 } }
    );

    res.json({ message: 'Registration deleted and capacity count updated' });
  } catch (error) {
    console.error('Error deleting registration:', error);
    res.status(500).json({ message: 'Server error deleting registration' });
  }
});

module.exports = router;
```

This is worth slowing down for — it's the file doing the most real-world work in the entire backend. Breaking the public `POST /` handler into its numbered steps (matching the comments in the code):

- **Step 1–3:** load the event being registered for, and bail out early with a clear message if registrations are closed or the deadline has already passed.
- **Step 4:** this is the trickiest part. If the event has a limited `capacity`, the code doesn't just check `if (registrationsCount < capacity)` and then separately increment it — it does both in *one atomic database operation*: `Event.updateOne({ …, $expr: { $lt: ['$registrationsCount', '$capacity'] } }, { $inc: { registrationsCount: 1 } })`. MongoDB only applies the increment if the condition is still true *at the exact moment it runs*, and reports back whether it actually modified anything.
- **Step 5:** only after that capacity check succeeds does it create and save the actual `Registration` document.
- **Step 6:** it builds an HTML confirmation email and tries to send it via Nodemailer — but if SMTP credentials aren't configured in `.env`, it falls back to simply logging the email contents to the console, and either way the registration itself has already been saved, so a failed email never loses someone's spot.

> **WHY DOES THE CAPACITY CHECK NEED TO BE ATOMIC?**
>
> Picture two people submitting the registration form for the last remaining seat within the same second. If the code did "read the count, check it in JavaScript, then save" as three separate steps, both requests could read the count *before* either one's increment is saved — both would see "39 of 40", both would think there's room, and both would be let in, overbooking the course by one. Folding the check and the increment into a single database command closes that gap: MongoDB itself guarantees only one of the two requests can win.

The remaining routes in this file are more straightforward reads/writes: fetching all registrations for one event, exporting them as a downloadable CSV file (built by hand as a string, with a small `escape()` helper to correctly quote any commas or quotes inside a name/field), toggling a registration's payment status between "Pending"/"Received", and deleting a registration (which also decrements the event's seat counter back down, so a cancelled seat becomes available again).

backend/routes/auth.js

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const AdminUser = require('../models/AdminUser');
const { protect } = require('../middleware/auth');

// @desc    Auth admin & get token
// @route   POST /api/auth/login
// @access  Public
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: 'Please enter all fields' });
  }

  try {
    const admin = await AdminUser.findOne({ email });
    if (!admin) {
      return res.status(401).json({ message: 'Invalid credentials' });
    }
```

```
    const isMatch = await bcrypt.compare(password, admin.passwordHash);
    if (!isMatch) {
      return res.status(401).json({ message: 'Invalid credentials' });
    }

    // Generate JWT
    const token = jwt.sign(
      { id: admin._id, email: admin.email, role: admin.role },
      process.env.JWT_SECRET || 'supersecretkeyforjumhcadmins1966',
      { expiresIn: '30d' }
    );

    res.json({
      token,
      admin: {
        id: admin._id,
        email: admin.email,
        role: admin.role
      }
    });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ message: 'Server error' });
  }
});

// @desc    Get current admin profile
// @route   GET /api/auth/me
// @access  Private
router.get('/me', protect, async (req, res) => {
  try {
    const admin = await AdminUser.findById(req.user.id).select('-passwordHash');
    if (!admin) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json(admin);
  } catch (error) {
    console.error('Get profile error:', error);
    res.status(500).json({ message: 'Server error' });
  }
});

module.exports = router;
```

- `POST /login` looks up the admin by email, then uses `bcrypt.compare(password, admin.passwordHash)` to check the submitted password against the stored hash — this is the only place a plain-text password ever exists in the backend, and only for the instant it takes to compare it. On success, it signs a JWT containing the admin's id/email/role, valid for 30 days.
- `GET /me` is a protected route the frontend calls on page load to ask "is my saved token still valid, and if so, who am I?" — it deliberately excludes the `passwordHash` field from the response with `.select('-passwordHash')`, so even the logged-in admin's own browser never receives that hash over the network.

WHY RESPOND WITH THE SAME "INVALID CREDENTIALS" MESSAGE FOR BOTH A WRONG EMAIL AND A WRONG PASSWORD?

If a wrong email produced a different error message than a wrong password ("no account with that email" vs. "wrong password"), an attacker could use that difference to figure out which email addresses have accounts at all, one guess at a time. Returning one identical, generic message either way avoids leaking that information.

9. Backend, Step 7: Seeding Starter Data

`backend/scripts/seed.js`

With every model and route in place, the database is still completely empty — a brand new MongoDB has no admin account, no events, no alumni. This script exists purely to fix that in one command (`npm run seed`), so the very first time you run the project it already looks like a real, populated website instead of a blank shell.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const dotenv = require('dotenv');
const AdminUser = require('../models/AdminUser');
const Secretary = require('../models/Secretary');
const Alumni = require('../models/Alumni');
const Event = require('../models/Event');
const GalleryItem = require('../models/GalleryItem');

dotenv.config();

const seed = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI || 'mongodb://localhost:27017/jumhc');
    console.log('MongoDB connected for seeding...');

    // Clear existing collection records
    await AdminUser.deleteMany();
    await Secretary.deleteMany();
    await Alumni.deleteMany();
    await Event.deleteMany();
    await GalleryItem.deleteMany();
    console.log('Cleared existing data.');

    // 1. Seed Admin User
    const passwordHash = await bcrypt.hash('admin1966', 10);
    const admin = await AdminUser.create({
      email: 'admin@jumhc.org',
      passwordHash,
      role: 'Super Admin'
    });
    console.log(`Seeded admin user: ${admin.email} (password: admin1966)`);

    // 2. Seed Secretaries Timeline (JU Mountaineering Club established in 1966)
    const secretariesData = [
      { yearStart: 1966, yearEnd: 1967, name: 'Sujal K. Sen', note: 'Founding Secretary' },
      { yearStart: 1967, yearEnd: 1969, name: 'Prabhat Mitra', note: 'First Himalayan expedition organized' },
      { yearStart: 1974, yearEnd: 1976, name: 'Amitabha Ghosh', note: 'Rock climbing training introduced at Susunia Hills' },
      { yearStart: 1982, yearEnd: 1984, name: 'Sabyasachi Roy', note: 'Successful ascent of peak Jogin-III' },
      { yearStart: 1990, yearEnd: 1992, name: 'Rupak Bhattacharya', note: 'Silver Jubilee celebrations' },
      { yearStart: 1998, yearEnd: 2000, name: 'Devasis Mukhopadhyay', note: 'Introduced artificial wall climbing' },
      { yearStart: 2006, yearEnd: 2008, name: 'Sayantan Bose', note: 'First inter-university climbing competition' },
      { yearStart: 2012, yearEnd: 2014, name: 'Aniruddha Das', note: 'Golden Jubilee pre-celebrations' },
      { yearStart: 2018, yearEnd: 2020, name: 'Debjyoti Sen', note: 'Modernized training wall equipment' },
      { yearStart: 2024, yearEnd: 2025, name: 'Rohan Banerjee', note: 'Post-pandemic expedition revival' },
      { yearStart: 2025, yearEnd: 2026, name: 'Ananya Chakraborty', note: 'Current serving Secretary' }
    ];
    await Secretary.insertMany(secretariesData);
    console.log('Seeded Secretaries timeline.');

    // 3. Seed Alumni
    const alumniData = [
      {
        name: 'Prof. Kalyan Roy',
        batchYear: 1972,
        roleHistory: 'Secretary 1972-1974 -> Life Member',
        currentContribution: 'Advisory Committee Chairman & Expeditions Sponsor',
        bio: 'Founding generation member. Led multiple treks in Garhwal Himalayas. Currently advises the executive committee.',
        photoUrl: 'https://images.unsplash.com/photo-1507003211169-0a1dd7228f2d?auto=format&fit=crop&q=80&w=200',
        isActiveInstructor: false
      },
      {
        name: 'Devasis Mukhopadhyay',
        batchYear: 1999,
        roleHistory: 'Secretary 1998-2000 -> Lead Instructor',
        currentContribution: 'Chief Instructor for Susunia Rock Climbing Courses',
        bio: 'Pioneered artificial wall climbing training on campus. Certified instructor from Nehru Institute of Mountaineering (NIM).',
        photoUrl: 'https://images.unsplash.com/photo-1500648767791-00dcc994a43e?auto=format&fit=crop&q=80&w=200',
        isActiveInstructor: true
      },
      {
        name: 'Sayantan Bose',
        batchYear: 2008,
        roleHistory: 'Secretary 2006-2008 -> Expedition Leader',
        currentContribution: 'Technical Advisor & Equipment Auditor',
        bio: 'Scaled peak Kedarnath Dome in 2011. Leads annual advanced rock climbing camps.',
        photoUrl: 'https://images.unsplash.com/photo-1472099645785-5658abf4ff4e?auto=format&fit=crop&q=80&w=200',
```

```
        isActiveInstructor: true
      },
      {
        name: 'Dr. Rimi Sen',
        batchYear: 2015,
        roleHistory: 'Vice President 2014-2015 -> Medical Advisor',
        currentContribution: 'First-aid & High-Altitude Sickness lecturer',
        bio: 'Medical practitioner specializing in sports medicine. Conducts annual fitness assessments for expedition teams.',
        photoUrl: 'https://images.unsplash.com/photo-1544005313-94ddf0286df2?auto=format&fit=crop&q=80&w=200',
        isActiveInstructor: false
      }
    ];
    await Alumni.insertMany(alumniData);
    console.log('Seeded Alumni profiles.');

    // 4. Seed Events
    const eventsData = [
      {
        title: '60th Annual Rock Climbing Course (Feb 2027)',
        type: 'course',
        date: new Date('2027-02-15T08:00:00.000Z'),
        description: 'Our annual certified offline rock climbing course at Susunia Hills. Includes physical conditioning, bouldering,
rappelling, and certificate distribution.',
        location: 'Susunia Hills, Purulia & Jadavpur University Campus',
        registrationOpen: true,
        capacity: 40,
        registrationsCount: 0,
        deadline: new Date('2027-02-10T23:59:59.000Z')
      },
      {
        title: 'Himalayan Expedition: Peak Mt. Jogin-I',
        type: 'expedition',
        date: new Date('2026-09-10T06:00:00.000Z'),
        description: 'High-altitude mountaineering expedition targeting Mount Jogin-I (6,465m) in the Garhwal Himalayas for senior climbing
members.',
        location: 'Gangotri Region, Uttarakhand',
        registrationOpen: false,
        capacity: 12,
        registrationsCount: 12,
        deadline: new Date('2026-06-01T23:59:59.000Z')
      },
      {
        title: 'All-Bengal Inter-University Bouldering Championship',
        type: 'competition',
        date: new Date('2026-11-20T09:00:00.000Z'),
        description: 'Exciting bouldering competition on our state-of-the-art campus artificial wall. Open to all college and university
students.',
        location: 'Sports Complex Artificial Wall, JU Campus',
        registrationOpen: true,
        capacity: 100,
        registrationsCount: 0,
        deadline: new Date('2026-11-15T23:59:59.000Z')
      },
      {
        title: 'Susunia Weekend Climbing Trip',
        type: 'other',
        date: new Date('2026-01-12T05:00:00.000Z'),
        description: 'A weekend rock-climbing and camping trip for club members to hone natural rock climbing skills.',
        location: 'Susunia Hills, Bankura',
        registrationOpen: false,
        capacity: 25,
        registrationsCount: 25,
        deadline: new Date('2026-01-05T23:59:59.000Z')
      }
    ];
    await Event.insertMany(eventsData);
    console.log('Seeded Events.');

    // 5. Seed Gallery Items
    const galleryData = [
      {
        imageUrl: 'https://images.unsplash.com/photo-1522163182402-834f871fd851?auto=format&fit=crop&q=80&w=600',
        year: 2025,
        eventTag: 'Rock Climbing',
        caption: 'Bouldering practice at Susunia Hills natural rock camp.'
      },
      {
        imageUrl: 'https://images.unsplash.com/photo-1464822759023-fed622ff2c3b?auto=format&fit=crop&q=80&w=600',
        year: 2024,
        eventTag: 'Expedition',
        caption: 'Base camp at 14,000 feet during Mt. Deo Tibba expedition.'
      },
      {
        imageUrl: 'https://images.unsplash.com/photo-1501555088652-021faa106b9b?auto=format&fit=crop&q=80&w=600',
        year: 2023,
        eventTag: 'Course',
        caption: 'Trainees learning rope knotting techniques on the Jadavpur campus.'
      },
      {
```

```
        imageUrl: 'https://images.unsplash.com/photo-1512641406448-6574e777bec6?auto=format&fit=crop&q=80&w=600',
        year: 2022,
        eventTag: 'Competition',
        caption: 'Speed climbing finals at the Inter-University Wall Climbing Cup.'
      },
      {
        imageUrl: 'https://images.unsplash.com/photo-1533240332313-0db49b439ad3?auto=format&fit=crop&q=80&w=600',
        year: 2020,
        eventTag: 'Expedition',
        caption: 'Snow craft practice during Winter Mountaineering Course.'
      },
      {
        imageUrl: 'https://images.unsplash.com/photo-1526625303811-1a90f8f25e3d?auto=format&fit=crop&q=80&w=600',
        year: 1998,
        eventTag: 'Archive Flyer',
        caption: 'Scanned cover flyer of the 32nd Annual Rock Climbing Course.'
      }
    ];
    await GalleryItem.insertMany(galleryData);
    console.log('Seeded Gallery items.');

    console.log('Data seeding completed successfully!');
    mongoose.connection.close();
  } catch (error) {
    console.error('Seeding Error:', error);
    mongoose.connection.close();
  }
};

seed();
```

- It connects to MongoDB directly (it's a standalone script, not part of the running server), then calls `deleteMany()` on every collection first — so re-running the seed script is always safe and repeatable, rather than piling up duplicate data on a second run.
- The one admin account is created with `bcrypt.hash('admin1966', 10)` — the same hashing function `routes/auth.js` later compares against, so this seeded account can immediately log in with password `admin1966`.
- Everything else is just realistic-looking sample arrays (secretaries, alumni, events, gallery items) inserted in bulk with `insertMany()`, which is faster than calling `.save()` in a loop for many documents at once.
- `mongoose.connection.close()` at the end (in both the success and error paths) makes sure the script's Node process actually exits instead of hanging open waiting on a connection nothing is using anymore.

THIS IS WHERE THE BACKEND BUILD ENDS

At this point, if you ran `npm run backend-dev` and then `npm run seed` in another terminal, you'd have a fully working JSON API you could already test with a tool like Postman or your browser at `http://localhost:5000/api/events` — even though not a single line of frontend code exists yet. That's exactly the order this project was meant to be built in.

10. Frontend, Step 1: Dependencies & Build Tools

Everything under /frontend

With a working API to talk to, we can start the React side. Like the backend, it starts with its own `package.json`.

frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^0.379.0",
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "react-router-dom": "^6.23.1"
  },
  "devDependencies": {
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.2",
    "autoprefixer": "^10.4.19",
    "oxlint": "^1.69.0",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "vite": "^8.1.0"
  }
}
```

- **react** / **react-dom** — the core library and the piece that actually renders React components into the browser's DOM.
- **react-router-dom** — gives the app client-side "pages" (like `/about` or `/events`) without a real page reload each time — it just swaps which component is shown based on the URL.
- **lucide-react** — a library of ready-made icon components (the compass, calendar, mail icons etc. you'll see imported throughout the pages).
- **vite** — the dev server and production bundler.
- **@vitejs/plugin-react** — teaches Vite how to understand JSX syntax and enables React's "fast refresh" (instant in-browser updates on save).
- **tailwindcss** / **postcss** / **autoprefixer** — the CSS utility framework and the pipeline that processes it.
- **oxlint** — a linter that checks your code for common mistakes (e.g. breaking React's rules of hooks) as you write it.

frontend/vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

About as minimal as a config file gets: it just tells Vite to use the React plugin. Nearly every Vite + React project starts from this exact file untouched.

frontend/postcss.config.js

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

PostCSS is a tool that transforms CSS through a chain of plugins. This file wires in exactly two: `tailwindcss` (which expands utility classes like `px-4` into real CSS rules) and `autoprefixer` (which automatically adds vendor prefixes like `-webkit-` where needed for older browsers).

frontend/tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        mountain: {
          50: '#f4f6f8',
          100: '#e9edf1',
          200: '#cbd6e2',
          300: '#a3b5ca',
          400: '#758fae',
          500: '#547093',
          600: '#415776',
          700: '#34465e',
          800: '#2d3b4e',
          900: '#0B2545', // Deep Navy
          950: '#051120'
        },
        forest: {
          50: '#f4f9f4',
          100: '#e5f3e5',
          200: '#cce6cc',
          300: '#a3d1a3',
          400: '#70b370',
          500: '#489448',
          600: '#357735',
          700: '#2b5f2b',
          800: '#244c24',
          900: '#1e3f20', // Forest Green
          950: '#0f2210'
        },
        earth: {
          50: '#fdf8f4',
          100: '#fbeee2',
          200: '#f6d9c1',
          300: '#f0be97',
          400: '#e59663',
          500: '#dc763c',
          600: '#cf5e2c',
          700: '#ab4822',
          800: '#893b1f',
          900: '#6f321d', // Earth Amber
          950: '#3c180d'
        }
      },
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
        display: ['Outfit', 'Montserrat', 'sans-serif'],
      }
    },
  },
  plugins: [],
}
```

- `content: […]` tells Tailwind exactly which files to scan for class names, so it only generates the CSS for classes you actually use (keeping the final CSS bundle small) instead of shipping every utility class that exists.
- `theme.extend.colors` defines three custom color palettes — `mountain` (navy/slate), `forest` (green), and `earth` (amber/orange) — each with shades 50 through 950. This is *why* you'll see classes like `bg-mountain-900` or `text-earth-600` all over the page components: they're not built-in Tailwind colors, they're this project's own branded palette, chosen to feel like a mountaineering/outdoors club.
- `theme.extend.fontFamily` registers two font stacks (`Inter` for regular body text, `Outfit` / `Montserrat` for headings) as the `font-sans` and `font-display` utility classes used throughout.

11. Frontend, Step 2: The HTML Shell & Entry Point

frontend/index.html

Every Vite + React app is, underneath everything, still just one real HTML file that the browser loads first.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220 0 100 100%22><text
y=%22.9em%22 font-size=%2290%22>🧗</text></svg>" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- Primary Meta Tags -->
    <title>Jadavpur University Mountaineering & Hiking Club (JUMHC) | Established 1966</title>
    <meta name="title" content="Jadavpur University Mountaineering & Hiking Club (JUMHC) | Established 1966" />
    <meta name="description" content="Official website of Jadavpur University Mountaineering & Hiking Club (JUMHC), Kolkata. Explore our legacy
of Himalayan expeditions, natural rock climbing certification courses at Susunia Hills, and artificial wall bouldering since 1966." />
    <meta name="keywords" content="JUMHC, Jadavpur University, Mountaineering, Hiking, Rock Climbing, Susunia Hills, Bouldering, Kolkata
Adventure Club" />
    <meta name="author" content="JUMHC Committee" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

- The `<head>` carries the SEO meta tags (title, description, keywords) — this is the content search engines and browser tabs actually see; it's plain HTML and has nothing to do with React.
- The entire visible app mounts into one empty `<div id="root"></div>` — React takes over everything inside that div and nothing outside it.
- The one `<script type="module" src="/src/main.jsx">` tag is the only script this page loads directly — it's the thread that pulls in your entire React application.

frontend/src/main.jsx *the true JavaScript entry point*

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Three things happen here: `createRoot` attaches React to that `#root` div from `index.html`, `index.css` is imported so Tailwind's generated styles apply globally, and `<App />` (which we'll get to shortly) is rendered inside `<StrictMode>` — a development-only wrapper that intentionally double-invokes some functions to help surface bugs early; it has no effect on the production build.

frontend/src/index.css

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@400;500;600;700;800&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-mountain-50 text-mountain-800 font-sans antialiased;
  }

  h1, h2, h3, h4, h5, h6 {
    @apply font-display text-mountain-900;
  }
}

/* Glassmorphism utility */
.glass {
  background: rgba(255, 255, 255, 0.7);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.2);
}

.glass-dark {
```

```
  background: rgba(11, 37, 69, 0.85);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.05);
}

/* Custom scrollbars */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}

::-webkit-scrollbar-track {
  @apply bg-mountain-100;
}

::-webkit-scrollbar-thumb {
  @apply bg-mountain-300 rounded-full hover:bg-mountain-400 transition-colors;
}
```

- The `@import url(…fonts.googleapis.com…)` line pulls in the Inter and Outfit fonts referenced by `tailwind.config.js`.
- `@tailwind base; @tailwind components; @tailwind utilities;` are the three directives that tell Tailwind's build step where to inject its generated CSS — every utility class used anywhere in the app (like `flex`, `px-4`, `bg-mountain-900`) ultimately gets generated here.
- The `@layer base` block sets sensible defaults once for the whole app (background/text colors, font family, and making all heading tags use the display font) instead of repeating those classes on every single page.
- `.glass` / `.glass-dark` are two small custom utility classes (a translucent, blurred "glassmorphism" background effect) that go beyond what Tailwind ships out of the box, defined by hand here.
- The `::-webkit-scrollbar` rules restyle the browser's scrollbar to match the site's color palette instead of the OS default.

frontend/src/App.css *leftover boilerplate — not actually used*

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
```

```
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }

  @media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;
```

```
    li {
      flex: 1 1 calc(50% - 8px);
    }

    a {
      width: 100%;
      justify-content: center;
      box-sizing: border-box;
    }
  }
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}
```

WORTH KNOWING: THIS FILE IS DEAD WEIGHT

This is the default stylesheet Vite generates automatically for its React starter template (classes like `.hero`, `#next-steps`, `.ticks` reference a demo counter page that shipped with the template). None of the actual pages in this project use any of these class names — it's simply left over from when the project was first scaffolded with `npm create vite` and never deleted. It's completely safe to delete this file; nothing in the app would break.

12. Frontend, Step 3: The API Helper

`frontend/src/utils/api.js`

Before writing a single page, it's worth building one shared helper for talking to the backend — otherwise every page would end up repeating the same `fetch()` boilerplate (setting headers, attaching the auth token, checking `response.ok`, parsing JSON, handling errors) over and over.

```
const BASE_URL = 'http://localhost:5000/api';

const getHeaders = (isFormData = false) => {
  const token = localStorage.getItem('jumhc_admin_token');
  const headers = {};

  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  if (!isFormData) {
    headers['Content-Type'] = 'application/json';
  }

  return headers;
};

export const api = {
  get: async (endpoint) => {
    const response = await fetch(`${BASE_URL}${endpoint}`, {
      method: 'GET',
      headers: getHeaders()
    });

    if (!response.ok) {
      const errData = await response.json().catch(() => ({}));
      throw new Error(errData.message || 'Something went wrong');
    }

    return response.json();
  },

  post: async (endpoint, data, isFormData = false) => {
    const response = await fetch(`${BASE_URL}${endpoint}`, {
      method: 'POST',
      headers: getHeaders(isFormData),
      body: isFormData ? data : JSON.stringify(data)
    });

    if (!response.ok) {
      const errData = await response.json().catch(() => ({}));
      throw new Error(errData.message || 'Something went wrong');
    }

    return response.json();
  },

  put: async (endpoint, data, isFormData = false) => {
    const response = await fetch(`${BASE_URL}${endpoint}`, {
      method: 'PUT',
      headers: getHeaders(isFormData),
      body: isFormData ? data : JSON.stringify(data)
    });

    if (!response.ok) {
      const errData = await response.json().catch(() => ({}));
      throw new Error(errData.message || 'Something went wrong');
    }

    return response.json();
  },

  delete: async (endpoint) => {
    const response = await fetch(`${BASE_URL}${endpoint}`, {
      method: 'DELETE',
      headers: getHeaders()
    });

    if (!response.ok) {
      const errData = await response.json().catch(() => ({}));
      throw new Error(errData.message || 'Something went wrong');
    }

    return response.json();
  },

  // Specialized method to download files (like CSV export)
```

```
  download: async (endpoint, filename) => {
    const token = localStorage.getItem('jumhc_admin_token');
    const headers = {};
    if (token) {
      headers['Authorization'] = `Bearer ${token}`;
    }

    const response = await fetch(`${BASE_URL}${endpoint}`, {
      method: 'GET',
      headers
    });

    if (!response.ok) {
      const errData = await response.json().catch(() => ({}));
      throw new Error(errData.message || 'Failed to download file');
    }

    const blob = await response.blob();
    const url = window.URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    a.remove();
    window.URL.revokeObjectURL(url);
  }
};
```

- `BASE_URL` is the one place the backend's address is written down — every call in the app goes through here rather than hard-coding `http://localhost:5000/api` in ten different files.
- `getHeaders()` centralizes two things every request needs: the `Authorization: Bearer <token>` header (read straight out of `localStorage`, only attached if a token actually exists) and the `Content-Type` header (skipped for file uploads, since the browser needs to set its own multipart boundary header for those).
- `api.get / post / put / delete` are thin wrappers around `fetch()` that all follow the same shape: make the request, and if `response.ok` is false, throw a JavaScript `Error` built from the backend's own error message — so every calling page can just wrap a call in `try/catch` and get a readable `err.message`, without re-implementing this check each time.
- `api.download` is a specialised one, used only for the CSV export feature: instead of parsing the response as JSON, it reads it as a `blob`, creates a temporary invisible `<a>` link pointing at that blob, and simulates a click on it — which is the standard browser trick for triggering a file download from JavaScript.

WHY CENTRALIZE THIS INSTEAD OF CALLING FETCH() DIRECTLY IN EACH PAGE?

Every page component in this project (Events, Alumni, Gallery, AdminDashboard, ...) calls something like `api.get('/events')` and just handles the returned data or a thrown error. If the auth-token header logic, or the base URL, or the error-handling convention ever needs to change, it changes in exactly one file instead of everywhere it was duplicated.

13. Frontend, Step 4: Global Login State (AuthContext)

`frontend/src/context/AuthContext.jsx`

Several unrelated parts of the app — the Navbar (to show "Admin Panel"/"Logout" instead of nothing), `ProtectedRoute` (to decide whether to redirect to the login page), and the `AdminDashboard` itself — all need to know the same thing: *is anyone logged in right now, and who?* React's Context API exists exactly for this kind of "many components need the same piece of shared state" situation, so it's built once here rather than passed down as props through every component in between.

```
import React, { createContext, useState, useEffect, useContext } from 'react';
import { api } from '../utils/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [admin, setAdmin] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const checkAuth = async () => {
      const token = localStorage.getItem('jumhc_admin_token');
      if (!token) {
        setLoading(false);
        return;
      }

      try {
        const userData = await api.get('/auth/me');
        setAdmin(userData);
      } catch (err) {
        console.error('Session validation failed:', err);
        localStorage.removeItem('jumhc_admin_token');
        setAdmin(null);
      } finally {
        setLoading(false);
      }
    };

    checkAuth();
  }, []);

  const login = async (email, password) => {
    try {
      const data = await api.post('/auth/login', { email, password });
      localStorage.setItem('jumhc_admin_token', data.token);
      setAdmin(data.admin);
      return data.admin;
    } catch (err) {
      throw new Error(err.message || 'Login failed');
    }
  };

  const logout = () => {
    localStorage.removeItem('jumhc_admin_token');
    setAdmin(null);
  };

  return (
    <AuthContext.Provider value={{ admin, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

- `createContext(null)` creates the context object itself; on its own it does nothing until it's provided and consumed.
- `AuthProvider` is a component that wraps the whole app (you'll see it wrapping `<Router>` in `App.jsx` next) and owns the real state: `admin` (the logged-in admin's data, or `null`) and `loading` (true only while it's still checking).
- Its `useEffect` runs once when the app first loads: if there's a saved token in `localStorage`, it calls `/auth/me` to check the token is still valid and fetch the admin's current data — this is what lets a logged-in admin refresh the page (or come back the next day) without being kicked back to the login screen.
- `login()` and `logout()` are the two actions exposed to the rest of the app: logging in stores the returned token and admin object; logging out just clears both.
- `useAuth()` is a small custom hook that any component can call to read `{ admin, loading, login, logout }` — it also throws a clear error

if it's ever accidentally called outside an `AuthProvider`, which is a much easier bug to spot than a silent `null` reference error later.

WHY STORE THE TOKEN IN LOCALSTORAGE RATHER THAN ONLY IN MEMORY?

State stored purely in a React component (or even in Context) is wiped the instant the browser tab is refreshed or closed. Saving the JWT in `localStorage` means it survives a page reload, so an admin who refreshes the dashboard doesn't get logged out — the `useEffect` above is what re-establishes the React state from that saved token on every fresh page load.

14. Frontend, Step 5: Shared Components

With data-fetching and auth state ready, next come the pieces of UI that appear on *every* page, so they're built once as their own components rather than copy-pasted into each page file.

`frontend/src/components/Navbar.jsx`

```
import React, { useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Menu, X, Compass, ShieldAlert, LogOut } from 'lucide-react';

const Navbar = () => {
  const [isOpen, setIsOpen] = useState(false);
  const { admin, logout } = useAuth();
  const location = useLocation();

  const navLinks = [
    { name: 'Home', path: '/' },
    { name: 'About & History', path: '/about' },
    { name: 'Activities', path: '/activities' },
    { name: 'Alumni', path: '/alumni' },
    { name: 'Gallery', path: '/gallery' },
    { name: 'Events', path: '/events' },
    { name: 'Contact', path: '/contact' }
  ];

  const isActive = (path) => {
    if (path === '/') {
      return location.pathname === '/';
    }
    return location.pathname.startsWith(path);
  };

  return (
    <nav className="sticky top-0 z-50 bg-mountain-900/95 backdrop-blur-md border-b border-mountain-800 text-white shadow-lg transition-all
duration-300">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-20">
          {/* Logo */}
          <Link to="/" className="flex items-center space-x-3 group">
            <Compass className="h-9 w-9 text-earth-400 group-hover:rotate-45 transition-transform duration-500 ease-out" />
            <div className="flex flex-col">
              <span className="font-display font-bold text-lg tracking-wider leading-none text-white">JUMHC</span>
              <span className="text-[10px] text-mountain-300 tracking-widest uppercase mt-1">Established 1966</span>
            </div>
          </Link>

          {/* Desktop Navigation */}
          <div className="hidden md:flex items-center space-x-1 lg:space-x-2">
            {navLinks.map((link) => (
              <Link
                key={link.path}
                to={link.path}
                className={`px-3 py-2 rounded-md text-sm font-medium tracking-wide transition-all duration-200 ${
                  isActive(link.path)
                    ? 'text-earth-400 bg-mountain-800/60'
                    : 'text-mountain-200 hover:text-white hover:bg-mountain-800/30'
                }`}
              >
                {link.name}
              </Link>
            ))}

            {admin && (
              <div className="flex items-center space-x-2 pl-4 border-l border-mountain-800">
                <Link
                  to="/admin"
                  className="flex items-center space-x-1 px-3 py-2 rounded-md text-sm font-bold text-forest-300 hover:bg-forest-950/40 border
border-forest-800/30 transition-all"
                >
                  <ShieldAlert className="w-4 h-4" />
                  <span>Admin Panel</span>
                </Link>
                <button
                  onClick={logout}
                  className="p-2 rounded-md text-mountain-300 hover:text-earth-400 hover:bg-mountain-800/30 transition-all"
                  title="Logout"
                >
                  <LogOut className="w-5 h-5" />
                </button>
              </div>
            )}
          </div>
```

```
          {/* Mobile menu button */}
          <div className="md:hidden flex items-center">
            {admin && (
              <Link to="/admin" className="p-2 mr-2 text-forest-300 hover:bg-mountain-800/30 rounded-md">
                <ShieldAlert className="w-5 h-5" />
              </Link>
            )}
            <button
              onClick={() => setIsOpen(!isOpen)}
              className="inline-flex items-center justify-center p-2 rounded-md text-mountain-300 hover:text-white hover:bg-mountain-800
focus:outline-none"
            >
              {isOpen ? <X className="h-6 w-6" /> : <Menu className="h-6 w-6" />}
            </button>
          </div>
        </div>
      </div>

      {/* Mobile Menu */}
      {isOpen && (
        <div className="md:hidden bg-mountain-950 border-b border-mountain-800 transition-all duration-300 ease-in-out">
          <div className="px-2 pt-2 pb-4 space-y-1 sm:px-3">
            {navLinks.map((link) => (
              <Link
                key={link.path}
                to={link.path}
                onClick={() => setIsOpen(false)}
                className={`block px-3 py-3 rounded-md text-base font-medium transition-all ${
                  isActive(link.path)
                    ? 'text-earth-400 bg-mountain-900'
                    : 'text-mountain-200 hover:text-white hover:bg-mountain-900/50'
                }`}
              >
                {link.name}
              </Link>
            ))}

            {admin && (
              <div className="pt-4 mt-4 border-t border-mountain-800 px-3 flex items-center justify-between">
                <Link
                  to="/admin"
                  onClick={() => setIsOpen(false)}
                  className="flex items-center space-x-2 text-forest-300 font-bold"
                >
                  <ShieldAlert className="w-5 h-5" />
                  <span>Admin Dashboard</span>
                </Link>
                <button
                  onClick={() => {
                    logout();
                    setIsOpen(false);
                  }}
                  className="flex items-center space-x-2 text-earth-400 font-medium"
                >
                  <LogOut className="w-5 h-5" />
                  <span>Logout</span>
                </button>
              </div>
            )}
          </div>
        </div>
      )}
    </nav>
  );
};

export default Navbar;
```

- `navLinks` is a plain array of `{ name, path }` objects, mapped over to render the links — adding a new nav link later just means adding one object to this array, not writing a new `<Link>` by hand.
- `useLocation()` (from react-router-dom) tells the component the current URL, which `isActive()` compares against each link's path to decide which one gets the highlighted "active" styling.
- `const { admin, logout } = useAuth();` is the Navbar consuming the AuthContext built in the last section — it's how the "Admin Panel" button and logout icon only appear at all when someone is actually logged in.
- `isOpen` is purely local UI state (not related to auth) that toggles the mobile hamburger menu open/closed on small screens.

frontend/src/components/Footer.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
import { Compass, Mail, MapPin, Phone, Heart } from 'lucide-react';

const Footer = () => {
  const currentYear = new Date().getFullYear();
```

```
  return (
    <footer className="bg-mountain-950 text-mountain-300 border-t border-mountain-900 pt-16 pb-8">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="grid grid-cols-1 md:grid-cols-4 gap-8 mb-12">
          {/* Col 1: About */}
          <div className="space-y-4">
            <div className="flex items-center space-x-3 text-white">
              <Compass className="h-8 w-8 text-earth-400" />
              <span className="font-display font-bold text-lg tracking-wider">JUMHC</span>
            </div>
            <p className="text-sm leading-relaxed text-mountain-400">
              Established in 1966, Jadavpur University Mountaineering & Hiking Club has fostered a spirit of adventure, outdoor discipline, and
nature conservation for over five decades.
            </p>
          </div>

          {/* Col 2: Quick Links */}
          <div>
            <h3 className="text-white font-display font-semibold text-base mb-4 tracking-wide">Quick Links</h3>
            <ul className="space-y-2 text-sm">
              <li><Link to="/about" className="hover:text-white transition-colors">Our History</Link></li>
              <li><Link to="/activities" className="hover:text-white transition-colors">Club Activities</Link></li>
              <li><Link to="/alumni" className="hover:text-white transition-colors font-medium text-earth-300">Alumni Network</Link></li>
              <li><Link to="/gallery" className="hover:text-white transition-colors">Gallery & Archive</Link></li>
              <li><Link to="/events" className="hover:text-white transition-colors">Upcoming Events</Link></li>
            </ul>
          </div>

          {/* Col 3: Contact Info */}
          <div>
            <h3 className="text-white font-display font-semibold text-base mb-4 tracking-wide">Contact Details</h3>
            <ul className="space-y-3 text-sm">
              <li className="flex items-start space-x-3">
                <MapPin className="w-5 h-5 text-earth-400 shrink-0 mt-0.5" />
                <span>Club Room, O.A.T. Building, Jadavpur University Main Campus, Kolkata - 700032</span>
              </li>
              <li className="flex items-center space-x-3">
                <Phone className="w-4 h-4 text-earth-400" />
                <span>+91 98765 43210</span>
              </li>
              <li className="flex items-center space-x-3">
                <Mail className="w-4 h-4 text-earth-400" />
                <span>contact@jumhc.org</span>
              </li>
            </ul>
          </div>

          {/* Col 4: Map & Socials */}
          <div className="space-y-4">
            <h3 className="text-white font-display font-semibold text-base tracking-wide">Stay Connected</h3>
            <div className="flex space-x-4">
              <a href="https://facebook.com" target="_blank" rel="noreferrer" className="w-10 h-10 rounded-full bg-mountain-900 flex items-
center justify-center text-mountain-300 hover:text-white hover:bg-earth-600 transition-all">
                <svg className="w-5 h-5 fill-current" viewBox="0 0 24 24"><path d="M9 8h-3v4h3v12h5v-12h3.642l.358-4h-4v-1.667c0-.955.192-1.333
1.115-1.333h2.885v-5h-3.808c-3.596 0-5.192 1.583-5.192 4.615v3.385z"/></svg>
              </a>
              <a href="https://instagram.com" target="_blank" rel="noreferrer" className="w-10 h-10 rounded-full bg-mountain-900 flex items-
center justify-center text-mountain-300 hover:text-white hover:bg-earth-600 transition-all">
                <svg className="w-5 h-5 fill-current" viewBox="0 0 24 24"><path d="M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691
4.919 4.919.058 1.265.069 1.645.069 4.849 0 3.205-.012 3.584-.069 4.849-.149 3.225-1.664 4.771-4.919 4.919-1.266.058-1.644.07-4.85.07-3.204 0-
3.584-.012-4.849-.07-3.26-.149-4.771-1.699-4.919-4.92-.058-1.265-.07-1.644-.07-4.849 0-3.204.013-3.583.07-4.849.149-3.227 1.664-4.771 4.919-
4.919 1.266-.057 1.645-.069 4.849-.069zm0-2c-3.259 0-3.667.014-4.947.072-4.358.2-6.78 2.618-6.98 6.98-.059 1.281-.073 1.689-.073 4.948 0
3.259.014 3.668.072 4.948.2 4.358 2.618 6.78 6.98 6.98 1.281.058 1.689.072 4.948.072 3.259 0 3.668-.014 4.948-.072 4.354-.2 6.782-2.618 6.979-
6.98.059-1.28.073-1.689.073-4.948 0-3.259-.014-3.667-.072-4.947-.196-4.354-2.617-6.78-6.979-6.98-1.281-.059-1.69-.073-4.949-.073zm0 5.838c-3.403
0-6.162 2.759-6.162 6.162s2.759 6.163 6.162 6.163 6.162-2.759 6.162-6.163c0-3.403-2.759-6.162-6.162-6.162zm0 10.162c-2.209 0-4-1.79-4-4 0-2.209
1.791-4 4-4s4 1.791 4 4c0 2.21-1.791 4-4 4zm6.406-11.845c-.796 0-1.441.645-1.441 1.44s.645 1.44 1.441 1.44c.795 0 1.439-.645 1.439-1.44s-.644-
1.44-1.439-1.44z"/></svg>
              </a>
              <a href="https://youtube.com" target="_blank" rel="noreferrer" className="w-10 h-10 rounded-full bg-mountain-900 flex items-center
justify-center text-mountain-300 hover:text-white hover:bg-earth-600 transition-all">
                <svg className="w-5 h-5 fill-current" viewBox="0 0 24 24"><path d="M23.498 6.163gM23.498 6.163c-.272-1.022-1.074-1.826-2.099-
2.099C19.558 3.5 12 3.5 12 3.5s-7.558 0-9.399.564C1.576 4.337.774 5.141.502 6.163.003 8.003 0 12 0 12s.003 3.997.502 5.837c.272 1.022 1.074
1.826 2.099 2.099C4.442 20.5 12 20.5 12 20.5s7.558 0 9.399-.564c1.025-.273 1.827-1.077 2.099-2.099.5-1.84 0-5.837 0-5.837s-.003-3.997-.502-
5.837zM9.545 15.568V8.432L15.818 12l-6.273 3.568z"/></svg>
              </a>
            </div>
            <p className="text-[11px] text-mountain-500 font-medium uppercase tracking-widest pt-2">
              Jadavpur University Main Campus
            </p>
          </div>
        </div>

        {/* Footer Bottom */}
        <div className="border-t border-mountain-900 pt-8 flex flex-col md:flex-row items-center justify-between text-xs text-mountain-500">
          <div className="flex items-center space-x-1 mb-4 md:mb-0">
            <span>&copy; {currentYear} JU Mountaineering & Hiking Club. All rights reserved. Made with</span>
            <Heart className="w-3 h-3 text-earth-500 fill-earth-500" />
            <span>for the outdoors.</span>
          </div>
```

```
          <div className="flex space-x-4">
            <Link to="/admin/login" className="hover:text-white transition-colors">Admin Login</Link>
            <span className="text-mountain-800">|</span>
            <a href="https://www.jaduniv.edu.in/" target="_blank" rel="noreferrer" className="hover:text-white transition-colors">Jadavpur
University</a>
          </div>
        </div>
      </div>
    </footer>
  );
};

export default Footer;
```

A largely static component: contact details, quick links, and social icons hard-coded as inline SVG paths (so no extra icon library calls are needed just for three social logos). The one dynamic bit is `currentYear = new Date().getFullYear()`, so the copyright line never needs to be manually updated year to year.

frontend/src/components/ProtectedRoute.jsx

```
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const ProtectedRoute = ({ children }) => {
  const { admin, loading } = useAuth();

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-mountain-50">
        <div className="flex flex-col items-center">
          <div className="w-12 h-12 border-4 border-mountain-600 border-t-transparent rounded-full animate-spin"></div>
          <p className="mt-4 text-mountain-600 font-medium">Verifying credentials...</p>
        </div>
      </div>
    );
  }

  if (!admin) {
    return <Navigate to="/admin/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

This is the gatekeeper for the frontend, mirroring what `backend/middleware/auth.js` does for the API. It wraps around whatever page needs protecting (just the AdminDashboard, in this project) and checks `useAuth()`'s state:

- While `loading` is true (AuthContext is still checking a saved token), it shows a spinner rather than flashing the login page for a split second.
- If loading has finished and there's still no `admin`, it renders `<Navigate to="/admin/login" />`, which react-router-dom treats as an instant redirect.
- Only if someone *is* logged in does it render `{children}` — whatever page was actually wrapped in `<ProtectedRoute>`.

frontend/src/components/Timeline.jsx

A self-contained, reusable component embedded inside the About page — it fetches and displays the secretaries' timeline entirely on its own, so `About.jsx` doesn't need to know anything about how that data is fetched, searched, or sorted.

```
import React, { useState, useEffect } from 'react';
import { api } from '../utils/api';
import { Search, ArrowUpDown, Compass, Award } from 'lucide-react';

const Timeline = () => {
  const [secretaries, setSecretaries] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState('');
  const [sortOrder, setSortOrder] = useState('desc'); // 'desc' = newest first, 'asc' = oldest first

  useEffect(() => {
    const fetchSecretaries = async () => {
      try {
        const data = await api.get('/secretaries');
        setSecretaries(data);
      } catch (err) {
        console.error('Error fetching secretaries:', err);
        setError('Failed to load secretaries timeline.');
      } finally {
        setLoading(false);
      }
```

```
    };

    fetchSecretaries();
  }, []);

  const handleSearchChange = (e) => {
    setSearchTerm(e.target.value);
  };

  const toggleSortOrder = () => {
    setSortOrder(prev => prev === 'asc' ? 'desc' : 'asc');
  };

  // Filter & Sort secretaries
  const filteredSecretaries = secretaries
    .filter(sec => {
      const matchName = sec.name.toLowerCase().includes(searchTerm.toLowerCase());
      const matchYear = sec.yearStart.toString().includes(searchTerm) ||
                        sec.yearEnd.toString().includes(searchTerm);
      const matchNote = sec.note ? sec.note.toLowerCase().includes(searchTerm.toLowerCase()) : false;
      return matchName || matchYear || matchNote;
    })
    .sort((a, b) => {
      if (sortOrder === 'asc') {
        return a.yearStart - b.yearStart;
      } else {
        return b.yearStart - a.yearStart;
      }
    });

  if (loading) {
    return (
      <div className="flex flex-col items-center justify-center py-16">
        <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
        <p className="mt-4 text-mountain-500 font-medium">Loading Secretaries Timeline...</p>
      </div>
    );
  }

  if (error) {
    return (
      <div className="text-center py-12 text-earth-700 bg-earth-50 rounded-xl border border-earth-100">
        <p className="font-semibold">{error}</p>
      </div>
    );
  }

  return (
    <div className="space-y-8">
      {/* Search & Sort Controls */}
      <div className="flex flex-col sm:flex-row items-center justify-between gap-4 bg-white p-4 rounded-xl shadow-sm border border-mountain-
100">
        <div className="relative w-full sm:w-80">
          <Search className="absolute left-3 top-2.5 h-5 w-5 text-mountain-400" />
          <input
            type="text"
            placeholder="Search by name, year, or note..."
            value={searchTerm}
            onChange={handleSearchChange}
            className="pl-10 pr-4 py-2 w-full rounded-lg border border-mountain-200 focus:outline-none focus:ring-2 focus:ring-mountain-600
focus:border-transparent text-sm bg-mountain-50/50"
          />
        </div>

        <button
          onClick={toggleSortOrder}
          className="flex items-center space-x-2 text-sm font-semibold text-mountain-700 hover:text-mountain-900 bg-mountain-100 hover:bg-
mountain-200/70 px-4 py-2.5 rounded-lg transition-colors w-full sm:w-auto justify-center"
        >
          <ArrowUpDown className="w-4 h-4" />
          <span>{sortOrder === 'desc' ? 'Newest First' : 'Oldest First'}</span>
        </button>
      </div>

      {filteredSecretaries.length === 0 ? (
        <div className="text-center py-12 text-mountain-500 bg-white rounded-xl border border-mountain-100">
          No records match your search criteria.
        </div>
      ) : (
        /* Timeline Container */
        <div className="relative border-l-2 border-mountain-200 ml-4 md:ml-32 space-y-12">
          {filteredSecretaries.map((sec, idx) => (
            <div key={sec._id} className="relative pl-8 md:pl-0 group">
              {/* Timeline Bullet Node */}
              <div className="absolute -left-[9px] top-1.5 w-4.5 h-4.5 rounded-full border-4 border-white bg-earth-600 ring-4 ring-earth-100
group-hover:scale-125 transition-transform duration-300"></div>

              {/* Grid block */}
              <div className="grid grid-cols-1 md:grid-cols-12 gap-4">
```

```
                {/* Year Label Column (Only visible on MD screens onwards, aligned left) */}
                <div className="hidden md:block md:col-span-3 text-right pr-8 pt-0.5">
                  <span className="font-display font-extrabold text-xl text-mountain-900 tracking-tight">
                    {sec.yearStart} – {sec.yearEnd}
                  </span>
                </div>

                {/* Card Column */}
                <div className="md:col-span-9 bg-white p-5 rounded-xl border border-mountain-100 shadow-sm hover:shadow-md hover:border-
mountain-200 transition-all duration-300 -mt-1 md:-ml-0.5 relative">
                  {/* Year display for mobile */}
                  <span className="md:hidden inline-block text-sm font-bold text-earth-700 mb-2">
                    Term: {sec.yearStart} – {sec.yearEnd}
                  </span>

                  <h4 className="font-display text-lg font-bold text-mountain-900 flex items-center space-x-2">
                    <Compass className="w-4 h-4 text-mountain-400 group-hover:text-earth-600 transition-colors" />
                    <span>{sec.name}</span>
                  </h4>

                  {sec.note && (
                    <p className="mt-2 text-sm text-mountain-600 border-t border-mountain-50 pt-2 flex items-start space-x-2 italic leading-
relaxed">
                      <Award className="w-4.5 h-4.5 text-forest-600 shrink-0 mt-0.5" />
                      <span>{sec.note}</span>
                    </p>
                  )}
                </div>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
};

export default Timeline;
```

- It manages its own `useEffect` fetch of `/secretaries`, independent of whatever page happens to render it.
- `searchTerm` and `sortOrder` are local UI state driving client-side filtering/sorting — the search box checks name, year, *and* note in one pass, and the sort toggle simply flips the comparison direction in `.sort((a, b) => …)`.
- The actual timeline visual (the vertical line with dots) is built entirely with Tailwind utility classes — a left border on the wrapping `<div>` plus an absolutely-positioned small circle (`absolute -left-[9px]…`) per entry, rather than any special "timeline" library.

15. Frontend, Step 6: The Router (App.jsx)

`frontend/src/App.jsx`

Every piece built so far — AuthProvider, Navbar, Footer, ProtectedRoute — comes together in this one file, which also decides which page component to show for each URL.

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import ProtectedRoute from './components/ProtectedRoute';

// Public Pages
import Home from './pages/Home';
import About from './pages/About';
import Activities from './pages/Activities';
import Alumni from './pages/Alumni';
import Gallery from './pages/Gallery';
import Events from './pages/Events';
import CourseRegistration from './pages/CourseRegistration';
import Contact from './pages/Contact';

// Admin Pages
import AdminLogin from './pages/AdminLogin';
import AdminDashboard from './pages/AdminDashboard';

function App() {
  return (
    <AuthProvider>
      <Router>
        <div className="flex flex-col min-h-screen">
          {/* Main Navigation */}
          <Navbar />

          {/* Page Contents */}
          <main className="flex-grow">
            <Routes>
              {/* Public Routes */}
              <Route path="/" element={<Home />} />
              <Route path="/about" element={<About />} />
              <Route path="/activities" element={<Activities />} />
              <Route path="/alumni" element={<Alumni />} />
              <Route path="/gallery" element={<Gallery />} />
              <Route path="/events" element={<Events />} />
              <Route path="/register/:eventId" element={<CourseRegistration />} />
              <Route path="/contact" element={<Contact />} />

              {/* Admin Auth Routes */}
              <Route path="/admin/login" element={<AdminLogin />} />

              {/* Protected Admin Routes */}
              <Route
                path="/admin"
                element={
                  <ProtectedRoute>
                    <AdminDashboard />
                  </ProtectedRoute>
                }
              />

              {/* Fallback route */}
              <Route path="*" element={<Navigate to="/" replace />} />
            </Routes>
          </main>

          {/* Footer */}
          <Footer />
        </div>
      </Router>
    </AuthProvider>
  );
}

export default App;
```

- `<AuthProvider>` wraps literally everything, so every component anywhere in the tree, including ones rendered inside routes, can call `useAuth()`.
- `<Router>` (react-router-dom's `BrowserRouter`) enables URL-based navigation without full page reloads.
- `<Navbar />` and `<Footer />` sit *outside* the `<Routes>` block — that's why they appear identically on every single page, while only the content in between (inside `<main>`) changes as you navigate.

- Each `<Route path="…" element={<Page />} />` is a one-line mapping from a URL to a component — adding a whole new page to the site later means writing the page component and adding exactly one line here.
- `/register/:eventId` uses a URL parameter (`:eventId`) — whatever value appears in that position of the URL becomes readable inside `CourseRegistration.jsx` via `useParams()`, which is how one single registration-form page can serve every different event without needing a separate page per event.
- The `/admin` route is the only one wrapped in `<ProtectedRoute><AdminDashboard /></ProtectedRoute>` — every other route is intentionally left open to anyone.
- The final `<Route path="*" element={<Navigate to="/" replace />} />` is a catch-all: any URL that doesn't match one of the routes above (a typo, an old bookmark, etc.) gets silently redirected back to the homepage instead of showing a broken page.

16. Frontend, Step 7: The Public Pages

Every page below lives in `frontend/src/pages/` and is wired into `App.jsx` exactly as shown in the previous section. Most of them share one recurring shape: `useState` for the fetched data + a loading flag, a `useEffect` that calls `api.get(…)` once on mount, and JSX that renders a spinner while loading and the real content once the data arrives. Once you recognise that shape, most of what's left to explain per page is just *what makes that page different*.

frontend/src/pages/Home.jsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../utils/api';
import { Compass, Calendar, Users, Award, ArrowRight } from 'lucide-react';

const Home = () => {
  const [featuredEvent, setFeaturedEvent] = useState(null);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const events = await api.get('/events');
        // Find the Feb 2027 course specifically, or fallback to the first open course/event
        const course = events.find(
          e => e.type === 'course' && e.title.includes('2027') && e.registrationOpen
        ) || events.find(e => e.registrationOpen);
        setFeaturedEvent(course);
      } catch (err) {
        console.error('Error fetching featured event:', err);
      }
    };
    fetchEvents();
  }, []);

  const stats = [
    { label: 'Established', value: '1966', icon: Compass },
    { label: 'Expeditions', value: '60+ Years', icon: Calendar },
    { label: 'Alumni Network', value: '500+ Members', icon: Users },
    { label: 'Certified Courses', value: '60 Annual', icon: Award },
  ];

  return (
    <div className="flex flex-col min-h-screen">
      {/* Hero Section */}
      <section className="relative h-[85vh] flex items-center justify-center text-white overflow-hidden">
        {/* Background Image with overlay */}
        <div className="absolute inset-0 bg-cover bg-center bg-no-repeat" style={{ backgroundImage: "url('https://images.unsplash.com/photo-
1464822759023-fed622ff2c3b?auto=format&fit=crop&q=80&w=1920')" }}>
          <div className="absolute inset-0 bg-gradient-to-r from-mountain-950/90 via-mountain-900/70 to-transparent"></div>
        </div>

        <div className="relative z-10 max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 w-full text-left">
          <div className="max-w-2xl">
            <span className="inline-block px-3 py-1 bg-earth-600 text-white text-xs font-bold uppercase tracking-widest rounded-full mb-6">
              Jadavpur University
            </span>
            <h1 className="text-4xl sm:text-6xl font-display font-extrabold tracking-tight text-white mb-6 leading-tight">
              Mountaineering & Hiking Club
            </h1>
            <p className="text-lg sm:text-xl text-mountain-200 mb-8 leading-relaxed font-light">
              Fostering the spirit of adventure, outdoor resilience, and technical climbing expertise in Jadavpur University since 1966.
            </p>
            <div className="flex flex-col sm:flex-row gap-4">
              {featuredEvent ? (
                <Link
                  to={`/register/${featuredEvent._id}`}
                  className="inline-flex items-center justify-center px-6 py-4 bg-earth-600 hover:bg-earth-700 text-white font-bold rounded-lg
shadow-lg hover:shadow-xl transition-all duration-200 group"
                >
                  <span>Register for Feb 2027 Course</span>
                  <ArrowRight className="w-5 h-5 ml-2 group-hover:translate-x-1 transition-transform" />
                </Link>
              ) : (
                <Link
                  to="/events"
                  className="inline-flex items-center justify-center px-6 py-4 bg-earth-600 hover:bg-earth-700 text-white font-bold rounded-lg
shadow-lg hover:shadow-xl transition-all duration-200 group"
                >
                  <span>View Upcoming Events</span>
                  <ArrowRight className="w-5 h-5 ml-2 group-hover:translate-x-1 transition-transform" />
                </Link>
              )}
              <Link
                to="/about"
                className="inline-flex items-center justify-center px-6 py-4 bg-white/10 hover:bg-white/20 text-white font-semibold rounded-lg
backdrop-blur-sm border border-white/20 transition-all duration-200"
```

```
              >
                Discover Our Story
              </Link>
            </div>
          </div>
        </div>

        {/* Diagonal Wave Bottom */}
        <div className="absolute bottom-0 left-0 right-0 h-16 bg-gradient-to-t from-mountain-50 to-transparent pointer-events-none"></div>
      </section>

      {/* Stats Section */}
      <section className="relative z-20 -mt-16 max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 w-full">
        <div className="grid grid-cols-2 lg:grid-cols-4 gap-4 sm:gap-6 bg-white p-6 sm:p-8 rounded-2xl shadow-xl border border-mountain-100">
          {stats.map((stat, idx) => {
            const Icon = stat.icon;
            return (
              <div key={idx} className="flex flex-col items-center text-center p-4 rounded-xl hover:bg-mountain-50/50 transition-all group">
                <div className="p-3 bg-mountain-100 rounded-lg text-mountain-900 group-hover:bg-mountain-900 group-hover:text-white transition-
all duration-300">
                  <Icon className="w-6 h-6" />
                </div>
                <span className="text-2xl sm:text-3xl font-display font-extrabold text-mountain-900 mt-4">
                  {stat.value}
                </span>
                <span className="text-xs sm:text-sm text-mountain-500 uppercase tracking-wider font-semibold mt-1">
                  {stat.label}
                </span>
              </div>
            );
          })}
        </div>
      </section>

      {/* Intro section */}
      <section className="py-20 bg-mountain-50">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-12 items-center">
            <div className="lg:col-span-7 space-y-6">
              <h2 className="text-3xl sm:text-4xl font-display font-bold text-mountain-900 leading-tight">
                Welcome to Jadavpur University's Oldest Adventure Club
              </h2>
              <div className="w-20 h-1.5 bg-earth-600 rounded-full"></div>
              <p className="text-mountain-600 leading-relaxed text-base">
                Since our inception in 1966, JUMHC has been a sanctuary for those who seek the mountains. We train students in bouldering,
artificial wall climbing, and natural rock craft, and lead expeditions to some of the highest peaks in India.
              </p>
              <p className="text-mountain-600 leading-relaxed text-base">
                Under the guidance of certified instructors and experienced alumni, we maintain a legacy of safe, disciplined, and rigorous
training. Every winter, our legendary Susunia Hills Rock Climbing Course challenges students to push their physical limits and discover the
heights.
              </p>
              <div className="pt-4">
                <Link
                  to="/activities"
                  className="inline-flex items-center text-earth-700 hover:text-earth-800 font-bold group"
                >
                  <span>Explore our training activities</span>
                  <ArrowRight className="w-4 h-4 ml-2 group-hover:translate-x-1 transition-transform" />
                </Link>
              </div>
            </div>

            <div className="lg:col-span-5 relative">
              {/* Floating photo grid */}
              <div className="relative rounded-2xl overflow-hidden shadow-2xl aspect-[4/3] group">
                <img
                  src="https://images.unsplash.com/photo-1522163182402-834f871fd851?auto=format&fit=crop&q=80&w=800"
                  alt="JU trainees climbing natural rock face"
                  className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
                />
                <div className="absolute inset-0 bg-gradient-to-t from-mountain-950/60 to-transparent flex items-end p-6">
                  <p className="text-white text-sm font-medium">Weekly training camps at Susunia Hills, Purulia</p>
                </div>
              </div>
            </div>
          </div>
        </div>
      </section>

      {/* CTA Box */}
      <section className="py-12 bg-mountain-900 text-white relative overflow-hidden">
        <div className="absolute inset-0 opacity-10 bg-cover bg-center" style={{ backgroundImage: "url('https://images.unsplash.com/photo-
1501555088652-021faa106b9b?auto=format&fit=crop&q=80&w=800')" }}></div>
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 relative z-10">
          <div className="bg-mountain-950/60 rounded-2xl p-8 sm:p-12 border border-mountain-800 flex flex-col lg:flex-row items-center justify-
between gap-8">
            <div className="space-y-4 text-center lg:text-left">
              <span className="px-3 py-1 bg-forest-900 text-forest-300 border border-forest-800/40 text-xs font-bold rounded-full uppercase
```

```
tracking-wider">
                Registration Open
              </span>
              <h3 className="text-2xl sm:text-3xl font-display font-bold">
                60th Annual Rock Climbing Course (Feb 2027)
              </h3>
              <p className="text-mountain-300 max-w-xl text-sm sm:text-base leading-relaxed">
                Join our certified course this February. Learn natural rock climbing, rappelling, rope management, and wilderness survival under
high-altitude specialists. Limited to 40 seats.
              </p>
            </div>
            <div className="shrink-0">
              {featuredEvent ? (
                <Link
                  to={`/register/${featuredEvent._id}`}
                  className="inline-flex items-center justify-center px-8 py-4 bg-earth-600 hover:bg-earth-700 text-white font-bold rounded-lg
shadow-lg hover:shadow-xl transition-all duration-200 group w-full sm:w-auto"
                >
                  <span>Apply Now</span>
                  <ArrowRight className="w-5 h-5 ml-2 group-hover:translate-x-1 transition-transform" />
                </Link>
              ) : (
                <Link
                  to="/events"
                  className="inline-flex items-center justify-center px-8 py-4 bg-earth-600 hover:bg-earth-700 text-white font-bold rounded-lg
shadow-lg hover:shadow-xl transition-all duration-200 group w-full sm:w-auto"
                >
                  <span>View Details</span>
                  <ArrowRight className="w-5 h-5 ml-2 group-hover:translate-x-1 transition-transform" />
                </Link>
              )}
            </div>
          </div>
        </div>
      </section>
    </div>
  );
};

export default Home;
```

The landing page: a full-bleed hero banner, a stats strip, an intro section, and a call-to-action box. The one bit of real logic is the `useEffect` that fetches all events and searches for a specific featured course (matching on title/type/open-status) to decide where the "Register" button should link — falling back to a generic "View Upcoming Events" link if no matching event is found, so the homepage never links to a dead page.

frontend/src/pages/About.jsx

```
import React from 'react';
import Timeline from '../components/Timeline';
import { Compass, Award, ShieldAlert, Footprints } from 'lucide-react';

const About = () => {
  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-
100">
            About the Club
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            Our Heritage & Mission
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            Empowering students with high-altitude technical training, building mountain leaders, and preserving the wilderness since 1966.
          </p>
        </div>

        {/* Story Section */}
        <div className="grid grid-cols-1 lg:grid-cols-12 gap-12 items-center mb-24">
          <div className="lg:col-span-6 space-y-6">
            <h2 className="text-2xl sm:text-3xl font-display font-bold text-mountain-900">
              The Legacy of Susunia and the High Himalayas
            </h2>
            <div className="w-16 h-1 bg-earth-600 rounded-full"></div>
            <p className="text-mountain-600 leading-relaxed">
              In 1966, a group of visionary adventure enthusiasts at Jadavpur University came together to form the Mountaineering & Hiking Club.
What started as simple weekend hikes in the nearby hills of Purulia soon evolved into a premier organization sending expeditions to peaks over
6,000 meters in the Himalayas.
            </p>
            <p className="text-mountain-600 leading-relaxed">
              For nearly six decades, the club has conducted its signature Rock Climbing Training Course at Susunia Hills. These courses are
certified and have served as the stepping stone for generations of mountaineers who went on to scale major peaks and represent the university in
```

```
climbing competitions nationwide.
            </p>
          </div>
          <div className="lg:col-span-6">
            <div className="relative rounded-2xl overflow-hidden shadow-xl aspect-video group">
              <img
                src="https://images.unsplash.com/photo-1501555088652-021faa106b9b?auto=format&fit=crop&q=80&w=800"
                alt="JUMHC historic expedition photo"
                className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
              />
              <div className="absolute inset-0 bg-gradient-to-t from-mountain-950/60 to-transparent flex items-end p-6">
                <span className="text-white text-sm font-semibold">Trainees climbing Susunia Hills rock face</span>
              </div>
            </div>
          </div>
        </div>

        {/* Mission and Pillars */}
        <div className="bg-white rounded-2xl p-8 sm:p-12 border border-mountain-100 shadow-sm mb-24">
          <div className="text-center max-w-xl mx-auto mb-10 space-y-2">
            <h3 className="text-2xl font-display font-bold text-mountain-900">Our Core Pillars</h3>
            <p className="text-mountain-500 text-sm">The four values driving our activities every year.</p>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8">
            <div className="space-y-3 text-center md:text-left">
              <div className="w-12 h-12 rounded-lg bg-mountain-100 text-mountain-900 flex items-center justify-center mx-auto md:mx-0">
                <Compass className="w-6 h-6" />
              </div>
              <h4 className="font-display font-bold text-base text-mountain-900">Adventure</h4>
              <p className="text-sm text-mountain-600 leading-relaxed">
                Exploring untrodden paths, scaling remote peaks, and learning to manage risks in extreme terrain.
              </p>
            </div>

            <div className="space-y-3 text-center md:text-left">
              <div className="w-12 h-12 rounded-lg bg-mountain-100 text-mountain-900 flex items-center justify-center mx-auto md:mx-0">
                <Award className="w-6 h-6" />
              </div>
              <h4 className="font-display font-bold text-base text-mountain-900">Discipline</h4>
              <p className="text-sm text-mountain-600 leading-relaxed">
                Rigorous physical conditioning, absolute safety adherence, and mastery of rope techniques.
              </p>
            </div>

            <div className="space-y-3 text-center md:text-left">
              <div className="w-12 h-12 rounded-lg bg-mountain-100 text-mountain-900 flex items-center justify-center mx-auto md:mx-0">
                <ShieldAlert className="w-6 h-6" />
              </div>
              <h4 className="font-display font-bold text-base text-mountain-900">Comradeship</h4>
              <p className="text-sm text-mountain-600 leading-relaxed">
                Building lifelong bonds through rope-partner responsibility and collective expedition work.
              </p>
            </div>

            <div className="space-y-3 text-center md:text-left">
              <div className="w-12 h-12 rounded-lg bg-mountain-100 text-mountain-900 flex items-center justify-center mx-auto md:mx-0">
                <Footprints className="w-6 h-6" />
              </div>
              <h4 className="font-display font-bold text-base text-mountain-900">Conservation</h4>
              <p className="text-sm text-mountain-600 leading-relaxed">
                Adhering to "Leave No Trace" camping rules and protecting natural mountain habitats.
              </p>
            </div>
          </div>
        </div>

        {/* Secretaries Timeline Section */}
        <div className="space-y-8">
          <div className="text-center max-w-xl mx-auto space-y-3">
            <h2 className="text-3xl font-display font-bold text-mountain-900">Secretaries Timeline</h2>
            <p className="text-mountain-600 text-sm sm:text-base leading-relaxed">
              Remembering the student leaders who have piloted the club through terms of expeditions, training, and heritage since 1966.
            </p>
          </div>
          <Timeline />
        </div>

      </div>
    </div>
  );
};

export default About;
```

Mostly static heritage/mission content written directly into the JSX, plus one dynamic piece: it renders `<Timeline />` (the component from Section 14) to show the secretaries' history without needing any of its own data-fetching logic.

frontend/src/pages/Activities.jsx

frontend/src/pages/Activities.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
import { Compass, ShieldAlert, Award, Footprints, Target, ArrowRight } from 'lucide-react';

const Activities = () => {
  const categories = [
    {
      title: 'Himalayan & Regional Expeditions',
      icon: Compass,
      image: 'https://images.unsplash.com/photo-1464822759023-fed622ff2c3b?auto=format&fit=crop&q=80&w=800',
      description: 'The club hosts high-altitude expeditions every year, scaling challenging peaks in the Garhwal, Himachal, and Sikkim Himalayas. Trainees who demonstrate advanced fitness and rock skills undergo specialized pre-expedition conditioning camps.',
      details: ['Mt. Jogin peaks', 'Kedarnath Dome ascent', 'Deo Tibba expedition base', 'Annual high-altitude medical fitness']
    },
    {
      title: 'Artificial Wall Climbing',
      icon: Target,
      image: 'https://images.unsplash.com/photo-1512641406448-6574e777bec6?auto=format&fit=crop&q=80&w=800',
      description: 'Our sports complex on campus houses a state-of-the-art artificial bouldering wall. Members train here weekly to master technical holds, footwork, speed climbing, and grip strength under expert supervision.',
      details: ['Weekly bouldering runs', 'Speed & lead climbing setups', 'Campus training nights', 'Safety harnesses and crash pads']
    },
    {
      title: 'Natural Rock Climbing',
      icon: Footprints,
      image: 'https://images.unsplash.com/photo-1522163182402-834f871fd851?auto=format&fit=crop&q=80&w=800',
      description: 'Natural rock training is key to real mountaineering. The club organizes frequent training trips to Susunia Hills, Purulia, teaching climbers how to negotiate cracks, slabs, chimneys, and overhangs on real granite.',
      details: ['Granite crack scaling', 'Rappelling & anchoring setups', 'Susunia Hills weekend camps', 'Susunia granite bouldering']
    },
    {
      title: 'University Climbing Competitions',
      icon: Award,
      image: 'https://images.unsplash.com/photo-1526625303811-1a90f8f25e3d?auto=format&fit=crop&q=80&w=800',
      description: 'JUMHC hosts and competes in inter-university bouldering and speed climbing tournaments. Our athletes have represented Jadavpur University at district, state, and national sport climbing levels, bringing home multiple titles.',
      details: ['All-Bengal Inter-Varsity Cup', 'District sport climbing meets', 'National bouldering categories', 'Speed climb time-trials']
    },
    {
      title: 'Rock Climbing Certification Courses',
      icon: ShieldAlert,
      image: 'https://images.unsplash.com/photo-1501555088652-021faa106b9b?auto=format&fit=crop&q=80&w=800',
      description: 'Our flagship 4-day Susunia Rock Climbing Course is held every winter. Students learn rock climbing from base concepts up to lead work, safety protocols, and camp survival. The course is certified and open to students from all institutions.',
      details: ['Susunia certificate distribution', 'Leave No Trace wilderness guidelines', 'Rigging and rescue drills', 'Pre-conditioned physical test'],
      cta: true
    }
  ];

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-100">
            Our Activities
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            How We Train & Explore
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            From indoor wall practice to high-altitude Himalayan summits, explore the full spectrum of our mountaineering activities.
          </p>
        </div>

        {/* Categories list */}
        <div className="space-y-20">
          {categories.map((cat, idx) => {
            const Icon = cat.icon;
            const isEven = idx % 2 === 0;

            return (
              <div
                key={idx}
                className={`grid grid-cols-1 lg:grid-cols-12 gap-8 items-center border-b border-mountain-200 pb-16 last:border-0 last:pb-0 ${
                  isEven ? '' : 'lg:flex-row-reverse'
                }`}
              >
                {/* Text column */}
                <div className={`lg:col-span-6 space-y-6 ${isEven ? 'lg:order-1' : 'lg:order-2'}`}>
                  <div className="flex items-center space-x-3">
                    <div className="p-2.5 bg-mountain-100 rounded-lg text-mountain-950">
                      <Icon className="w-6 h-6" />
                    </div>
```

```
                  <h2 className="text-2xl sm:text-3xl font-display font-bold text-mountain-900">
                    {cat.title}
                  </h2>
                </div>
                <p className="text-mountain-600 leading-relaxed text-sm sm:text-base">
                  {cat.description}
                </p>

                {/* Bullet checklist */}
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 pt-2">
                  {cat.details.map((detail, dIdx) => (
                    <div key={dIdx} className="flex items-center space-x-2 text-sm text-mountain-700">
                      <span className="w-1.5 h-1.5 rounded-full bg-earth-600 shrink-0"></span>
                      <span>{detail}</span>
                    </div>
                  ))}
                </div>

                {cat.cta && (
                  <div className="pt-4">
                    <Link
                      to="/events"
                      className="inline-flex items-center px-5 py-3 bg-earth-600 hover:bg-earth-700 text-white font-bold rounded-lg shadow-md
hover:shadow-lg transition-all duration-200 group text-sm"
                    >
                      <span>Join the Next Certification Course</span>
                      <ArrowRight className="w-4 h-4 ml-2 group-hover:translate-x-1 transition-transform" />
                    </Link>
                  </div>
                )}
              </div>

              {/* Image column */}
              <div className={`lg:col-span-6 ${isEven ? 'lg:order-2' : 'lg:order-1'}`}>
                <div className="rounded-2xl overflow-hidden shadow-xl aspect-[16/10] relative group">
                  <img
                    src={cat.image}
                    alt={cat.title}
                    className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
                  />
                  <div className="absolute inset-0 bg-mountain-950/20 group-hover:bg-mountain-950/10 transition-colors"></div>
                </div>
              </div>
            </div>
          );
        })}
      </div>

    </div>
  </div>
);
};

export default Activities;
```

Also static — the five activity categories (expeditions, wall climbing, natural rock climbing, competitions, certification courses) live in one `categories` array right in the file, then get mapped into alternating left/right image-and-text rows using `idx % 2 === 0` to flip the layout direction every other item — a common technique for making a long list of similar sections feel visually varied without hand-writing each row separately.

frontend/src/pages/Alumni.jsx

```
import React, { useState, useEffect } from 'react';
import { api } from '../utils/api';
import { Search, UserCheck, Mail, Compass, Star } from 'lucide-react';

const Alumni = () => {
  const [alumni, setAlumni] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState('');
  const [filterActiveInstructors, setFilterActiveInstructors] = useState(false);

  useEffect(() => {
    const fetchAlumni = async () => {
      try {
        const data = await api.get('/alumni');
        setAlumni(data);
      } catch (err) {
        console.error('Error fetching alumni:', err);
        setError('Failed to load alumni profiles.');
      } finally {
        setLoading(false);
      }
    };
    fetchAlumni();
```

```
  }, []);

  const filteredAlumni = alumni.filter(member => {
    const matchesSearch = member.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
                          member.roleHistory.toLowerCase().includes(searchTerm.toLowerCase()) ||
                          (member.bio && member.bio.toLowerCase().includes(searchTerm.toLowerCase()));

    const matchesActiveFilter = !filterActiveInstructors || member.isActiveInstructor;

    return matchesSearch && matchesActiveFilter;
  });

  if (loading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[50vh] bg-mountain-50">
        <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
        <p className="mt-4 text-mountain-500 font-medium">Loading Alumni Network...</p>
      </div>
    );
  }

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-
100">
            Alumni Network
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            Our Supporters & Mentors
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            Meet the veterans, former secretaries, and mountaineers who established our legacy and continue to guide our trainees.
          </p>
        </div>

        {/* Filters */}
        <div className="flex flex-col sm:flex-row items-center justify-between gap-4 bg-white p-4 rounded-xl shadow-sm border border-mountain-
100 mb-10">
          <div className="relative w-full sm:w-80">
            <Search className="absolute left-3 top-2.5 h-5 w-5 text-mountain-400" />
            <input
              type="text"
              placeholder="Search by name, role, bio..."
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
              className="pl-10 pr-4 py-2 w-full rounded-lg border border-mountain-200 focus:outline-none focus:ring-2 focus:ring-mountain-600
focus:border-transparent text-sm bg-mountain-50/50"
            />
          </div>

          <button
            onClick={() => setFilterActiveInstructors(prev => !prev)}
            className={`flex items-center space-x-2 text-sm font-semibold px-4 py-2.5 rounded-lg border transition-all w-full sm:w-auto justify-
center ${
              filterActiveInstructors
                ? 'bg-forest-900 text-forest-300 border-forest-800 shadow-sm'
                : 'bg-mountain-100 text-mountain-700 border-mountain-200 hover:bg-mountain-200/50'
            }`}
          >
            <UserCheck className="w-4.5 h-4.5" />
            <span>Active Instructors Only</span>
          </button>
        </div>

        {error && (
          <div className="text-center py-12 text-earth-700 bg-earth-50 rounded-xl border border-earth-100 mb-10">
            <p className="font-semibold">{error}</p>
          </div>
        )}

        {/* Alumni Grid */}
        {filteredAlumni.length === 0 ? (
          <div className="text-center py-16 bg-white rounded-2xl border border-mountain-100 shadow-sm">
            <p className="text-mountain-500">No alumni matches found in our database.</p>
          </div>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
            {filteredAlumni.map((member) => (
              <div
                key={member._id}
                className="bg-white rounded-2xl overflow-hidden border border-mountain-100 shadow-sm hover:shadow-md hover:-translate-y-1
transition-all duration-300 flex flex-col group relative"
              >
                {/* Header background decoration */}
                <div className="h-16 bg-mountain-900/5 group-hover:bg-mountain-900/10 transition-colors"></div>
```

```
                {/* Profile Photo */}
                <div className="absolute top-4 left-6">
                  <div className="w-20 h-20 rounded-xl overflow-hidden border-4 border-white shadow-md bg-mountain-100 relative">
                    {member.photoUrl ? (
                      <img
                        src={member.photoUrl.startsWith('/uploads/') ? `http://localhost:5000${member.photoUrl}` : member.photoUrl}
                        alt={member.name}
                        className="w-full h-full object-cover"
                        onError={(e) => { e.target.src = 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?
auto=format&fit=crop&q=80&w=200'; }}
                      />
                    ) : (
                      <div className="w-full h-full flex items-center justify-center text-mountain-400">
                        <Compass className="w-8 h-8" />
                      </div>
                    )}
                  </div>
                </div>

                {/* Instructors Badge */}
                {member.isActiveInstructor && (
                  <span className="absolute top-4 right-4 flex items-center space-x-1 px-2.5 py-1 bg-forest-900 text-forest-300 text-[10px]
font-bold rounded-full uppercase tracking-wider border border-forest-800/40">
                    <Star className="w-3 h-3 fill-current" />
                    <span>Active Instructor</span>
                  </span>
                )}

                {/* Content */}
                <div className="px-6 pt-12 pb-6 flex-grow flex flex-col justify-between">
                  <div className="space-y-4">
                    <div>
                      <h3 className="font-display font-bold text-xl text-mountain-900 leading-tight">
                        {member.name}
                      </h3>
                      <p className="text-xs font-semibold text-earth-700 uppercase tracking-widest mt-1">
                        Batch: {member.batchYear}
                      </p>
                    </div>

                    <div className="space-y-2">
                      <div>
                        <span className="text-[10px] uppercase font-bold text-mountain-400 tracking-wider">Role History:</span>
                        <p className="text-xs text-mountain-700 leading-normal font-medium bg-mountain-50 p-2 rounded-lg border border-mountain-
100/50 mt-1">
                          {member.roleHistory}
                        </p>
                      </div>

                      {member.currentContribution && (
                        <div>
                          <span className="text-[10px] uppercase font-bold text-mountain-400 tracking-wider">Current Contribution:</span>
                          <p className="text-xs text-forest-700 leading-normal font-semibold bg-forest-50 p-2 rounded-lg border border-forest-
100/50 mt-1">
                            {member.currentContribution}
                          </p>
                        </div>
                      )}
                    </div>

                    {member.bio && (
                      <p className="text-sm text-mountain-600 leading-relaxed font-light line-clamp-3">
                        {member.bio}
                      </p>
                    )}
                  </div>

                  {/* Connect link in card footer */}
                  <div className="pt-6 border-t border-mountain-50 flex items-center justify-between text-xs text-mountain-400">
                    <span>JU Mountaineering Club</span>
                    <span className="flex items-center text-mountain-500 font-medium">
                      Est. 1966
                    </span>
                  </div>
                </div>
              </div>
            ))}
          </div>
        )}

      </div>
    </div>
  );
};

export default Alumni;
```

Fetches `/alumni` on load, then layers two independent client-side filters on top of the already-downloaded list: a text search (checking name, role history, and bio) and an "Active Instructors Only" toggle. Both filters run together in one `.filter()` call, entirely in the browser — no extra network request is made when you type in the search box or flip the toggle, since all the alumni data is already sitting in the component's state.

frontend/src/pages/Gallery.jsx

```
import React, { useState, useEffect } from 'react';
import { api } from '../utils/api';
import { Search, Image, X, Calendar, Tag, Info } from 'lucide-react';

const Gallery = () => {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState('');
  const [selectedTag, setSelectedTag] = useState('All');
  const [selectedYear, setSelectedYear] = useState('All');
  const [previewItem, setPreviewItem] = useState(null);

  useEffect(() => {
    const fetchGallery = async () => {
      try {
        const data = await api.get('/gallery');
        setItems(data);
      } catch (err) {
        console.error('Error fetching gallery:', err);
        setError('Failed to load gallery items.');
      } finally {
        setLoading(false);
      }
    };
    fetchGallery();
  }, []);

  // Extract unique tags and years for filter dropdowns
  const tags = ['All', ...new Set(items.map(item => item.eventTag))];
  const years = ['All', ...new Set(items.map(item => item.year.toString()))].sort((a, b) => b - a);

  // Filter gallery items
  const filteredItems = items.filter(item => {
    const matchesSearch = item.caption ? item.caption.toLowerCase().includes(searchTerm.toLowerCase()) : false;
    const matchesTag = selectedTag === 'All' || item.eventTag === selectedTag;
    const matchesYear = selectedYear === 'All' || item.year.toString() === selectedYear;

    // If search term is empty, bypass search check
    return (searchTerm === '' || matchesSearch) && matchesTag && matchesYear;
  });

  if (loading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[50vh] bg-mountain-50">
        <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
        <p className="mt-4 text-mountain-500 font-medium">Loading Gallery & Flyers Archive...</p>
      </div>
    );
  }

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-
100">
            Archive & Gallery
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            Expedition Logs & Flyers
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            Browse our historic archive containing scanning copy of old event flyers and photographic logs of high-altitude Himalayan ascents.
          </p>
        </div>

        {/* Filters Controls */}
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4 bg-white p-4 rounded-xl shadow-sm border border-mountain-100 mb-10">
          {/* Search bar */}
          <div className="relative w-full">
            <Search className="absolute left-3 top-2.5 h-5 w-5 text-mountain-400" />
            <input
              type="text"
              placeholder="Search captions..."
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
              className="pl-10 pr-4 py-2 w-full rounded-lg border border-mountain-200 focus:outline-none focus:ring-2 focus:ring-mountain-600
```

```
focus:border-transparent text-sm bg-mountain-50/50"
            />
          </div>

          {/* Tag Filter */}
          <div className="relative flex items-center space-x-2">
            <Tag className="w-4 h-4 text-mountain-400 shrink-0" />
            <select
              value={selectedTag}
              onChange={(e) => setSelectedTag(e.target.value)}
              className="py-2 px-3 w-full rounded-lg border border-mountain-200 focus:outline-none focus:ring-2 focus:ring-mountain-600 text-sm
bg-mountain-50/50"
            >
              {tags.map((tag) => (
                <option key={tag} value={tag}>{tag === 'All' ? 'All Activities' : tag}</option>
              ))}
            </select>
          </div>

          {/* Year Filter */}
          <div className="relative flex items-center space-x-2">
            <Calendar className="w-4 h-4 text-mountain-400 shrink-0" />
            <select
              value={selectedYear}
              onChange={(e) => setSelectedYear(e.target.value)}
              className="py-2 px-3 w-full rounded-lg border border-mountain-200 focus:outline-none focus:ring-2 focus:ring-mountain-600 text-sm
bg-mountain-50/50"
            >
              <option value="All">All Years</option>
              {years.map((year) => (
                <option key={year} value={year}>{year}</option>
              ))}
            </select>
          </div>
        </div>

        {error && (
          <div className="text-center py-12 text-earth-700 bg-earth-50 rounded-xl border border-earth-100 mb-10">
            <p className="font-semibold">{error}</p>
          </div>
        )}

        {/* Gallery Grid */}
        {filteredItems.length === 0 ? (
          <div className="text-center py-16 bg-white rounded-2xl border border-mountain-100 shadow-sm">
            <p className="text-mountain-500">No media matches found in the archive.</p>
          </div>
        ) : (
          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
            {filteredItems.map((item) => (
              <div
                key={item._id}
                onClick={() => setPreviewItem(item)}
                className="bg-white rounded-xl overflow-hidden border border-mountain-100 shadow-sm hover:shadow-md transition-all duration-300
group cursor-pointer"
              >
                {/* Image Wrap */}
                <div className="aspect-[4/3] w-full overflow-hidden bg-mountain-100 relative">
                  <img
                    src={item.imageUrl.startsWith('/uploads/') ? `http://localhost:5000${item.imageUrl}` : item.imageUrl}
                    alt={item.caption || 'Gallery Image'}
                    loading="lazy"
                    className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
                  />
                  <div className="absolute inset-0 bg-mountain-900/0 group-hover:bg-mountain-900/10 transition-colors"></div>

                  {/* Floating Year Tag */}
                  <span className="absolute top-3 left-3 px-2 py-1 bg-mountain-950/80 backdrop-blur-sm text-white text-[10px] font-bold
rounded">
                    {item.year}
                  </span>

                  {/* Activity Tag */}
                  <span className="absolute top-3 right-3 px-2.5 py-1 bg-earth-600 text-white text-[10px] font-bold rounded-full uppercase
tracking-wider">
                    {item.eventTag}
                  </span>
                </div>

                {/* Caption info */}
                {item.caption && (
                  <div className="p-4 border-t border-mountain-50">
                    <p className="text-xs sm:text-sm text-mountain-700 font-medium line-clamp-2 leading-relaxed">
                      {item.caption}
                    </p>
                  </div>
                )}
              </div>
            ))}
```

```
          </div>
        )}

        {/* Lightbox Preview Modal */}
        {previewItem && (
          <div className="fixed inset-0 z-[100] flex items-center justify-center bg-mountain-950/95 p-4 backdrop-blur-sm">
            {/* Click outside target */}
            <div className="absolute inset-0 cursor-zoom-out" onClick={() => setPreviewItem(null)}></div>

            {/* Modal Box */}
            <div className="relative max-w-4xl w-full bg-mountain-900 rounded-2xl overflow-hidden shadow-2xl z-10 border border-mountain-800">
              {/* Close Button */}
              <button
                onClick={() => setPreviewItem(null)}
                className="absolute top-4 right-4 z-20 p-2 bg-mountain-950/80 hover:bg-mountain-950 text-white rounded-full transition-all
focus:outline-none"
              >
                <X className="w-5 h-5" />
              </button>

              {/* Photo Area */}
              <div className="flex flex-col md:flex-row min-h-[50vh]">
                <div className="flex-grow bg-black flex items-center justify-center aspect-[4/3] md:aspect-auto">
                  <img
                    src={previewItem.imageUrl.startsWith('/uploads/') ? `http://localhost:5000${previewItem.imageUrl}` : previewItem.imageUrl}
                    alt={previewItem.caption || 'Preview'}
                    className="max-h-[75vh] max-w-full object-contain"
                  />
                </div>

                {/* Description Column */}
                <div className="w-full md:w-80 bg-mountain-950 p-6 flex flex-col justify-between border-t md:border-t-0 md:border-l border-
mountain-800 text-white">
                  <div className="space-y-4">
                    <div className="flex items-center space-x-2">
                      <span className="px-2 py-0.5 bg-mountain-800 text-mountain-200 text-xs font-bold rounded">
                        Year: {previewItem.year}
                      </span>
                      <span className="px-2.5 py-0.5 bg-earth-600 text-white text-[10px] font-bold rounded-full uppercase tracking-wider">
                        {previewItem.eventTag}
                      </span>
                    </div>

                    <h4 className="font-display font-bold text-lg text-white border-b border-mountain-800 pb-3 flex items-center space-x-2">
                      <Image className="w-4 h-4 text-earth-400" />
                      <span>Archive Record</span>
                    </h4>

                    {previewItem.caption ? (
                      <p className="text-sm text-mountain-300 leading-relaxed font-light">
                        {previewItem.caption}
                      </p>
                    ) : (
                      <p className="text-sm text-mountain-500 italic">No caption available.</p>
                    )}
                  </div>

                  <div className="pt-6 text-[10px] text-mountain-500 border-t border-mountain-800 mt-6 flex items-center space-x-1">
                    <Info className="w-3 h-3" />
                    <span>JU Mountaineering & Hiking Club (Est. 1966)</span>
                  </div>
                </div>
              </div>
            </div>
          </div>
        )}

      </div>
    </div>
  );
};

export default Gallery;
```

Similar filtering pattern to the Alumni page (search by caption, plus dropdown filters for tag and year, with the dropdown options themselves generated dynamically from whatever tags/years actually exist in the fetched data via `new Set(…)`). The one new idea here is the lightbox: clicking a photo sets `previewItem` to that item, which conditionally renders a full-screen modal on top of everything else; clicking the backdrop or the close button just sets it back to `null`.

frontend/src/pages/Events.jsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../utils/api';
import { Calendar, MapPin, Tag, Compass, Users, Clock, ArrowRight } from 'lucide-react';
```

```
const Events = () => {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const data = await api.get('/events');
        setEvents(data);
      } catch (err) {
        console.error('Error fetching events:', err);
        setError('Failed to fetch events list.');
      } finally {
        setLoading(false);
      }
    };
    fetchEvents();
  }, []);

  const getEventStatus = (event) => {
    if (!event.registrationOpen) return 'Closed';
    if (event.deadline && new Date() > new Date(event.deadline)) return 'Closed';
    if (event.capacity !== null && event.registrationsCount >= event.capacity) return 'Full';
    return 'Open';
  };

  const getStatusBadge = (status) => {
    switch (status) {
      case 'Open':
        return (
          <span className="px-3 py-1 bg-forest-900 text-forest-300 text-xs font-bold rounded-full uppercase tracking-wider border border-forest-
800/40">
            Open
          </span>
        );
      case 'Full':
        return (
          <span className="px-3 py-1 bg-earth-900 text-earth-300 text-xs font-bold rounded-full uppercase tracking-wider border border-earth-
800/40">
            Full
          </span>
        );
      case 'Closed':
      default:
        return (
          <span className="px-3 py-1 bg-mountain-800 text-mountain-300 text-xs font-bold rounded-full uppercase tracking-wider border border-
mountain-700/40">
            Closed
          </span>
        );
    }
  };

  const today = new Date();
  const upcomingEvents = events.filter(e => new Date(e.date) >= today);
  const pastEvents = events.filter(e => new Date(e.date) < today).sort((a, b) => new Date(b.date) - new Date(a.date));

  if (loading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[50vh] bg-mountain-50">
        <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
        <p className="mt-4 text-mountain-500 font-medium">Loading Events & Courses...</p>
      </div>
    );
  }

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-
100">
            Club Events
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            Upcoming Courses & Archives
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            Register for our next certification camps, join upcoming climbs, or browse the archives of our historical trips.
          </p>
        </div>

        {error && (
          <div className="text-center py-12 text-earth-700 bg-earth-50 rounded-xl border border-earth-100 mb-10">
            <p className="font-semibold">{error}</p>
          </div>
```

```
        )}

        {/* Section 1: Upcoming Events */}
        <div className="mb-20">
          <h2 className="text-2xl sm:text-3xl font-display font-bold text-mountain-900 mb-8 flex items-center space-x-3">
            <Clock className="w-6 h-6 text-earth-600" />
            <span>Upcoming Activities</span>
          </h2>

          {upcomingEvents.length === 0 ? (
            <div className="text-center py-12 bg-white rounded-2xl border border-mountain-100 shadow-sm">
              <p className="text-mountain-500">No upcoming events listed at this time. Check back soon!</p>
            </div>
          ) : (
            <div className="space-y-6">
              {upcomingEvents.map((event) => {
                const status = getEventStatus(event);
                const seatsLeft = event.capacity !== null ? event.capacity - event.registrationsCount : null;

                return (
                  <div
                    key={event._id}
                    className="bg-white rounded-2xl p-6 sm:p-8 border border-mountain-100 shadow-sm hover:shadow-md hover:border-mountain-200
transition-all duration-300 grid grid-cols-1 lg:grid-cols-12 gap-6 items-center"
                  >
                    {/* Column 1: Info */}
                    <div className="lg:col-span-8 space-y-4">
                      <div className="flex flex-wrap items-center gap-3">
                        <span className="px-2.5 py-0.5 bg-mountain-100 text-mountain-700 text-[10px] font-bold rounded uppercase tracking-
wider">
                          {event.type}
                        </span>
                        {getStatusBadge(status)}
                      </div>

                      <h3 className="font-display font-bold text-xl sm:text-2xl text-mountain-900">
                        {event.title}
                      </h3>

                      <p className="text-mountain-600 text-sm sm:text-base leading-relaxed">
                        {event.description}
                      </p>

                      <div className="flex flex-wrap gap-x-6 gap-y-2 text-xs sm:text-sm text-mountain-500">
                        <div className="flex items-center space-x-2">
                          <Calendar className="w-4.5 h-4.5 text-mountain-400" />
                          <span>{new Date(event.date).toLocaleDateString(undefined, { weekday: 'long', year: 'numeric', month: 'long', day:
'numeric' })}</span>
                        </div>
                        <div className="flex items-center space-x-2">
                          <MapPin className="w-4.5 h-4.5 text-mountain-400" />
                          <span>{event.location}</span>
                        </div>
                      </div>
                    </div>

                    {/* Column 2: Status / Registration CTA */}
                    <div className="lg:col-span-4 lg:border-l lg:border-mountain-100 lg:pl-8 flex flex-col justify-center space-y-4 h-full">
                      {event.capacity !== null && (
                        <div className="space-y-1.5">
                          <div className="flex justify-between items-center text-xs font-semibold text-mountain-500">
                            <span className="flex items-center space-x-1">
                              <Users className="w-3.5 h-3.5" />
                              <span>Enrolled</span>
                            </span>
                            <span>{event.registrationsCount} / {event.capacity} seats</span>
                          </div>

                          {/* Progress bar */}
                          <div className="w-full bg-mountain-100 h-2 rounded-full overflow-hidden">
                            <div
                              className={`h-full rounded-full transition-all duration-500 ${
                                status === 'Full' ? 'bg-earth-600' : 'bg-forest-600'
                              }`}
                              style={{ width: `${Math.min(100, (event.registrationsCount / event.capacity) * 100)}%` }}
                            ></div>
                          </div>

                          {status === 'Open' && (
                            <p className="text-[11px] font-bold text-forest-700">
                              {seatsLeft} seat{seatsLeft > 1 ? 's' : ''} left!
                            </p>
                          )}
                        </div>
                      )}

                      {event.deadline && (
                        <p className="text-[11px] text-mountain-400 font-medium">
                          Deadline: {new Date(event.deadline).toLocaleDateString()}
```

```
                      </p>
                    )}

                    <div>
                      {status === 'Open' ? (
                        <Link
                          to={`/register/${event._id}`}
                          className="w-full inline-flex items-center justify-center px-5 py-3 bg-earth-600 hover:bg-earth-700 text-white font-
bold rounded-lg shadow shadow-earth-500/10 hover:shadow-lg transition-all duration-200 group text-sm"
                        >
                          <span>Register Online</span>
                          <ArrowRight className="w-4 h-4 ml-2 group-hover:translate-x-1 transition-transform" />
                        </Link>
                      ) : (
                        <button
                          disabled
                          className="w-full cursor-not-allowed inline-flex items-center justify-center px-5 py-3 bg-mountain-100 text-
mountain-400 font-bold rounded-lg border border-mountain-200/50 text-sm"
                        >
                          {status === 'Full' ? 'Course Full' : 'Registration Closed'}
                        </button>
                      )}
                    </div>
                  </div>
                </div>
              );
            })}
          </div>
        )}
      </div>

      {/* Section 2: Past Events Archive */}
      <div>
        <h2 className="text-2xl sm:text-3xl font-display font-bold text-mountain-900 mb-8 flex items-center space-x-3">
          <Compass className="w-6 h-6 text-mountain-500" />
          <span>Past Events Archive</span>
        </h2>

        {pastEvents.length === 0 ? (
          <div className="text-center py-8 bg-white rounded-2xl border border-mountain-100 shadow-sm">
            <p className="text-mountain-500">No past events recorded in the archive.</p>
          </div>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            {pastEvents.map((event) => (
              <div
                key={event._id}
                className="bg-white rounded-xl p-6 border border-mountain-100 shadow-sm hover:shadow-md transition-all duration-300 space-y-3"
              >
                <div className="flex items-center justify-between">
                  <span className="px-2 py-0.5 bg-mountain-100 text-mountain-500 text-[10px] font-bold rounded uppercase tracking-wider">
                    {event.type}
                  </span>
                  <span className="text-xs text-mountain-400 font-semibold">
                    {new Date(event.date).getFullYear()}
                  </span>
                </div>

                <h3 className="font-display font-bold text-lg text-mountain-900 line-clamp-1">
                  {event.title}
                </h3>

                <p className="text-mountain-500 text-xs sm:text-sm line-clamp-2 leading-relaxed">
                  {event.description}
                </p>

                <div className="flex items-center space-x-4 pt-2 text-[11px] text-mountain-400 border-t border-mountain-50">
                  <span className="flex items-center space-x-1">
                    <MapPin className="w-3.5 h-3.5" />
                    <span className="line-clamp-1">{event.location}</span>
                  </span>
                </div>
              </div>
            ))}
          </div>
        )}
      </div>

    </div>
  </div>
);
};

export default Events;
```

Fetches all events, then splits them into two lists by comparing each event's date to `today`: upcoming events (shown with full registration details and a progress bar) and past events (shown as a compact archive grid). `getEventStatus()` is a small pure function that turns three

raw fields (`registrationOpen` , `deadline` , `capacity` vs. `registrationsCount`) into one of three simple labels — "Open", "Full", or "Closed" — which `getStatusBadge()` then turns into the right colored badge. Keeping that decision in one function means the badge, the progress-bar color, and whether the "Register" button is disabled can never disagree with each other.

frontend/src/pages/CourseRegistration.jsx

```
import React, { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import { api } from '../utils/api';
import { Calendar, MapPin, CheckCircle2, AlertTriangle, ArrowLeft, Send } from 'lucide-react';

const CourseRegistration = () => {
  const { eventId } = useParams();
  const [event, setEvent] = useState(null);
  const [loadingEvent, setLoadingEvent] = useState(true);
  const [errorEvent, setErrorEvent] = useState(null);

  // Form states
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: '',
    rollNo: '',
    experienceLevel: '',
    emergencyContact: '',
    tShirtSize: ''
  });
  const [submitting, setSubmitting] = useState(false);
  const [success, setSuccess] = useState(false);
  const [submitError, setSubmitError] = useState(null);

  useEffect(() => {
    const fetchEventDetails = async () => {
      try {
        const data = await api.get(`/events/${eventId}`);
        setEvent(data);
      } catch (err) {
        console.error('Error fetching event details:', err);
        setErrorEvent('Failed to load course details. Please verify the link.');
      } finally {
        setLoadingEvent(false);
      }
    };
    fetchEventDetails();
  }, [eventId]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSubmitting(true);
    setSubmitError(null);

    // Client-side validations
    if (!formData.name.trim() || !formData.email.trim() || !formData.phone.trim() || !formData.experienceLevel ||
!formData.emergencyContact.trim()) {
      setSubmitError('Please fill out all required fields.');
      setSubmitting(false);
      return;
    }

    try {
      await api.post('/registrations', {
        eventId,
        ...formData
      });
      setSuccess(true);
    } catch (err) {
      console.error('Registration failed:', err);
      setSubmitError(err.message || 'Registration failed. The course might be full or closed.');
    } finally {
      setSubmitting(false);
    }
  };

  if (loadingEvent) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[50vh] bg-mountain-50">
        <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
        <p className="mt-4 text-mountain-500 font-medium">Loading Course registration...</p>
      </div>
    );
  }
```

```
  if (errorEvent || !event) {
    return (
      <div className="max-w-md mx-auto my-16 p-8 bg-white rounded-2xl border border-mountain-100 shadow text-center space-y-4">
        <AlertTriangle className="w-12 h-12 text-earth-600 mx-auto" />
        <h2 className="text-xl font-display font-bold text-mountain-900">Registration Error</h2>
        <p className="text-mountain-600 text-sm leading-relaxed">{errorEvent || 'Event not found'}</p>
        <Link to="/events" className="inline-flex items-center text-earth-700 hover:text-earth-850 font-bold text-sm">
          <ArrowLeft className="w-4 h-4 mr-2" />
          <span>Back to Events list</span>
        </Link>
      </div>
    );
  }

  const isClosed = !event.registrationOpen || (event.deadline && new Date() > new Date(event.deadline));
  const isFull = event.capacity !== null && event.registrationsCount >= event.capacity;

  if (isClosed || isFull) {
    return (
      <div className="max-w-md mx-auto my-16 p-8 bg-white rounded-2xl border border-mountain-100 shadow text-center space-y-4">
        <AlertTriangle className="w-12 h-12 text-mountain-400 mx-auto" />
        <h2 className="text-xl font-display font-bold text-mountain-900">Registration Closed</h2>
        <p className="text-mountain-600 text-sm leading-relaxed">
          {isFull ? 'This course is already full.' : 'Registrations for this course are closed.'}
        </p>
        <Link to="/events" className="inline-flex items-center text-earth-700 hover:text-earth-850 font-bold text-sm">
          <ArrowLeft className="w-4 h-4 mr-2" />
          <span>Back to Events list</span>
        </Link>
      </div>
    );
  }

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-4xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Back Link */}
        <Link to="/events" className="inline-flex items-center text-mountain-500 hover:text-mountain-900 font-semibold text-sm mb-6 transition-
colors">
          <ArrowLeft className="w-4 h-4 mr-2" />
          <span>Back to Events</span>
        </Link>

        {success ? (
          /* Success Screen */
          <div className="bg-white rounded-2xl p-8 sm:p-12 border border-mountain-100 shadow text-center space-y-6">
            <CheckCircle2 className="w-16 h-16 text-forest-600 mx-auto" />
            <h2 className="text-3xl font-display font-extrabold text-mountain-900">Registration Successful!</h2>
            <p className="text-mountain-600 text-base max-w-xl mx-auto leading-relaxed">
              We have received your application for <strong>{event.title}</strong>. A confirmation email has been logged / sent. Please check
your inbox (or console) for details and payment instructions.
            </p>
            <div className="pt-6">
              <Link to="/events" className="px-6 py-3 bg-mountain-900 hover:bg-mountain-950 text-white font-bold rounded-lg transition-all
shadow-md text-sm">
                Return to Events
              </Link>
            </div>
          </div>
        ) : (
          /* Form Screen */
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-8 items-start">

            {/* Left: Course Card Info */}
            <div className="lg:col-span-5 bg-mountain-900 text-white p-6 sm:p-8 rounded-2xl shadow-xl space-y-6 border border-mountain-850">
              <span className="px-2 py-0.5 bg-mountain-800 text-mountain-200 text-[10px] font-bold rounded uppercase tracking-wider">
                {event.type}
              </span>

              <div className="space-y-2">
                <h2 className="text-2xl font-display font-bold leading-tight">
                  {event.title}
                </h2>
                <div className="w-12 h-1 bg-earth-500 rounded-full"></div>
              </div>

              <p className="text-mountain-300 text-sm leading-relaxed font-light">
                {event.description}
              </p>

              <div className="space-y-3 pt-4 border-t border-mountain-800 text-xs sm:text-sm text-mountain-200">
                <div className="flex items-center space-x-3">
                  <Calendar className="w-5 h-5 text-earth-400 shrink-0" />
                  <span>{new Date(event.date).toLocaleDateString(undefined, { year: 'numeric', month: 'long', day: 'numeric' })}</span>
                </div>
                <div className="flex items-center space-x-3">
                  <MapPin className="w-5 h-5 text-earth-400 shrink-0" />
                  <span>{event.location}</span>
```

```
              </div>
            </div>

            {event.capacity !== null && (
              <div className="pt-4 border-t border-mountain-800">
                <div className="flex justify-between text-xs text-mountain-300 font-semibold mb-1">
                  <span>Seats Remaining</span>
                  <span>{event.capacity - event.registrationsCount} left</span>
                </div>
                <div className="w-full bg-mountain-850 h-1.5 rounded-full overflow-hidden">
                  <div
                    className="bg-earth-550 h-full rounded-full"
                    style={{ width: `${(event.registrationsCount / event.capacity) * 100}%` }}
                  ></div>
                </div>
              </div>
            )}
          </div>

          {/* Right: Registration Form */}
          <div className="lg:col-span-7 bg-white p-6 sm:p-8 rounded-2xl border border-mountain-100 shadow-sm space-y-6">
            <div>
              <h3 className="text-xl font-display font-bold text-mountain-900">Registration Details</h3>
              <p className="text-xs text-mountain-500">Please provide accurate contact details for high-altitude security.</p>
            </div>

            {submitError && (
              <div className="bg-earth-50 border border-earth-200 text-earth-700 px-4 py-3 rounded-lg flex items-center space-x-3 text-sm">
                <AlertTriangle className="w-5 h-5 shrink-0" />
                <span>{submitError}</span>
              </div>
            )}

            <form onSubmit={handleSubmit} className="space-y-4">
              {/* Name */}
              <div>
                <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                  Full Name <span className="text-earth-650">*</span>
                </label>
                <input
                  type="text"
                  name="name"
                  required
                  value={formData.name}
                  onChange={handleChange}
                  className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                  placeholder="e.g. Amit Sen"
                />
              </div>

              {/* Email & Phone grid */}
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    Email Address <span className="text-earth-650">*</span>
                  </label>
                  <input
                    type="email"
                    name="email"
                    required
                    value={formData.email}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                    placeholder="e.g. amit@gmail.com"
                  />
                </div>
                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    Phone Number <span className="text-earth-650">*</span>
                  </label>
                  <input
                    type="tel"
                    name="phone"
                    required
                    value={formData.phone}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                    placeholder="e.g. +91 9876543210"
                  />
                </div>
              </div>

              {/* Roll number & Experience Level */}
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
```

```
                    JU Roll No. / Institution (If applicable)
                  </label>
                  <input
                    type="text"
                    name="rollNo"
                    value={formData.rollNo}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                    placeholder="e.g. 002110201042"
                  />
                </div>

                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    Climbing Experience <span className="text-earth-650">*</span>
                  </label>
                  <select
                    name="experienceLevel"
                    required
                    value={formData.experienceLevel}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600"
                  >
                    <option value="">Select Experience Level</option>
                    <option value="Beginner (No climbing history)">Beginner (No climbing history)</option>
                    <option value="Basic (Trekked or indoor wall)">Basic (Trekked or indoor wall)</option>
                    <option value="Intermediate (Susunia course completed)">Intermediate (Susunia course completed)</option>
                    <option value="Advanced (Himalayan height scaled)">Advanced (Himalayan height scaled)</option>
                  </select>
                </div>
              </div>

              {/* Emergency Contact & T Shirt Size */}
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    Emergency Contact Name & No. <span className="text-earth-650">*</span>
                  </label>
                  <input
                    type="text"
                    name="emergencyContact"
                    required
                    value={formData.emergencyContact}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                    placeholder="e.g. Father, +91 9876543211"
                  />
                </div>

                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    T-Shirt Size
                  </label>
                  <select
                    name="tShirtSize"
                    value={formData.tShirtSize}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600"
                  >
                    <option value="">Select T-Shirt Size</option>
                    <option value="S">Small (S)</option>
                    <option value="M">Medium (M)</option>
                    <option value="L">Large (L)</option>
                    <option value="XL">Extra Large (XL)</option>
                    <option value="XXL">Double Extra Large (XXL)</option>
                  </select>
                </div>
              </div>

              <div className="pt-4 border-t border-mountain-100 flex items-center justify-end">
                <button
                  type="submit"
                  disabled={submitting}
                  className="inline-flex items-center justify-center px-6 py-3 bg-forest-900 hover:bg-forest-950 text-white font-bold rounded-
lg shadow hover:shadow-md transition-all duration-200 group text-sm disabled:opacity-50 disabled:cursor-not-allowed"
                >
                  {submitting ? (
                    <>
                      <div className="w-4 h-4 border-2 border-white border-t-transparent rounded-full animate-spin mr-2"></div>
                      <span>Submitting Application...</span>
                    </>
                  ) : (
                    <>
                      <Send className="w-4 h-4 mr-2 group-hover:translate-x-0.5 transition-transform" />
                      <span>Submit Registration</span>
```

```
                    </>
                  )}
                </button>
              </div>
            </form>
          </div>

        </div>
      )}

    </div>
  </div>
);
};

export default CourseRegistration;
```

- `const { eventId } = useParams();` reads the `:eventId` URL parameter from the route defined in `App.jsx`, then fetches that specific event's details.
- Before showing the form at all, it checks `isClosed` and `isFull` and shows a dedicated "Registration Closed" screen instead if either is true — this is a *frontend convenience* only; the backend's atomic capacity check from Section 8 is what actually enforces this for real, since a determined user could bypass this frontend check entirely.
- The form itself is a **controlled form**: every input's `value` is driven by the `formData` state object, and every keystroke runs `handleChange`, which updates just that one field with `{ ...prev, [name]: value }` — the spread keeps every other field untouched.
- On submit, it does a quick client-side check that required fields aren't blank, then `api.post('/registrations', …)`; on success it swaps to a celebratory confirmation screen instead of clearing back to the empty form.

WHY RE-CHECK "IS THIS FULL/CLOSED" ON THE FRONTEND IF THE BACKEND ALREADY CHECKS IT?

Purely for a better user experience: it's much friendlier to show "this course is full" the instant the page loads than to let someone fill out an entire form and only find out it failed after clicking Submit. But it's never *trusted* as the real security/business-rule boundary — that's always the backend's job, since frontend JavaScript can always be inspected or bypassed by a determined visitor.

frontend/src/pages/Contact.jsx

```
import React, { useState } from 'react';
import { MapPin, Phone, Mail, Compass, Send, CheckCircle2 } from 'lucide-react';

const Contact = () => {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    subject: '',
    message: ''
  });
  const [submitting, setSubmitting] = useState(false);
  const [success, setSuccess] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitting(true);
    // Simulate sending message
    setTimeout(() => {
      setSubmitting(false);
      setSuccess(true);
      setFormData({ name: '', email: '', subject: '', message: '' });
    }, 1200);
  };

  return (
    <div className="bg-mountain-50 min-h-screen py-16">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">

        {/* Page Header */}
        <div className="text-center max-w-3xl mx-auto mb-16 space-y-4">
          <span className="text-xs font-bold uppercase tracking-widest text-earth-700 bg-earth-50 px-3 py-1 rounded-full border border-earth-
100">
            Get in Touch
          </span>
          <h1 className="text-4xl sm:text-5xl font-display font-extrabold text-mountain-900 tracking-tight">
            Contact JUMHC Committee
          </h1>
          <p className="text-base sm:text-lg text-mountain-600 leading-relaxed font-light">
            Have questions about registrations, joining the club, or sponsorship? Reach out to us or drop by our campus club room.
          </p>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-12 gap-8 items-start mb-16">
```

```
          {/* Left Column: Details */}
          <div className="lg:col-span-5 space-y-6">

            {/* Club room Address card */}
            <div className="bg-white p-6 sm:p-8 rounded-2xl border border-mountain-100 shadow-sm space-y-4">
              <h3 className="font-display font-bold text-lg text-mountain-900 flex items-center space-x-2">
                <Compass className="w-5 h-5 text-earth-650" />
                <span>Headquarters</span>
              </h3>

              <div className="space-y-4 text-sm text-mountain-600">
                <div className="flex items-start space-x-3">
                  <MapPin className="w-5 h-5 text-mountain-400 shrink-0 mt-0.5" />
                  <div>
                    <p className="font-semibold text-mountain-900">Club Room</p>
                    <p>O.A.T. Building, Jadavpur University Main Campus,</p>
                    <p>188, Raja S.C. Mallick Road, Kolkata - 700032</p>
                  </div>
                </div>

                <div className="flex items-center space-x-3">
                  <Phone className="w-5 h-5 text-mountain-400 shrink-0" />
                  <div>
                    <p className="font-semibold text-mountain-900">Phone</p>
                    <p>+91 98765 43210 (Secretariat)</p>
                  </div>
                </div>

                <div className="flex items-center space-x-3">
                  <Mail className="w-5 h-5 text-mountain-400 shrink-0" />
                  <div>
                    <p className="font-semibold text-mountain-900">Email</p>
                    <p>contact@jumhc.org</p>
                  </div>
                </div>
              </div>
            </div>

            {/* Meeting Timings Card */}
            <div className="bg-mountain-900 text-white p-6 sm:p-8 rounded-2xl shadow-xl space-y-3 border border-mountain-850">
              <h3 className="font-display font-bold text-lg text-earth-450">Meeting Hours</h3>
              <p className="text-sm text-mountain-300 leading-relaxed font-light">
                We meet weekly for physical training at the artificial bouldering wall and run planning meetings at the club room.
              </p>
              <ul className="text-xs sm:text-sm text-mountain-200 space-y-1.5 pt-2">
                <li><strong>Wall Training:</strong> Tuesday & Thursday (5:00 PM – 7:00 PM)</li>
                <li><strong>General Meetup:</strong> Friday (4:30 PM – 6:30 PM)</li>
                <li><strong>Location:</strong> Sports Complex Wall / Club Room</li>
              </ul>
            </div>
          </div>

          {/* Right Column: Contact Form */}
          <div className="lg:col-span-7 bg-white p-6 sm:p-8 rounded-2xl border border-mountain-100 shadow-sm space-y-6">
            <div>
              <h3 className="text-xl font-display font-bold text-mountain-900">Send a Message</h3>
              <p className="text-xs text-mountain-500">Fill out this form and our committee members will get back to you within 24 hours.</p>
            </div>

            {success ? (
              <div className="bg-forest-50 border border-forest-200 text-forest-800 p-6 rounded-xl text-center space-y-3">
                <CheckCircle2 className="w-10 h-10 text-forest-600 mx-auto" />
                <h4 className="font-bold text-lg">Message Sent!</h4>
                <p className="text-sm">Thank you for writing to us. We will get back to you shortly.</p>
                <button
                  onClick={() => setSuccess(false)}
                  className="mt-2 text-xs font-bold text-forest-750 hover:underline"
                >
                  Send another message
                </button>
              </div>
            ) : (
              <form onSubmit={handleSubmit} className="space-y-4">
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                      Your Name
                    </label>
                    <input
                      type="text"
                      name="name"
                      required
                      value={formData.name}
                      onChange={handleChange}
                      className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                      placeholder="e.g. Rahul Das"
                    />
                  </div>
```

```
                <div>
                  <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                    Your Email
                  </label>
                  <input
                    type="email"
                    name="email"
                    required
                    value={formData.email}
                    onChange={handleChange}
                    className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                    placeholder="e.g. rahul@example.com"
                  />
                </div>
              </div>

              <div>
                <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                  Subject
                </label>
                <input
                  type="text"
                  name="subject"
                  required
                  value={formData.subject}
                  onChange={handleChange}
                  className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent"
                  placeholder="e.g. Query regarding Susunia Course fees"
                />
              </div>

              <div>
                <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
                  Message
                </label>
                <textarea
                  name="message"
                  required
                  rows="5"
                  value={formData.message}
                  onChange={handleChange}
                  className="w-full border border-mountain-200 rounded-lg p-2.5 text-sm bg-mountain-50/50 focus:outline-none focus:ring-2
focus:ring-mountain-600 focus:border-transparent resize-none"
                  placeholder="Write your message here..."
                ></textarea>
              </div>

              <div className="flex justify-end">
                <button
                  type="submit"
                  disabled={submitting}
                  className="inline-flex items-center justify-center px-6 py-3 bg-mountain-900 hover:bg-mountain-950 text-white font-bold
rounded-lg shadow hover:shadow-md transition-all duration-200 group text-sm disabled:opacity-50"
                >
                  {submitting ? (
                    <>
                      <div className="w-4 h-4 border-2 border-white border-t-transparent rounded-full animate-spin mr-2"></div>
                      <span>Sending message...</span>
                    </>
                  ) : (
                    <>
                      <Send className="w-4 h-4 mr-2" />
                      <span>Send Message</span>
                    </>
                  )}
                </button>
              </div>
            </form>
          )}
        </div>
      </div>

      {/* Embedded Map Section */}
      <div className="bg-white p-4 rounded-2xl border border-mountain-100 shadow-sm overflow-hidden h-[400px]">
        <iframe
          title="JU Campus Map"
          src="https://www.google.com/maps/embed?
pb=!1m18!1m12!1m3!1d3686.068832961817!2d88.37039707530467!3d22.499732979543167!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x3a0271237f30d
071%3A0xcf2f42a9df273a5a!2sJadavpur%20University!5e0!3m2!1sen!2sin!4v1716300000000!5m2!1sen!2sin"
          width="100%"
          height="100%"
          style={{ border: 0 }}
          allowFullScreen=""
          loading="lazy"
          referrerPolicy="no-referrer-when-downgrade"
          className="rounded-xl"
        ></iframe>
```

```
        </div>

      </div>
    </div>
  );
};

export default Contact;
```

THIS FORM DOESN'T ACTUALLY SEND ANYTHING ANYWHERE

Notice `handleSubmit` here doesn't call `api.post(…)` at all — it just runs a `setTimeout` to simulate a short delay, then shows the success message. There's no corresponding `/api/contact` route on the backend. This page is otherwise fully built (validation, loading state, success state), so wiring it up for real would only need one backend route and swapping this one line for a real `api.post` call — worth knowing before you assume messages sent through it are actually reaching anyone.

frontend/src/pages/AdminLogin.jsx

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { ShieldAlert, Mail, Lock, Eye, EyeOff, AlertTriangle } from 'lucide-react';

const AdminLogin = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  const { admin, login } = useAuth();
  const navigate = useNavigate();

  // If already authenticated, redirect to admin dashboard
  useEffect(() => {
    if (admin) {
      navigate('/admin', { replace: true });
    }
  }, [admin, navigate]);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);
    setLoading(true);

    if (!email.trim() || !password.trim()) {
      setError('Please fill in both fields.');
      setLoading(false);
      return;
    }

    try {
      await login(email, password);
      navigate('/admin');
    } catch (err) {
      console.error('Login error:', err);
      setError(err.message || 'Invalid email or password.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="bg-mountain-50 min-h-[80vh] flex items-center justify-center py-12 px-4 sm:px-6 lg:px-8">
      <div className="max-w-md w-full bg-white p-8 rounded-2xl border border-mountain-100 shadow-xl space-y-8">

        {/* Header Branding */}
        <div className="flex flex-col items-center text-center space-y-3">
          <div className="w-14 h-14 rounded-full bg-mountain-900 text-white flex items-center justify-center">
            <ShieldAlert className="w-8 h-8" />
          </div>
          <div className="space-y-1">
            <h2 className="text-2xl font-display font-bold text-mountain-900">JU Mountaineering</h2>
            <p className="text-xs text-mountain-500 uppercase tracking-widest font-semibold">Committee Admin Login</p>
          </div>
        </div>

        {error && (
          <div className="bg-earth-50 border border-earth-200 text-earth-700 px-4 py-3 rounded-lg flex items-center space-x-3 text-xs sm:text-
sm">
            <AlertTriangle className="w-5 h-5 shrink-0" />
            <span>{error}</span>
          </div>
        )}
```

```
        <form onSubmit={handleSubmit} className="space-y-6">
          {/* Email input */}
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
              Admin Email
            </label>
            <div className="relative">
              <span className="absolute inset-y-0 left-0 pl-3 flex items-center text-mountain-400">
                <Mail className="w-4 h-4" />
              </span>
              <input
                type="email"
                required
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                className="w-full border border-mountain-200 rounded-lg py-2.5 pl-10 pr-4 text-sm bg-mountain-50/50 focus:outline-none
focus:ring-2 focus:ring-mountain-600 focus:border-transparent"
                placeholder="admin@jumhc.org"
              />
            </div>
          </div>

          {/* Password input */}
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-mountain-600 mb-1">
              Secret Password
            </label>
            <div className="relative">
              <span className="absolute inset-y-0 left-0 pl-3 flex items-center text-mountain-400">
                <Lock className="w-4 h-4" />
              </span>
              <input
                type={showPassword ? 'text' : 'password'}
                required
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                className="w-full border border-mountain-200 rounded-lg py-2.5 pl-10 pr-10 text-sm bg-mountain-50/50 focus:outline-none
focus:ring-2 focus:ring-mountain-600 focus:border-transparent"
                placeholder="••••••••"
              />
              <button
                type="button"
                onClick={() => setShowPassword(!showPassword)}
                className="absolute inset-y-0 right-0 pr-3 flex items-center text-mountain-400 hover:text-mountain-650"
              >
                {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
              </button>
            </div>
          </div>

          <div>
            <button
              type="submit"
              disabled={loading}
              className="w-full inline-flex items-center justify-center py-3 px-4 bg-mountain-900 hover:bg-mountain-950 text-white font-bold
rounded-lg shadow-lg hover:shadow-xl transition-all duration-200 text-sm disabled:opacity-50"
            >
              {loading ? (
                <>
                  <div className="w-4 h-4 border-2 border-white border-t-transparent rounded-full animate-spin mr-2"></div>
                  <span>Verifying Credentials...</span>
                </>
              ) : (
                <span>Access Dashboard</span>
              )}
            </button>
          </div>
        </form>

        <div className="border-t border-mountain-100 pt-6 text-center">
          <p className="text-xs text-mountain-400">
            For security reasons, public registrations are disabled. Pre-seeded admin keys can only be created by system script database roots.
          </p>
        </div>
      </div>
    </div>
  );
};

export default AdminLogin;
```

- A `useEffect` watches the `admin` value from `useAuth()` and redirects straight to `/admin` if it's already truthy — so a logged-in admin who navigates back to the login URL by mistake is bounced straight to the dashboard instead of seeing a login form again.
- `handleSubmit` calls the shared `login()` function from AuthContext (Section 13) rather than calling `api.post` directly — that's what keeps the token storage and global `admin` state update happening in one single place no matter which page triggers a login.
- `showPassword` is a small, self-contained bit of UI state that toggles the password input between `type="password"` and `type="text"`,

paired with an eye/eye-off icon button.

17. Frontend, Step 8: The Admin Dashboard (the big one)

`frontend/src/pages/AdminDashboard.jsx` *801 lines — the largest file in the project*

This is the single biggest file in the whole codebase, but it's not actually more *complicated* than the other pages — it's five very similar admin screens (Events, Registrations, Secretaries, Alumni, Gallery) living inside one component, switched between with a simple tab system, so it's really the same CRUD pattern from the backend routes repeated five times with slightly different fields. We'll go through it in chunks, in the order the code actually runs.

Chunk 1 17.1 — State: what this component needs to remember

```
import React, { useState, useEffect } from 'react';
import { api } from '../utils/api';
import { useAuth } from '../context/AuthContext';
import {
  Compass, Calendar, Users, Image, ShieldAlert,
  Trash2, Edit, Plus, FileSpreadsheet, Check, X, Star, Upload
} from 'lucide-react';

const AdminDashboard = () => {
  const { admin, logout } = useAuth();
  const [activeTab, setActiveTab] = useState('events');

  // Dynamic Data Lists
  const [events, setEvents] = useState([]);
  const [secretaries, setSecretaries] = useState([]);
  const [alumni, setAlumni] = useState([]);
  const [gallery, setGallery] = useState([]);
  const [registrations, setRegistrations] = useState([]);

  // Selection/filter state
  const [selectedEventId, setSelectedEventId] = useState('');

  // Loading States
  const [loading, setLoading] = useState(false);
  const [actionLoading, setActionLoading] = useState(false);

  // Form Editor State
  const [editItem, setEditItem] = useState(null); // general object for items being edited
  const [showAddForm, setShowAddForm] = useState(false);
```

- Five separate list states (`events, secretaries, alumni, gallery, registrations`) — one per tab, all starting empty.
- `activeTab` is the single source of truth for which of the five admin screens is currently showing.
- `editItem` and `showAddForm` are the clever bit: rather than having separate "add form" and "edit form" components/state for every single tab (which would be ten near-identical forms), this dashboard uses *one* generic idea — if `editItem` holds an existing object, the shared form pre-fills itself from it and submits as an update; if it's `null` but `showAddForm` is true, the same form starts blank and submits as a create.

Chunk 2 17.2 — Fetching data for whichever tab is active

```
  // 1. Fetching logic
  const fetchData = async () => {
    setLoading(true);
    try {
      if (activeTab === 'events') {
        const data = await api.get('/events');
        setEvents(data);
      } else if (activeTab === 'secretaries') {
        const data = await api.get('/secretaries');
        setSecretaries(data);
      } else if (activeTab === 'alumni') {
        const data = await api.get('/alumni');
        setAlumni(data);
      } else if (activeTab === 'gallery') {
        const data = await api.get('/gallery');
        setGallery(data);
      } else if (activeTab === 'registrations') {
        const evts = await api.get('/events');
        setEvents(evts);
        if (evts.length > 0) {
          const defaultEventId = selectedEventId || evts[0]._id;
          setSelectedEventId(defaultEventId);
          const regs = await api.get(`/registrations/event/${defaultEventId}`);
          setRegistrations(regs);
        }
      }
    } catch (err) {
      console.error('Error loading dashboard data:', err);
```

```
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchData();
}, [activeTab]);

// Handle registration event change
useEffect(() => {
  if (activeTab === 'registrations' && selectedEventId) {
    const fetchRegs = async () => {
      try {
        const regs = await api.get(`/registrations/event/${selectedEventId}`);
        setRegistrations(regs);
      } catch (err) {
        console.error(err);
      }
    };
    fetchRegs();
  }
}, [selectedEventId]);
```

- `fetchData()` is one function that branches on `activeTab` and calls the matching API endpoint — it's re-run automatically by the `useEffect(…, [activeTab])` right below it every time an admin clicks a different tab.
- The `registrations` branch is the one exception with extra logic: it also fetches the list of events (to populate the "choose an event" dropdown), and if no event is selected yet, it defaults to the first one in the list.
- A second, separate `useEffect` watches `selectedEventId` specifically, so switching which event's registrations you're viewing re-fetches just that data without needing to re-run the whole `fetchData` function or re-fetch the events list itself.

Chunk 3 17.3 — Delete, payment toggle, and CSV export actions

```
// 2. Delete actions
const handleDelete = async (endpoint, id) => {
  if (!window.confirm('Are you sure you want to delete this item?')) return;
  setActionLoading(true);
  try {
    await api.delete(`${endpoint}/${id}`);
    fetchData();
  } catch (err) {
    alert('Delete failed: ' + err.message);
  } finally {
    setActionLoading(false);
  }
};

// 3. Status changes (Payment)
const togglePayment = async (regId, currentStatus) => {
  const nextStatus = currentStatus === 'Pending' ? 'Received' : 'Pending';
  try {
    await api.put(`/registrations/${regId}/payment`, { paymentStatus: nextStatus });
    fetchData();
  } catch (err) {
    alert('Update failed: ' + err.message);
  }
};

// CSV download
const handleExportCSV = async () => {
  if (!selectedEventId) return;
  const selectedEventObj = events.find(e => e._id === selectedEventId);
  const title = selectedEventObj ? selectedEventObj.title : 'registrations';
  try {
    await api.download(
      `/registrations/event/${selectedEventId}/export`,
      `registrations-${title.toLowerCase().replace(/[^a-z0-9]+/g, '-')}.csv`
    );
  } catch (err) {
    alert('CSV Export failed: ' + err.message);
  }
};
```

- `handleDelete` is generic across every tab — it takes the API endpoint prefix and an id as arguments, so the exact same function deletes an event, a secretary, an alumni profile, or a gallery item, just called with a different `endpoint` string each time. It confirms with the browser's native `window.confirm` first, since deletes here are permanent.
- `togglePayment` flips a registration between "Pending" and "Received" with one PUT request — this is what powers the clickable payment-status button you'll see in the registrations table.
- `handleExportCSV` calls the specialised `api.download` helper from Section 12 to trigger a real file download of the currently-selected event's registrations.

Chunk 4 17.4 — One submit handler for every create/edit form

```
// 4. Form Submit logic
const handleFormSubmit = async (e, type, method, id = null) => {
  e.preventDefault();
  setActionLoading(true);
  const formData = new FormData(e.target);

  try {
    if (type === 'secretary') {
      const payload = Object.fromEntries(formData.entries());
      if (method === 'POST') await api.post('/secretaries', payload);
      else await api.put(`/secretaries/${id}`, payload);
    }
    else if (type === 'event') {
      const entries = Object.fromEntries(formData.entries());
      // Handle checkbox
      entries.registrationOpen = formData.get('registrationOpen') === 'true';
      if (method === 'POST') await api.post('/events', entries);
      else await api.put(`/events/${id}`, entries);
    }
    else if (type === 'alumni') {
      // Needs multipart because of file upload
      const body = new FormData();
      body.append('name', formData.get('name'));
      body.append('batchYear', formData.get('batchYear'));
      body.append('roleHistory', formData.get('roleHistory'));
      body.append('currentContribution', formData.get('currentContribution'));
      body.append('bio', formData.get('bio'));
      body.append('isActiveInstructor', formData.get('isActiveInstructor') === 'true');

      const photoFile = formData.get('photo');
      if (photoFile && photoFile.size > 0) {
        body.append('photo', photoFile);
      } else if (editItem && editItem.photoUrl) {
        body.append('photoUrl', editItem.photoUrl);
      }

      if (method === 'POST') await api.post('/alumni', body, true);
      else await api.put(`/alumni/${id}`, body, true);
    }
    else if (type === 'gallery') {
      const body = new FormData();
      body.append('year', formData.get('year'));
      body.append('eventTag', formData.get('eventTag'));
      body.append('caption', formData.get('caption'));

      const imgFile = formData.get('image');
      if (imgFile && imgFile.size > 0) {
        body.append('image', imgFile);
      }

      if (method === 'POST') await api.post('/gallery', body, true);
    }

    setShowAddForm(false);
    setEditItem(null);
    fetchData();
  } catch (err) {
    alert('Action failed: ' + err.message);
  } finally {
    setActionLoading(false);
  }
};
```

This is the most interesting piece of the whole dashboard. Rather than writing a separate submit function for each of the five tabs' forms, `handleFormSubmit(e, type, method, id)` is one function used by *all* of them, branching internally on the `type` string:

- For `'secretary'` and `'event'`, it reads the submitted `<form>` with `new FormData(e.target)` and converts it straight to a plain object with `Object.fromEntries(…)`, since neither of those forms uploads a file — a normal JSON body is enough.
- For `'alumni'` and `'gallery'`, it instead builds a fresh `FormData` object field-by-field and passes it to `api.post(…, body, true)` with the `isFormData` flag set — this is required because those two forms can include an uploaded photo/image file, which a plain JSON body can't carry.
- The `method` argument (`'POST'` vs `'PUT'`) is what decides whether this call creates a brand-new record or updates the one referenced by `id` — and that argument is set based on whether `editItem` was `null` at the moment the form was opened, tying back to the state design from Chunk 1.
- Whatever happens, the `finally` block always calls `fetchData()` again at the end, re-pulling the fresh list from the server — rather than trying to manually patch the local `events` / `alumni` /etc. array in place, which would be more code and an easy place to introduce a bug where the screen shows something slightly different from what's actually saved.

Chunk 5 17.5 — The dashboard shell: banner and tab bar

```
  return (
    <div className="bg-mountain-50 min-h-screen">

      {/* Top dashboard Banner */}
      <div className="bg-mountain-900 text-white py-6 border-b border-mountain-850 shadow-md">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 flex flex-col sm:flex-row items-center justify-between gap-4">
          <div className="flex items-center space-x-3">
            <ShieldAlert className="w-8 h-8 text-earth-400" />
            <div>
              <h1 className="text-xl sm:text-2xl font-display font-extrabold">JUMHC Management Panel</h1>
              <p className="text-xs text-mountain-300">Logged in as {admin?.email} ({admin?.role})</p>
            </div>
          </div>
          <button
            onClick={logout}
            className="px-4 py-2 bg-mountain-800 hover:bg-mountain-950 hover:text-earth-400 rounded-lg text-sm font-semibold border border-
mountain-700 transition-all"
          >
            Sign Out
          </button>
        </div>
      </div>

      {/* Main Core View */}
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-10">

        {/* Navigation Tabs */}
        <div className="flex flex-wrap gap-2 border-b border-mountain-200 pb-4 mb-8">
          {[
            { id: 'events', label: 'Events & Courses', icon: Calendar },
            { id: 'registrations', label: 'Registrations', icon: Users },
            { id: 'secretaries', label: 'Secretaries Timeline', icon: Compass },
            { id: 'alumni', label: 'Alumni Directory', icon: Users },
            { id: 'gallery', label: 'Gallery Archive', icon: Image }
          ].map(tab => {
            const Icon = tab.icon;
            const isSelected = activeTab === tab.id;
            return (
              <button
                key={tab.id}
                onClick={() => {
                  setActiveTab(tab.id);
                  setShowAddForm(false);
                  setEditItem(null);
                }}
                className={`flex items-center space-x-2 px-4 py-2.5 rounded-lg text-sm font-semibold border transition-all ${
                  isSelected
                    ? 'bg-mountain-900 text-white border-mountain-900 shadow-md'
                    : 'bg-white text-mountain-700 border-mountain-200 hover:bg-mountain-100/50'
                }`}
              >
                <Icon className="w-4 h-4" />
                <span>{tab.label}</span>
              </button>
            );
          })}
        </div>

        {/* Global CRUD Actions Loader */}
        {loading && (
          <div className="flex justify-center py-20">
            <div className="w-10 h-10 border-4 border-earth-600 border-t-transparent rounded-full animate-spin"></div>
          </div>
        )}

        {!loading && (
          <div className="space-y-6">
```

The visible part of the component starts here: a dark banner showing who's logged in with a Sign Out button, then a row of five tab buttons built by mapping over a small array of `{ id, label, icon }` objects — clicking one sets `activeTab` and also resets `showAddForm` / `editItem` back to closed, so switching tabs never leaves a stray edit form open from whatever you were doing on the previous tab.

Chunk 6 17.6 — The Events tab: form + table, in full

This is the one tab worth showing completely, since every other tab (Secretaries, Alumni, Gallery) is built from the exact same two-part shape: a conditional create/edit form followed by a table listing existing records with Edit/Delete buttons on each row.

```
            {/* 1. TABS CONTENT FOR EVENTS */}
            {activeTab === 'events' && (
              <div className="space-y-6">
                <div className="flex justify-between items-center">
                  <h2 className="text-xl font-display font-extrabold text-mountain-900">Manage Club Events</h2>
                  {!showAddForm && !editItem && (
                    <button
                      onClick={() => setShowAddForm(true)}
```

```
                    className="inline-flex items-center space-x-1.5 px-4 py-2.5 bg-forest-900 text-forest-300 font-bold rounded-lg hover:bg-
forest-950 text-sm shadow transition-all border border-forest-800"
                  >
                    <Plus className="w-4 h-4" />
                    <span>Create Event</span>
                  </button>
                )}
              </div>

              {/* Create/Edit Form */}
              {(showAddForm || editItem) && (
                <div className="bg-white p-6 rounded-2xl border border-mountain-100 shadow-sm space-y-4">
                  <h3 className="text-lg font-display font-bold text-mountain-900">
                    {editItem ? 'Edit Event Details' : 'Create New Event'}
                  </h3>
                  <form
                    onSubmit={(e) => handleFormSubmit(e, 'event', editItem ? 'PUT' : 'POST', editItem?._id)}
                    className="space-y-4"
                  >
                    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Title</label>
                        <input type="text" name="title" required defaultValue={editItem?.title || ''} className="w-full border border-
mountain-200 rounded-lg p-2 text-sm" />
                      </div>
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Type</label>
                        <select name="type" required defaultValue={editItem?.type || ''} className="w-full border border-mountain-200 rounded-
lg p-2 text-sm bg-white">
                          <option value="course">Course</option>
                          <option value="expedition">Expedition</option>
                          <option value="competition">Competition</option>
                          <option value="other">Other</option>
                        </select>
                      </div>
                    </div>

                    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Date</label>
                        <input type="date" name="date" required defaultValue={editItem?.date ? new
Date(editItem.date).toISOString().split('T')[0] : ''} className="w-full border border-mountain-200 rounded-lg p-2 text-sm" />
                      </div>
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Location</label>
                        <input type="text" name="location" required defaultValue={editItem?.location || ''} className="w-full border border-
mountain-200 rounded-lg p-2 text-sm" />
                      </div>
                    </div>

                    <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Capacity (Seats, leave blank if none)
</label>
                        <input type="number" name="capacity" defaultValue={editItem?.capacity || ''} className="w-full border border-mountain-
200 rounded-lg p-2 text-sm" />
                      </div>
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Deadline Date (optional)</label>
                        <input type="date" name="deadline" defaultValue={editItem?.deadline ? new
Date(editItem.deadline).toISOString().split('T')[0] : ''} className="w-full border border-mountain-200 rounded-lg p-2 text-sm" />
                      </div>
                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Registration Status</label>
                        <select name="registrationOpen" defaultValue={editItem?.registrationOpen !== undefined ?
editItem.registrationOpen.toString() : 'true'} className="w-full border border-mountain-200 rounded-lg p-2 text-sm bg-white">
                          <option value="true">Open</option>
                          <option value="false">Closed</option>
                        </select>
                      </div>
                    </div>

                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Description</label>
                      <textarea name="description" required rows="3" defaultValue={editItem?.description || ''} className="w-full border
border-mountain-200 rounded-lg p-2 text-sm resize-none"></textarea>
                    </div>

                    <div className="flex justify-end space-x-2 pt-2">
                      <button type="button" onClick={() => { setShowAddForm(false); setEditItem(null); }} className="px-4 py-2 border border-
mountain-200 rounded-lg text-sm font-semibold">Cancel</button>
                      <button type="submit" disabled={actionLoading} className="px-5 py-2 bg-mountain-900 text-white rounded-lg text-sm font-
bold shadow hover:bg-mountain-950">
                        {actionLoading ? 'Saving...' : 'Save Event'}
                      </button>
                    </div>
                  </form>
                </div>
              )}
```

```
              {/* Events list */}
              <div className="bg-white rounded-2xl border border-mountain-100 shadow-sm overflow-hidden">
                <div className="overflow-x-auto">
                  <table className="w-full text-left text-sm">
                    <thead className="bg-mountain-50 text-mountain-500 font-semibold uppercase text-xs border-b border-mountain-100">
                      <tr>
                        <th className="p-4">Title</th>
                        <th className="p-4">Type</th>
                        <th className="p-4">Date</th>
                        <th className="p-4">Location</th>
                        <th className="p-4 text-center">Registrations</th>
                        <th className="p-4">Reg Status</th>
                        <th className="p-4 text-right">Actions</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-mountain-100">
                      {events.map(event => (
                        <tr key={event._id} className="hover:bg-mountain-50/40">
                          <td className="p-4 font-bold text-mountain-900">{event.title}</td>
                          <td className="p-4 capitalize"><span className="px-2 py-0.5 bg-mountain-100 text-mountain-600 rounded text-xs">
{event.type}</span></td>
                          <td className="p-4">{new Date(event.date).toLocaleDateString()}</td>
                          <td className="p-4 truncate max-w-[150px]">{event.location}</td>
                          <td className="p-4 text-center font-semibold">
                            {event.registrationsCount} {event.capacity ? `/ ${event.capacity}` : ''}
                          </td>
                          <td className="p-4">
                            <span className={`px-2 py-0.5 rounded text-xs font-bold ${event.registrationOpen ? 'bg-forest-50 text-forest-700'
: 'bg-earth-50 text-earth-700'}`}>
                              {event.registrationOpen ? 'Active' : 'Closed'}
                            </span>
                          </td>
                          <td className="p-4 text-right">
                            <div className="flex justify-end space-x-2">
                              <button onClick={() => { setEditItem(event); setShowAddForm(false); }} className="p-1 text-mountain-500
hover:text-mountain-900" title="Edit"><Edit className="w-4 h-4" /></button>
                              <button onClick={() => handleDelete('/events', event._id)} className="p-1 text-mountain-500 hover:text-earth-
650" title="Delete"><Trash2 className="w-4 h-4" /></button>
                            </div>
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              </div>
            </div>
          )}
```

- `{(showAddForm || editItem) && (…)}` is the single condition deciding whether the form appears at all — it's shown for *either* "creating something new" or "editing something existing", which is why the heading right above it reads `{editItem ? 'Edit Event Details' : 'Create New Event'}` instead of needing two separate form blocks.
- Every input uses `defaultValue={editItem?.title || ''}` rather than a controlled `value` — these are **uncontrolled** inputs. The form doesn't track keystrokes in React state at all; it just pre-fills from `editItem` when opened, and reads the final values back out with `new FormData(e.target)` only at submit time (back in Chunk 4). This is a deliberate simplification — for a form this size, wiring up individual `onChange` handlers for every field (like `CourseRegistration.jsx` does) would be a lot more code for no real benefit here.
- The table at the bottom maps over the `events` array; each row's Edit button just calls `setEditItem(event)` — which is all it takes to re-open the very same form above, now pre-filled, since the form reads from whatever `editItem` currently holds.

WHAT TO EXPECT FROM THE REMAINING TABS

The Registrations, Secretaries, Alumni, and Gallery tabs that follow (shown in full in the next few chunks) repeat this identical form-then-table shape against different fields and models — Secretaries adds/edits `yearStart/yearEnd/name/note`, Alumni adds a file-upload input and an "Active Instructor" dropdown, Gallery is upload-only with no edit form, and Registrations swaps the create-form for a read-only table with a payment-toggle button and a CSV export instead. Once the Events tab's pattern above makes sense, reading the rest should feel like recognising the same shape with different labels, not new concepts — so the remaining notes will be shorter.

Chunk 7 17.7 — The Registrations tab: read-only table + actions

```
          {/* 2. TABS CONTENT FOR REGISTRATIONS */}
          {activeTab === 'registrations' && (
            <div className="space-y-6">
              <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4">
                <div className="space-y-1">
                  <h2 className="text-xl font-display font-extrabold text-mountain-900">Course Applications</h2>
                  <p className="text-xs text-mountain-500">Filter registrations by event / course and manage their payment status.</p>
                </div>

                {/* Select event dropdown */}
                <div className="flex flex-wrap items-center gap-3 w-full sm:w-auto">
```

```
                    <select
                      value={selectedEventId}
                      onChange={(e) => setSelectedEventId(e.target.value)}
                      className="p-2 border border-mountain-200 rounded-lg text-sm bg-white flex-grow sm:flex-grow-0"
                    >
                      {events.map(e => (
                        <option key={e._id} value={e._id}>{e.title}</option>
                      ))}
                    </select>

                    {registrations.length > 0 && (
                      <button
                        onClick={handleExportCSV}
                        className="inline-flex items-center space-x-1 px-4 py-2 bg-mountain-900 text-white rounded-lg text-sm font-bold shadow
hover:bg-mountain-950 w-full sm:w-auto justify-center"
                      >
                        <FileSpreadsheet className="w-4 h-4" />
                        <span>Export CSV</span>
                      </button>
                    )}
                  </div>
                </div>

                {/* Registrations Table */}
                <div className="bg-white rounded-2xl border border-mountain-100 shadow-sm overflow-hidden">
                  <div className="overflow-x-auto">
                    {registrations.length === 0 ? (
                      <div className="text-center py-16 text-mountain-500">No registrants found for this event.</div>
                    ) : (
                      <table className="w-full text-left text-sm">
                        <thead className="bg-mountain-50 text-mountain-500 font-semibold uppercase text-xs border-b border-mountain-100">
                          <tr>
                            <th className="p-4">Name</th>
                            <th className="p-4">Contact</th>
                            <th className="p-4">Roll No</th>
                            <th className="p-4">Experience</th>
                            <th className="p-4 text-center">T-Shirt</th>
                            <th className="p-4">Emergency</th>
                            <th className="p-4 text-center">Payment</th>
                            <th className="p-4 text-right">Delete</th>
                          </tr>
                        </thead>
                        <tbody className="divide-y divide-mountain-100">
                          {registrations.map(reg => (
                            <tr key={reg._id} className="hover:bg-mountain-50/40">
                              <td className="p-4 font-bold text-mountain-900">{reg.name}</td>
                              <td className="p-4 text-xs">
                                <p>{reg.email}</p>
                                <p className="text-mountain-400 mt-0.5">{reg.phone}</p>
                              </td>
                              <td className="p-4">{reg.rollNo || <span className="text-mountain-400">N/A</span>}</td>
                              <td className="p-4 text-xs font-medium text-mountain-600">{reg.experienceLevel}</td>
                              <td className="p-4 text-center font-bold">{reg.tShirtSize || <span className="text-mountain-400">N/A</span>}</td>
                              <td className="p-4 text-xs truncate max-w-[120px]">{reg.emergencyContact}</td>
                              <td className="p-4 text-center">
                                <button
                                  onClick={() => togglePayment(reg._id, reg.paymentStatus)}
                                  className={`px-3 py-1 rounded text-xs font-bold transition-all border ${
                                    reg.paymentStatus === 'Received'
                                      ? 'bg-forest-50 text-forest-700 border-forest-200'
                                      : 'bg-earth-50 text-earth-700 border-earth-200'
                                  }`}
                                >
                                  {reg.paymentStatus}
                                </button>
                              </td>
                              <td className="p-4 text-right">
                                <button
                                  onClick={() => handleDelete('/registrations', reg._id)}
                                  className="p-1 text-mountain-500 hover:text-earth-650"
                                >
                                  <Trash2 className="w-4 h-4" />
                                </button>
                              </td>
                            </tr>
                          ))}
                        </tbody>
                      </table>
                    )}
                  </div>
                </div>
              </div>
            )}
```

No create form here at all — registrations only come from the public registration form, never typed in by an admin. The event-picker `<select>` drives `selectedEventId`, which (from Chunk 2) triggers a re-fetch of just that event's registrants. Each row's payment badge is itself a button wired to `togglePayment` from Chunk 3, and the CSV button only appears once there's at least one registration to export.

Chunk 8 17.8 — The Secretaries tab

```
            {/* 3. TABS CONTENT FOR SECRETARIES */}
            {activeTab === 'secretaries' && (
              <div className="space-y-6">
                <div className="flex justify-between items-center">
                  <h2 className="text-xl font-display font-extrabold text-mountain-900">Manage Secretaries Timeline</h2>
                  {!showAddForm && !editItem && (
                    <button
                      onClick={() => setShowAddForm(true)}
                      className="inline-flex items-center space-x-1 px-4 py-2.5 bg-forest-900 text-forest-300 font-bold rounded-lg hover:bg-
forest-950 text-sm shadow transition-all border border-forest-800"
                    >
                      <Plus className="w-4 h-4" />
                      <span>Add Entry</span>
                    </button>
                  )}
                </div>

                {/* Form Add/Edit */}
                {(showAddForm || editItem) && (
                  <div className="bg-white p-6 rounded-2xl border border-mountain-100 shadow-sm space-y-4">
                    <h3 className="text-lg font-display font-bold text-mountain-900">
                      {editItem ? 'Edit Secretary Details' : 'Add New Term'}
                    </h3>
                    <form
                      onSubmit={(e) => handleFormSubmit(e, 'secretary', editItem ? 'PUT' : 'POST', editItem?._id)}
                      className="space-y-4"
                    >
                      <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Start Year</label>
                          <input type="number" name="yearStart" required defaultValue={editItem?.yearStart || ''} className="w-full border
border-mountain-200 rounded-lg p-2 text-sm" />
                        </div>
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">End Year</label>
                          <input type="number" name="yearEnd" required defaultValue={editItem?.yearEnd || ''} className="w-full border border-
mountain-200 rounded-lg p-2 text-sm" />
                        </div>
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Name</label>
                          <input type="text" name="name" required defaultValue={editItem?.name || ''} className="w-full border border-mountain-
200 rounded-lg p-2 text-sm" />
                        </div>
                      </div>

                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Timeline Achievement / Milestone
Note</label>
                        <input type="text" name="note" defaultValue={editItem?.note || ''} className="w-full border border-mountain-200 rounded-
lg p-2 text-sm" placeholder="e.g. Led successful Mt. Jogin-III expedition" />
                      </div>

                      <div className="flex justify-end space-x-2 pt-2">
                        <button type="button" onClick={() => { setShowAddForm(false); setEditItem(null); }} className="px-4 py-2 border border-
mountain-200 rounded-lg text-sm font-semibold">Cancel</button>
                        <button type="submit" disabled={actionLoading} className="px-5 py-2 bg-mountain-900 text-white rounded-lg text-sm font-
bold shadow hover:bg-mountain-950">
                          {actionLoading ? 'Saving...' : 'Save entry'}
                        </button>
                      </div>
                    </form>
                  </div>
                )}

                {/* Secretaries list */}
                <div className="bg-white rounded-2xl border border-mountain-100 shadow-sm overflow-hidden">
                  <table className="w-full text-left text-sm">
                    <thead className="bg-mountain-50 text-mountain-500 font-semibold uppercase text-xs border-b border-mountain-100">
                      <tr>
                        <th className="p-4">Term Year</th>
                        <th className="p-4">Secretary Name</th>
                        <th className="p-4">Milestone / Achievement note</th>
                        <th className="p-4 text-right">Actions</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-mountain-100">
                      {secretaries.map(sec => (
                        <tr key={sec._id} className="hover:bg-mountain-50/40">
                          <td className="p-4 font-bold text-mountain-900">{sec.yearStart} – {sec.yearEnd}</td>
                          <td className="p-4 font-medium">{sec.name}</td>
                          <td className="p-4 text-mountain-600 text-xs italic">{sec.note || '-'}</td>
                          <td className="p-4 text-right">
                            <div className="flex justify-end space-x-2">
                              <button onClick={() => { setEditItem(sec); setShowAddForm(false); }} className="p-1 text-mountain-500 hover:text-
mountain-900"><Edit className="w-4 h-4" /></button>
```

```
                          <button onClick={() => handleDelete('/secretaries', sec._id)} className="p-1 text-mountain-500 hover:text-earth-
650"><Trash2 className="w-4 h-4" /></button>
                        </div>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          </div>
        )}
```

The same create/edit-form-plus-table shape as Events (Chunk 6), just for three fields (`yearStart` , `yearEnd` , `name`) plus an optional milestone `note` . It submits through the same generic `handleFormSubmit` with `type='secretary'` .

Chunk 9 17.9 — The Alumni tab: adds a photo upload

```
        {/* 4. TABS CONTENT FOR ALUMNI */}
        {activeTab === 'alumni' && (
          <div className="space-y-6">
            <div className="flex justify-between items-center">
              <h2 className="text-xl font-display font-extrabold text-mountain-900">Manage Alumni Profiles</h2>
              {!showAddForm && !editItem && (
                <button
                  onClick={() => setShowAddForm(true)}
                  className="inline-flex items-center space-x-1 px-4 py-2.5 bg-forest-900 text-forest-300 font-bold rounded-lg hover:bg-
forest-950 text-sm shadow transition-all border border-forest-800"
                >
                  <Plus className="w-4 h-4" />
                  <span>Add Alumni</span>
                </button>
              )}
            </div>

            {/* Form Add/Edit */}
            {(showAddForm || editItem) && (
              <div className="bg-white p-6 rounded-2xl border border-mountain-100 shadow-sm space-y-4">
                <h3 className="text-lg font-display font-bold text-mountain-900">
                  {editItem ? 'Edit Alumni Profile' : 'Add Alumni Profile'}
                </h3>
                <form
                  onSubmit={(e) => handleFormSubmit(e, 'alumni', editItem ? 'PUT' : 'POST', editItem?._id)}
                  className="space-y-4"
                  encType="multipart/form-data"
                >
                  <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Name</label>
                      <input type="text" name="name" required defaultValue={editItem?.name || ''} className="w-full border border-mountain-
200 rounded-lg p-2 text-sm" />
                    </div>
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Batch Year</label>
                      <input type="number" name="batchYear" required defaultValue={editItem?.batchYear || ''} className="w-full border
border-mountain-200 rounded-lg p-2 text-sm" />
                    </div>
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Active Climbing Instructor?</label>
                      <select name="isActiveInstructor" defaultValue={editItem?.isActiveInstructor?.toString() || 'false'} className="w-full
border border-mountain-200 rounded-lg p-2 text-sm bg-white">
                        <option value="false">No</option>
                        <option value="true">Yes (Add Star Badge)</option>
                      </select>
                    </div>
                  </div>

                  <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Role History (e.g. Secretary 1998 to
Instructor)</label>
                      <input type="text" name="roleHistory" required defaultValue={editItem?.roleHistory || ''} className="w-full border
border-mountain-200 rounded-lg p-2 text-sm" />
                    </div>
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Current Contribution / Involvement</label>
                      <input type="text" name="currentContribution" defaultValue={editItem?.currentContribution || ''} className="w-full
border border-mountain-200 rounded-lg p-2 text-sm" placeholder="e.g. Guest Instructor for Susunia course" />
                    </div>
                  </div>

                  <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Upload Photo (optional)</label>
                      <div className="flex items-center space-x-3">
                        <input type="file" name="photo" accept="image/*" className="w-full text-xs text-mountain-500 file:mr-4 file:py-2
file:px-4 file:rounded-lg file:border-0 file:text-xs file:font-semibold file:bg-mountain-100 file:text-mountain-700 hover:file:bg-mountain-200"
```

```
/>
                        </div>
                      </div>
                      {editItem?.photoUrl && (
                        <div>
                          <span className="block text-xs text-mountain-400">Current Photo:</span>
                          <img src={editItem.photoUrl.startsWith('/uploads/') ? `http://localhost:5000${editItem.photoUrl}` :
editItem.photoUrl} className="h-10 w-10 object-cover rounded mt-1 border" />
                        </div>
                      )}
                    </div>

                    <div>
                      <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Brief Bio</label>
                      <textarea name="bio" rows="2" defaultValue={editItem?.bio || ''} className="w-full border border-mountain-200 rounded-lg
p-2 text-sm resize-none"></textarea>
                    </div>

                    <div className="flex justify-end space-x-2 pt-2">
                      <button type="button" onClick={() => { setShowAddForm(false); setEditItem(null); }} className="px-4 py-2 border border-
mountain-200 rounded-lg text-sm font-semibold">Cancel</button>
                      <button type="submit" disabled={actionLoading} className="px-5 py-2 bg-mountain-900 text-white rounded-lg text-sm font-
bold shadow hover:bg-mountain-950">
                        {actionLoading ? 'Saving...' : 'Save Profile'}
                      </button>
                    </div>
                  </form>
                </div>
              )}

              {/* Alumni list */}
              <div className="bg-white rounded-2xl border border-mountain-100 shadow-sm overflow-hidden">
                <table className="w-full text-left text-sm">
                  <thead className="bg-mountain-50 text-mountain-500 font-semibold uppercase text-xs border-b border-mountain-100">
                    <tr>
                      <th className="p-4">Photo</th>
                      <th className="p-4">Name</th>
                      <th className="p-4">Batch</th>
                      <th className="p-4">Role History</th>
                      <th className="p-4">Instructor?</th>
                      <th className="p-4 text-right">Actions</th>
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-mountain-100">
                    {alumni.map(member => (
                      <tr key={member._id} className="hover:bg-mountain-50/40">
                        <td className="p-4">
                          <img
                            src={member.photoUrl ? (member.photoUrl.startsWith('/uploads/') ? `http://localhost:5000${member.photoUrl}` :
member.photoUrl) : 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?auto=format&fit=crop&q=80&w=200'}
                            className="w-8 h-8 rounded-full object-cover border"
                            onError={(e) => { e.target.src = 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?
auto=format&fit=crop&q=80&w=200'; }}
                          />
                        </td>
                        <td className="p-4 font-bold text-mountain-900">{member.name}</td>
                        <td className="p-4 font-semibold text-earth-700">{member.batchYear}</td>
                        <td className="p-4 text-xs font-medium max-w-[200px] truncate">{member.roleHistory}</td>
                        <td className="p-4">
                          {member.isActiveInstructor ? (
                            <span className="inline-flex items-center text-forest-700 bg-forest-50 border border-forest-100 text-[10px] font-
bold px-2 py-0.5 rounded-full">
                              <Star className="w-2.5 h-2.5 fill-current mr-1" />
                              Yes
                            </span>
                          ) : 'No'}
                        </td>
                        <td className="p-4 text-right">
                          <div className="flex justify-end space-x-2">
                            <button onClick={() => { setEditItem(member); setShowAddForm(false); }} className="p-1 text-mountain-500
hover:text-mountain-900"><Edit className="w-4 h-4" /></button>
                            <button onClick={() => handleDelete('/alumni', member._id)} className="p-1 text-mountain-500 hover:text-earth-
650"><Trash2 className="w-4 h-4" /></button>
                          </div>
                        </td>
                      </tr>
                    ))}
                  </tbody>
                </table>
              </div>
            </div>
          )}
```

Same shape again, with two additions: `encType="multipart/form-data"` on the `<form>` and a `<input type="file" name="photo">`, which is why this tab's submissions go through the `FormData` branch of `handleFormSubmit` (Chunk 4) instead of the plain-JSON one. When editing an existing profile that already has a photo, the current image is shown as a small preview next to the file picker so the admin can see what's already there before deciding whether to replace it.

Chunk 10 17.10 — The Gallery tab: upload-only, no edit form

```
            {/* 5. TABS CONTENT FOR GALLERY */}
            {activeTab === 'gallery' && (
              <div className="space-y-6">
                <div className="flex justify-between items-center">
                  <h2 className="text-xl font-display font-extrabold text-mountain-900">Manage Media Gallery</h2>
                  {!showAddForm && (
                    <button
                      onClick={() => setShowAddForm(true)}
                      className="inline-flex items-center space-x-1 px-4 py-2.5 bg-forest-900 text-forest-300 font-bold rounded-lg hover:bg-
forest-950 text-sm shadow transition-all border border-forest-800"
                    >
                      <Upload className="w-4 h-4" />
                      <span>Upload Item</span>
                    </button>
                  )}
                </div>

                {/* Form Upload */}
                {showAddForm && (
                  <div className="bg-white p-6 rounded-2xl border border-mountain-100 shadow-sm space-y-4">
                    <h3 className="text-lg font-display font-bold text-mountain-900">Upload Image / Flyer</h3>
                    <form
                      onSubmit={(e) => handleFormSubmit(e, 'gallery', 'POST')}
                      className="space-y-4"
                      encType="multipart/form-data"
                    >
                      <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Year</label>
                          <input type="number" name="year" required defaultValue={new Date().getFullYear()} className="w-full border border-
mountain-200 rounded-lg p-2 text-sm" />
                        </div>
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Category / Tag</label>
                          <select name="eventTag" required className="w-full border border-mountain-200 rounded-lg p-2 text-sm bg-white">
                            <option value="Expedition">Expedition</option>
                            <option value="Rock Climbing">Rock Climbing</option>
                            <option value="Course">Course</option>
                            <option value="Competition">Competition</option>
                            <option value="Archive Flyer">Archive Flyer</option>
                          </select>
                        </div>
                        <div>
                          <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Select File</label>
                          <input type="file" name="image" accept="image/*" required className="w-full text-xs text-mountain-500 file:mr-4
file:py-2 file:px-4 file:rounded-lg file:border-0 file:text-xs file:font-semibold file:bg-mountain-100 file:text-mountain-700 hover:file:bg-
mountain-200" />
                        </div>
                      </div>

                      <div>
                        <label className="block text-xs font-bold uppercase text-mountain-500 mb-1">Caption / description (optional)</label>
                        <input type="text" name="caption" className="w-full border border-mountain-200 rounded-lg p-2 text-sm" placeholder="e.g.
Susunia Hill bouldering session during 58th annual course" />
                      </div>

                      <div className="flex justify-end space-x-2 pt-2">
                        <button type="button" onClick={() => setShowAddForm(false)} className="px-4 py-2 border border-mountain-200 rounded-lg
text-sm font-semibold">Cancel</button>
                        <button type="submit" disabled={actionLoading} className="px-5 py-2 bg-mountain-900 text-white rounded-lg text-sm font-
bold shadow hover:bg-mountain-950">
                          {actionLoading ? 'Uploading...' : 'Upload Image'}
                        </button>
                      </div>
                    </form>
                  </div>
                )}

                {/* Grid gallery */}
                <div className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-6 gap-4">
                  {gallery.map(item => (
                    <div key={item._id} className="bg-white rounded-xl overflow-hidden border border-mountain-100 shadow-sm relative group flex
flex-col justify-between">
                      <div className="aspect-square bg-mountain-50 relative overflow-hidden">
                        <img
                          src={item.imageUrl.startsWith('/uploads/') ? `http://localhost:5000${item.imageUrl}` : item.imageUrl}
                          className="w-full h-full object-cover"
                        />
                        {/* Tags */}
                        <span className="absolute top-1.5 left-1.5 px-1.5 py-0.5 bg-mountain-950/80 text-white text-[8px] font-bold rounded">
                          {item.year}
                        </span>

                        <button
                          onClick={() => handleDelete('/gallery', item._id)}
```

```
                        className="absolute bottom-1.5 right-1.5 p-1 bg-white hover:bg-earth-50 rounded shadow border border-mountain-200/55
hover:text-earth-650 transition-colors"
                      >
                        <Trash2 className="w-3.5 h-3.5" />
                      </button>
                    </div>

                    {item.caption && (
                      <div className="p-2 border-t border-mountain-50 text-[10px] text-mountain-600 truncate leading-relaxed">
                        {item.caption}
                      </div>
                    )}
                  </div>
                ))}
              </div>
            )}

          </div>
        )}

      </div>
    </div>
  );
};

export default AdminDashboard;
```

The last tab, and the simplest: a single upload form (year, category tag, image file, optional caption) and a photo grid below it. There's no Edit button on any gallery item — only Delete — matching the backend's `gallery.js` route from Section 8, which likewise never implemented a `PUT` handler. Frontend and backend agree here on purpose: there was never a reason to build an edit form for an endpoint that doesn't exist to receive it.

18. Running Everything Together

With every file explained, here's the practical sequence to get this project running from a freshly cloned/extracted folder:

- **1.** Make sure MongoDB is installed and running locally (or have a cloud MongoDB URL ready).
- **2.** Copy `backend/.env.example` to `backend/.env` and fill in real values (at minimum, `MONGO_URI` and `JWT_SECRET`).
- **3.** From the project root, run `npm run install-all` — this installs the root, backend, and frontend dependencies in one go (Section 2).
- **4.** Run `npm run seed` to populate the database with the starter admin account, secretaries, alumni, events, and gallery items (Section 9).
- **5.** Run `npm run dev` — this starts the Express API on port 5000 *and* the Vite dev server on port 5173 at the same time.
- **6.** Open `http://localhost:5173` in a browser to see the public site, or go straight to `/admin/login` and sign in with `admin@jumhc.org` / `admin1966` (the credentials created by the seed script) to see the dashboard.

Tracing one request through the entire stack

As a last exercise tying every section together, here's exactly what happens, file by file, the moment someone opens the Events page:

- **1.** React Router matches the URL `/events` to `<Events />` (Section 15, `App.jsx`).
- **2.** `Events.jsx`'s `useEffect` runs and calls `api.get('/events')` (Section 16).
- **3.** That goes through the shared helper in `utils/api.js` (Section 12), which attaches headers and sends a real HTTP `fetch` to `http://localhost:5000/api/events`.
- **4.** Express (running from `server.js`, Section 5) matches that URL prefix to `routes/events.js` (Section 8), which runs `router.get('/', …)`.
- **5.** That handler calls `Event.find().sort({ date: 1 })` — Mongoose (using the schema from Section 6) translates that into a real MongoDB query and returns matching documents.
- **6.** The handler sends those documents back as JSON with `res.json(events)`.
- **7.** Back in the browser, `api.get` resolves with that parsed JSON, `Events.jsx` calls `setEvents(data)`, and React re-renders the page — turning raw database documents into the upcoming/past event cards on screen.

Every page in this project, and every action in the admin dashboard, is some variation of this exact same round trip.

19. Appendix: Small Config Files

A handful of files in the zip aren't application logic at all — they're tooling configuration or leftover template defaults. They're listed here briefly for completeness, since the goal of this document is to account for every file in the project, even the ones with nothing conceptually new to teach.

- **frontend/.gitignore** — tells Git which files/folders to never track (like `node_modules`, build output, and editor settings). Standard boilerplate generated by Vite's project scaffolder.
- **frontend/.oxlintrc.json** — configuration for the `oxlint` linter mentioned in Section 10, turning on two React-specific rules (enforcing the Rules of Hooks, and allowing constant exports alongside components).
- **frontend/README.md** — the default readme Vite generates for its React template, describing the official Vite/React plugins. Not customized for this project specifically.
- **frontend/public/favicon.svg** and **frontend/public/icons.svg** — static image assets served as-is (anything in Vite's `public/` folder is copied to the final build untouched, rather than being processed/bundled like the files in `src/`).
- **frontend/src/assets/hero.png, react.svg, vite.svg** — image assets bundled from inside `src/` (react.svg and vite.svg are, again, unused leftovers from the default template, in the same spirit as `App.css` from Section 11).

> **THAT'S THE WHOLE PROJECT**
>
> Every file in JUMHC-Portal.zip has now been shown and explained, in the order you'd realistically write them: database schemas first, then the API routes that use them, then the React pages that call that API, finishing with the admin dashboard that ties every route together into one management screen.